

KAMP 제조AI데이터셋 활용 권한

No. GB20260921-PU1789564111-364766



⚠ 제조AI데이터셋 활용 주의사항

연구/공식적으로 사용하실 때에는 꼭 아래와 같이 'KAMP 출처(reference)'를 남겨주시고, 인용 시 활용한 내용과 문서 등은 아래 이메일로 보내주시기를 바랍니다.
(E-mail : kamp@kaist.ac.kr)

국문 출처 표기 양식

중소벤처기업부, Korea AI Manufacturing Platform(KAMP), 전자부품(배터리팩) 품질보증 AI 데이터셋, 스마트제조혁신추진단((주)인터엑스, 네스트필드(주)), 2022.12.23., www.kamp-ai.kr

영문 출처 표기 양식

Ministry of SMEs and Startups., and KAIST (Korea Advanced Institute of Science and Technology). (2022, December 23). Electronic components(battery pack) quality assurance AI dataset. Korea AI Manufacturing Platform(KAMP). <https://www.kamp-ai.kr/>

제조AI데이터셋 개요

제조AI데이터셋명 : 전자부품(배터리팩) 품질보증 AI 데이터셋

업종 : 전자부품 | **목적** : 품질보증 | **유형** : csv

알고리즘 : MTadGAN (Multivariate Time Series Anomaly Detection by Generative Adversarial Network)

사용조건 : 콘텐츠 변경허용

제공기관 : 스마트제조혁신추진단 (수행기관 : (주)인터엑스/네스트필드(주))

등록일 : 2022-12-23

이용자 정보

회원 ID : MSS_zas357

다운로드일 : 2026-09-21 14:45:43

활용목적 : 제조AI 솔루션 개발 및 고도화

인공지능 제조 플랫폼(KAMP, Korea AI Manufacturing Platform)에서 다운로드한 제조AI데이터셋의 저작권을 위하여 문서 고유 번호와 워터마크를 부여하여, 해당 문서의 표준을 준수하였음을 증명합니다.

운영기관 : KAIST 제조AI빅데이터센터

주소 : 대전광역시 유성구 문지로 193 KAIST 문지캠퍼스 행정동

전화번호 : 042-350-1331 | 이메일 : kamp@kaist.ac.kr

「전자부품(배터리팩) 품질보증 AI 데이터셋」

분석실습 가이드북



「전자부품(배터리팩) 품질보증 AI 데이터셋」

분석실습 가이드북





Contents

1 분석요약	04
2 분석 실습	
1. 분석 개요	05
1.1 분석 배경	05
1) 공정(설비) 개요	
2) 이슈사항(Pain point)	
1.2 분석 목표	09
1) 분석 목표	
2) 데이터 정의 및 소개	
3) 데이터 분석 기대효과	
4) 시사점(implication) 요약기술	
2. 분석실습	10
2.1 제조데이터 소개	10
1) 데이터 수집 방법	
2) 데이터 유형/구조	
3) 데이터 (품질) 전처리	
2.2 분석 모델 소개	28
1) 데이터 흐름 및 인공지능 모델 적용 흐름도	
2) AI 분석모델	
2.3 분석 체험	34
1) 필요 SW, 패키지 설치 방법 및 절차 가이드	
2) 분석 단계별 프로세스 - Flow Chart	
[단계 ①] 데이터 준비	
[단계 ②] 데이터 구조 및 특징 탐색	
[단계 ③] 학습, 테스트 데이터 선택	
[단계 ④] AI모델 알고리즘 선택	
[단계 ⑤] AI모델 학습	
[단계 ⑥] 테스트 데이터에 대한 계산 수행	
[단계 ⑦] 테스트 셋 결과 분석	
[단계 ⑧] 알고리즘 평가	
3. 유사 타 현장의 「전자부품(배터리팩) 품질보증 AI 데이터셋」 분석 적용	103
부 록	
분석환경 구축을 위한 설치 가이드	105
데이터 품질 전처리 [실습 코드]	114
부록_AAS S/W 설치 가이드	121



「전자부품(배터리팩) 품질보증 AI 데이터셋」

분석실습 가이드북

- **필요 SW** : Python, Anaconda, Jupyter Notebook
- **필요 패키지** : matplotlib, pandas, numpy, plotly, mpl_toolkits, sklearn, random, datetime, math,
- **분석 환경** : os, sys, argparse, collections, tensorflow, logging, pydot, pydotplus, scipy, pyts
[CPU] AMD Ryzen 9 5950X, [Memory] 128GB, [GPU] NVIDIA GeForce RTX 3090
- **필요 데이터** : 10XX_chg.csv, 10XX_dchg.csv, Test0X_OK_chg.csv, Test0X_NG_dchg.csv,
Test0X_OK_chg_label.csv, Test0X_NG_dchg_label.csv

1 분석요약

No	구분		내용
1	분석 목적 (현장 이슈, 목적)		배터리팩 충·방전 시험 데이터를 AI 모델로 학습하여 데이터 생산 근로자 역량 향상 및 품질관리를 강화하고 납품된 배터리팩의 라이프사이클 관리를 위한 품질 보증 체계를 개발한다.
2	데이터셋 형태 및 수집방법		1) 분석에 사용된 변수 : M01CV01 ~ M16CV11(176개), M01T01 ~ M16T02(32개) 2) 데이터 수집 방법 : AAS 표준기반 제조데이터 수집/저장 체계를 통해 배터리팩 충·방전 시험 설비 공정 데이터 추출 3) 데이터셋 파일 확장자 : CSV
3	데이터 개수 데이터셋 총량		- 데이터 개수 : 116,939,945개 - 데이터셋 총량 : 630 MB
4	분석적용 알고리즘	알고리즘	MTadGAN (Multivariate Time Series Anomaly Detection by Generative Adversarial Network)
		알고리즘 간략소개	MTadGAN 알고리즘은 다변량 시계열에 대한 이상 탐지를 위해 적대적 생성자 네트워크(GAN)를 적용하고 효율적인 학습을 위해 Wasserstein 손실함수를 적용하는 알고리즘이다. 시계열 입력 데이터의 공간 X에서 잠재 공간 Z로 보내는 부호화 생성자 E와 역으로 잠재공간에서 가짜 샘플을 만드는 생성자 G를 네트워크로 구성하고 이들 사이에 두 개의 적대적인 판별자 네트워크를 도입하여 최대한 가짜와 실제 데이터를 구별할 수 있도록 하고 더하여 순환-일관성이 있도록 학습하며 이를 이용해서 구한 이상 점수를 기반으로 이상 신호를 탐지하는 알고리즘이다.
5	분석결과 및 시사점		- 배터리팩의 양품과 불량률 예측하기 위해 MTadGAN (Multivariate Time Series Anomaly Detection by Generative Adversarial Network)을 활용하여 AI 모델을 개발하였다. - 배터리팩 충·방전 시험 설비로부터 매초마다 수집되는 공정 데이터를 기반으로 AI 모델을 생성하여 시스템 상에서 제품의 품질예측과 불량 발생 원인을 자동으로 판단할 수 있도록 지원한다. - 향후 납품된 배터리팩의 라이프사이클 관리에도 활용될 수 있으므로 생산 제품의 품질 향상과 함께 지속관리가 필요한 현장에 본 분석이 활용될 수 있을 것으로 기대된다.

2 분석 실습

1. 분석 개요

1.1 분석 배경

1) 공정(설비) 개요

• 공정(설비) 정의 및 특징

- 전기버스 및 전기상용차에 사용되는 배터리팩을 공급하기 전 배터리팩 조립공정에서 출고된 배터리팩의 양/불 여부를 판정하기 위해 충·방전 시험하는 공정이다.



[그림 1] 배터리팩 제조공정

- 배터리팩 충·방전 시험: 배터리팩 전용 충·방전 시험 설비를 사용하여 양/불 여부를 판정한다. 엔지니어는 배터리팩 전압, 전류, 온도, 전력, PCS 상태, 충·방전에너지 등의 각종 상태 데이터를 분석하여 최종 판정을 내리고 조립 및 납품을 지시한다.



[그림 2] 배터리팩 충·방전 시험 설비 및 환경

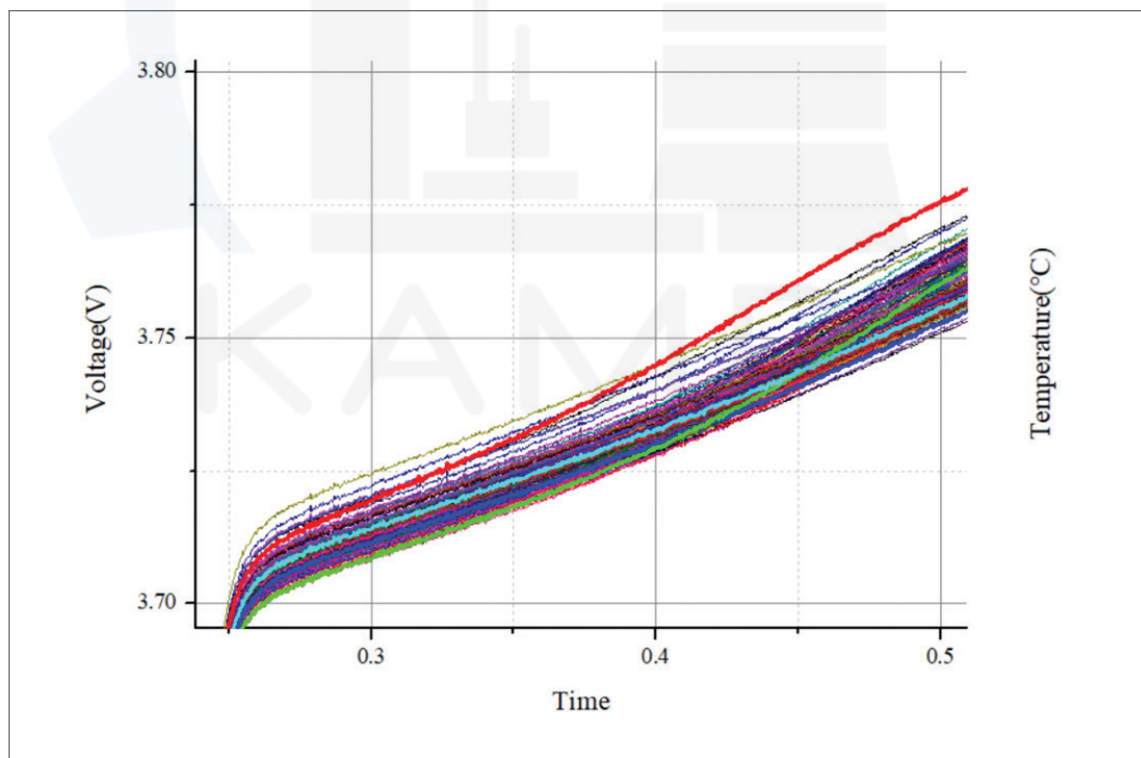
2) 이슈사항(Pain point)

• 공정(설비)상의 문제 현황

- 배터리팩 충·방전 시험은 자동화 설비가 없이 수작업으로 진행되고 있으며, 작업자 역량에 의존하고 있어 작업관리 및 품질관리에 어려움이 있다. 특히, 배터리팩은 다수의 배터리모듈과 배터리모듈을 구성하는 배터리셀들의 집합체이기 때문에 충·방전 시험 중 불량 발생하면 불량원인을 분석하고 불량품을 교체하는 과정을 거쳐야 한다. 그러나 불량원인을 분석하는 과정에서 경계성 판단이 필요한 경우가 발생하기 때문에 숙련된 핵심 기술자의 역량에서 대처 가능한 상황이다.

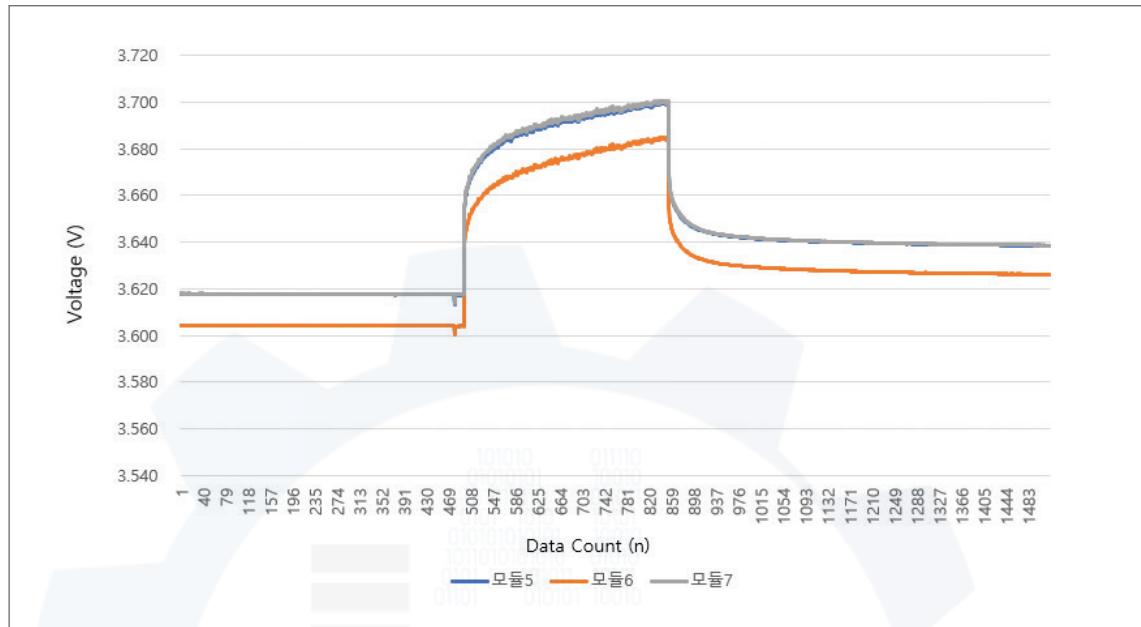
• 불량현상 및 원인

- 용량불량: 배터리팩은 수많은 배터리 모듈과 배터리셀들로 구성되기 때문에 테스트 과정에서 불량 배터리셀이 발견되면 모듈 단위로 교체를 해야 하는 경우가 많다. 용량 불량인 경우 충·방전 시험 과정에서 특정 배터리셀에서 급격하게 전압이 상승/하강하는 구간이 발견된다는 특징을 가진다. 충전의 경우 초기에 비슷하게 출발하나 충전이 완료된 후 비교해 보면 그 차이가 뚜렷하다. 아래 그림에서 붉은색과 녹색의 배터리 셀이 이 경우에 해당하지만 배터리모듈 마다 평균 전압 차이도 고려해야 하기 때문에 불량을 판정하는 기준에 경계성 판단이 요구되므로 전문가 검토가 필수적이다.



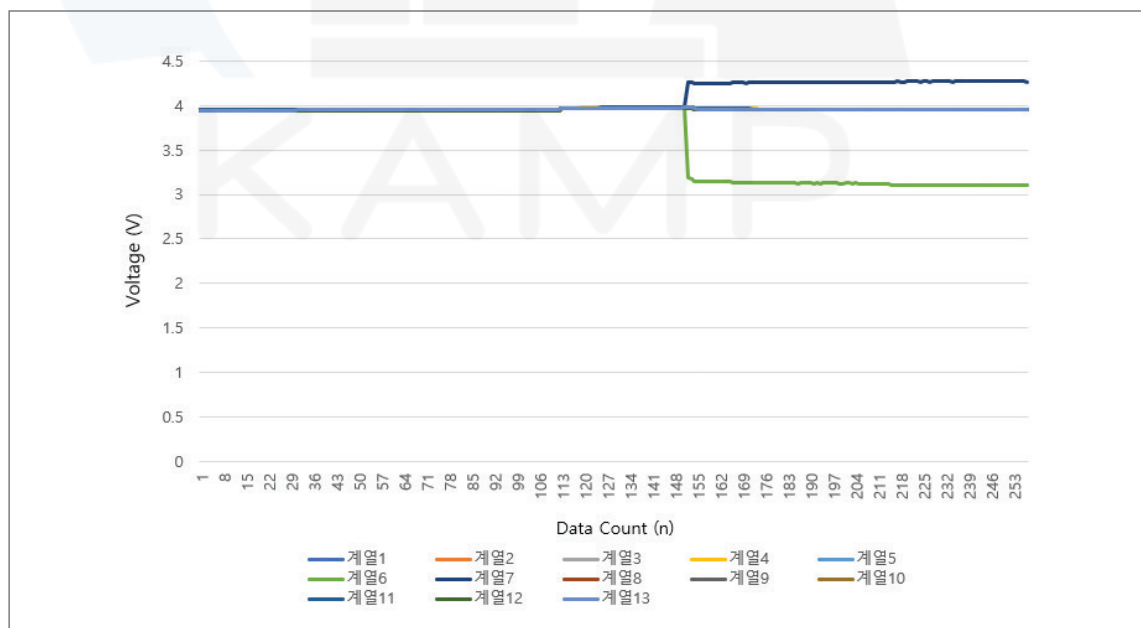
[그림 3] 배터리셀 용량불량

- 용접불량: 입고 검사 과정에서 통과된 배터리셀들이 배터리모듈의 형태로 조립되어 용접이 진행된다. 용접한 부위는 외관상 문제가 없어 보이더라도 특정 배터리셀에서 전압이 측정되지 않거나 배터리셀 전체 전압이 떨어져 있는 현상이 나타날 수 있는데, 이 경우 용접불량을 의심해봐야 한다.



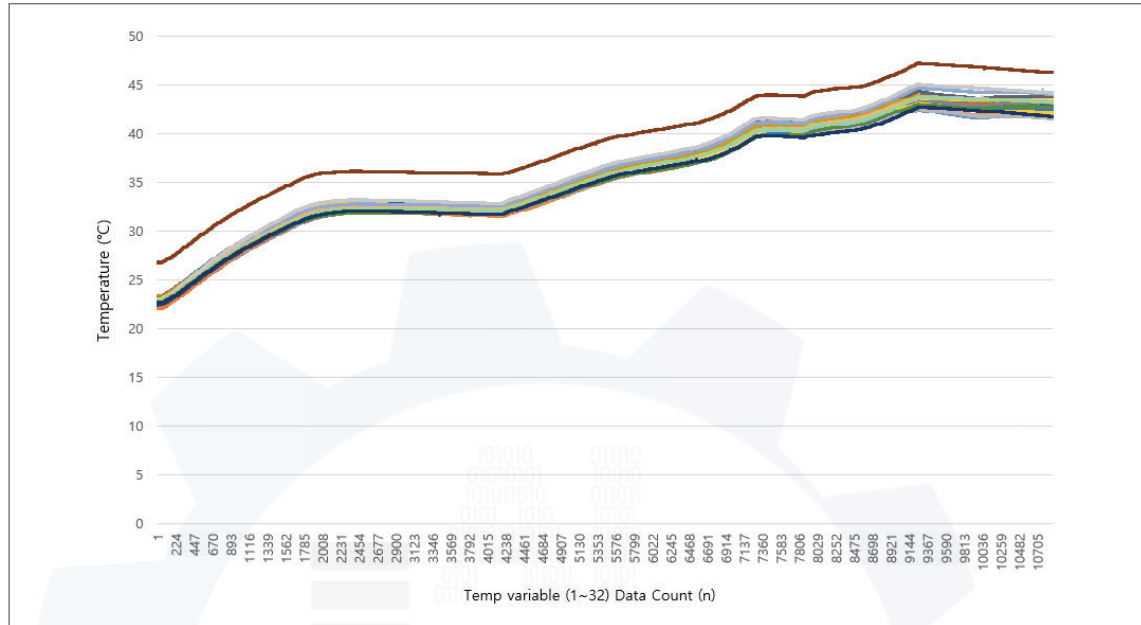
[그림 4] 배터리모듈 용접불량

- 센싱와이어불량: 배터리모듈을 구성하는 배터리셀들의 온도나 전압을 측정하기 위해 센싱와이어로 서로 연결되어있다. 와이어에서 불량 발생 하는 경우 인접한 배터리셀에 영향을 미치므로 인접한 배터리셀들의 전압에 차이가 발생하게 된다.



[그림 5] 배터리모듈 센싱와이어불량

- 센서불량: 배터리모듈에 장착된 온도센서의 측정값이 너무 높거나 낮게 출력되는 경우 센서불량을 의심해야 한다. 일반적으로 센서를 교체하면 해당 문제를 해결할 수 있으나 간혹 배터리모듈에 장착된 BMS의 파라미터 설정이 잘못되어 값이 이상하게 측정될 수 있으므로 양측 점검이 필요하다.



[그림 6] 배터리모듈 센서불량

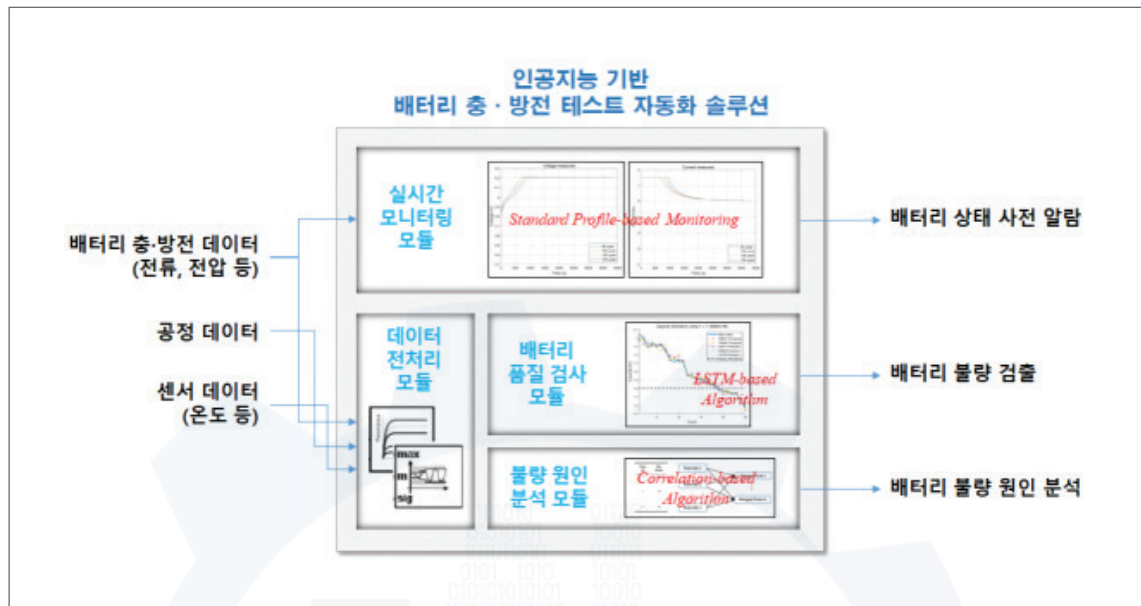
• 문제해결 장애요인

- 현재 제조회장에서 사용되고 있는 배터리팩 충·방전 설비는 조립된 배터리팩의 품질을 시험하기 위해 사용되고 있다. 그러나 충·방전 시험 시 상태 변동에 따른 관리를 수작업으로 진행하다 보니 작업관리나 품질관리에 어려움이 있다.
- 또한, 시험 결과를 분석하는 과정에서 뚜렷한 Pass/Fail 대응은 일반 작업자들도 가능하나, 경계성 판단이 필요한 부분은 숙련된 핵심 기술자의 역량 수준에서 판단 가능한 상황이다.

• 극복 방안

- 배터리팩 충·방전 시험 데이터에 대한 시계열 분석 기법(예: Multivariate Time Series Anomaly Detection by Generative Adversarial Network, MTadGAN)의 적용을 통해 배터리의 품질 수준을 예측하고, 불량 데이터 검출 시 알람을 발생하여 작업자가 이를 파악하고 조치할 수 있도록 실시간 모니터링 기능을 제공할 수 있는 기반을 마련하고자 한다.

- AI 분석가는 현장 작업자와 지속적으로 의사소통하여 충전과정에서의 불량 데이터 패턴과 방전과정에서의 불량 데이터 패턴을 분류하여 데이터셋을 구성하고 검증을 진행한다.



[그림 7] 배터리팩 품질보증을 위한 불량 분석 개념도

1.2 분석 목표

1) 분석 목표

- 배터리팩 충·방전 시험 데이터를 AI 모델로 학습하여 데이터 생산 근로자 역량 향상 및 품질관리를 강화하고, 납품된 배터리팩의 라이프사이클 관리를 위한 품질 보증 체계를 개발한다.

2) 데이터 정의 및 소개

- 본 분석에서 배터리팩 충·방전 시험 데이터를 AAS 표준기반 제조데이터 수집/저장 체계로 수집하기 위해 별도의 데이터 수집 PC를 설치하여 OPC UA 통신 가능하도록 인터페이스를 개발하고, 이를 기반으로 제품의 품질관리 및 지속적인 라이프사이클 관리가 가능한 기계학습 데이터셋을 구성한다.

3) 데이터 분석 기대효과

- 배터리팩 충·방전 시험 설비공정을 통해 수집한 데이터를 기반으로 제품의 품질을 예측하여 품질관리를 강화하고 불량의 발생 원인을 분석하여 생산성을 높일 수 있다. 또한, 본 분석에서는 AAS 표준기반 제조데이터 수집/저장 체계를 적용하여 납품된 배터리팩의 라이프사이클 관리에도 활용될 수 있으므로 지속관리가 필요한 현장에서 본 분석이 폭넓게 활용될 수 있을 것으로 기대된다.



4) 시사점(implication) 요약기술

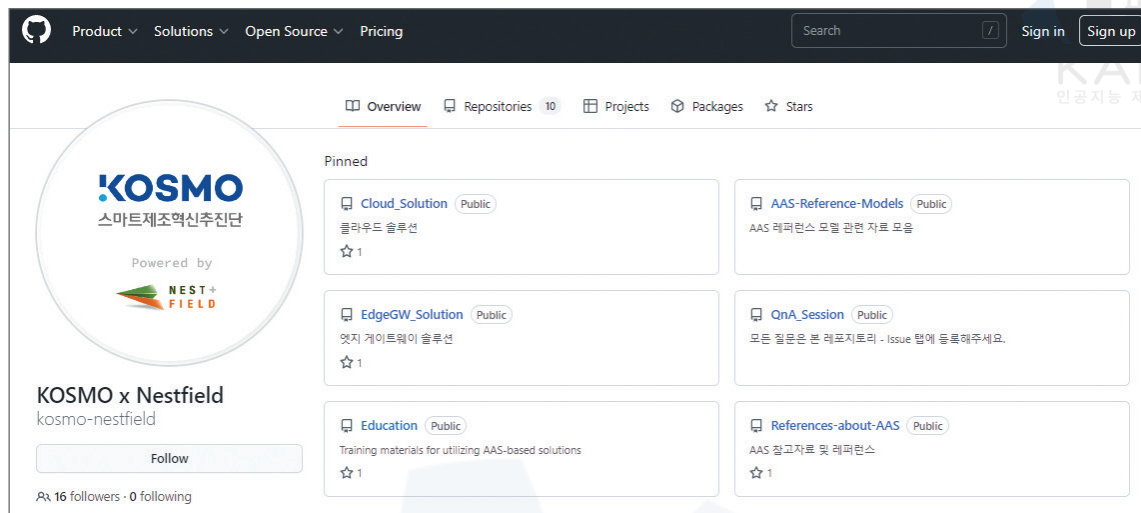
- 현재 배터리팩은 충·방전 시험 설비를 통해 측정된 데이터를 기준으로 양품과 불량품이 선별되고 있지만, 대부분은 작업자의 노하우에 의존하여 선별이 진행되고 있어 품질을 보장하기 어려운 상황이다.
- 실제 불량 사례가 있음에도 불구하고 문제가 발생한 시점과 원인을 정확하게 파악하기 어려워 생산 지연으로 이어지는 경우가 빈번하게 나타나고 있다.
- 본 분석에서는 배터리팩 충·방전 시험 설비로부터 매초마다 수집되는 공정 데이터를 기반으로 AI 모델을 생성하여 시스템 상에서 제품의 품질예측과 불량이 발생한 원인을 자동으로 판단할 수 있도록 지원한다.
- 또한, AAS 표준기반 제조데이터 수집/저장 체계를 통해 향후 납품된 배터리팩의 라이프사이클 관리에도 활용될 수 있으므로 생산 제품의 품질 향상과 함께 지속관리가 필요한 현장에 본 분석이 활용될 수 있다는 점에서 시사점이 크다고 판단된다.

2. 분석실습

2.1 제조데이터 소개

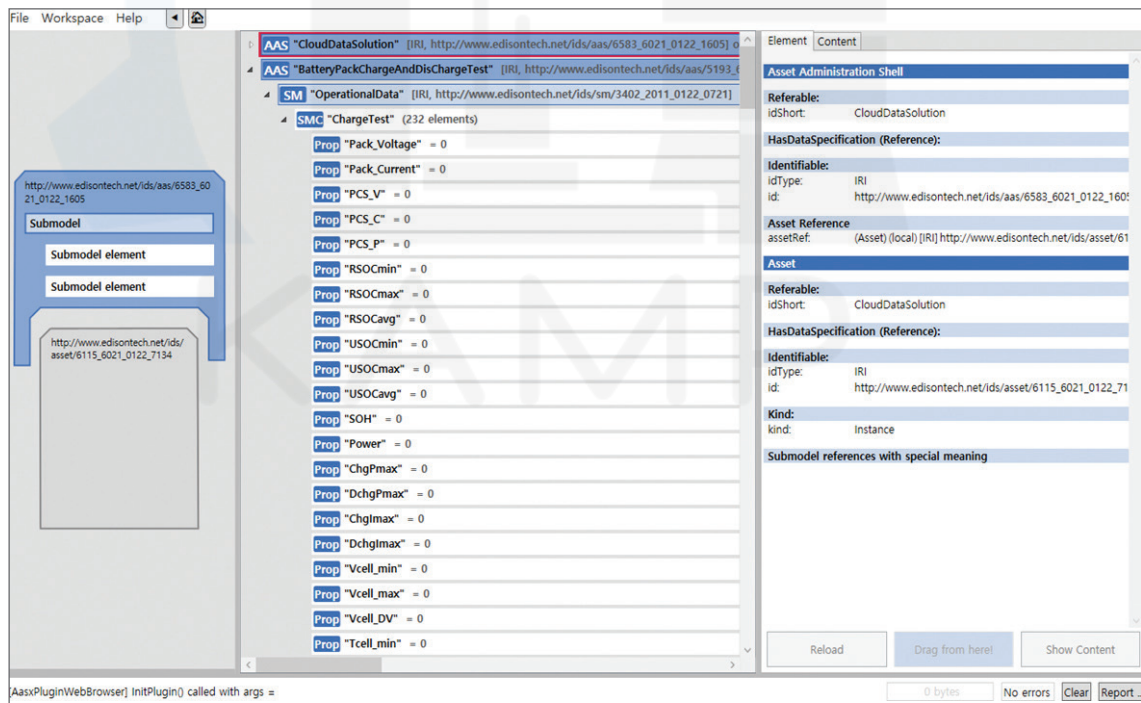
1) 데이터 수집 방법

- **제조(설비) 분야:** 전기차용 배터리팩 제조
- **제조 공정명:** 배터리팩 충·방전 시험 공정
- **수집 장비:** AAS (Asset Administration Shell) 서버
 - AAS 표준기반 제조데이터 수집/저장 체계를 통해 AI 학습용 데이터 확보
- **수집 기간:** 2022. 07. 01 ~ 2022. 09. 30
- **수집 주기:** 1 sec
- **수집 방법:**
 - 본 분석에서는 데이터 수집을 위해 AAS 표준기반 제조데이터 수집/저장 체계가 준비된 상태를 가정한다. 단, 본 데이터 수집/저장 체계 구축을 위한 AAS 모델링 방법 및 소프트웨어 설치 가이드는 스마트제조혁신추진단과 네스트필드(주)에서 제공하는 오픈 소스 제공 사이트인 깃허브(<https://github.com/kosmo-nestfield>)에서 교육자료를 제공하고 있으므로 본 가이드북에서는 해당 내용을 다루지 않는다.



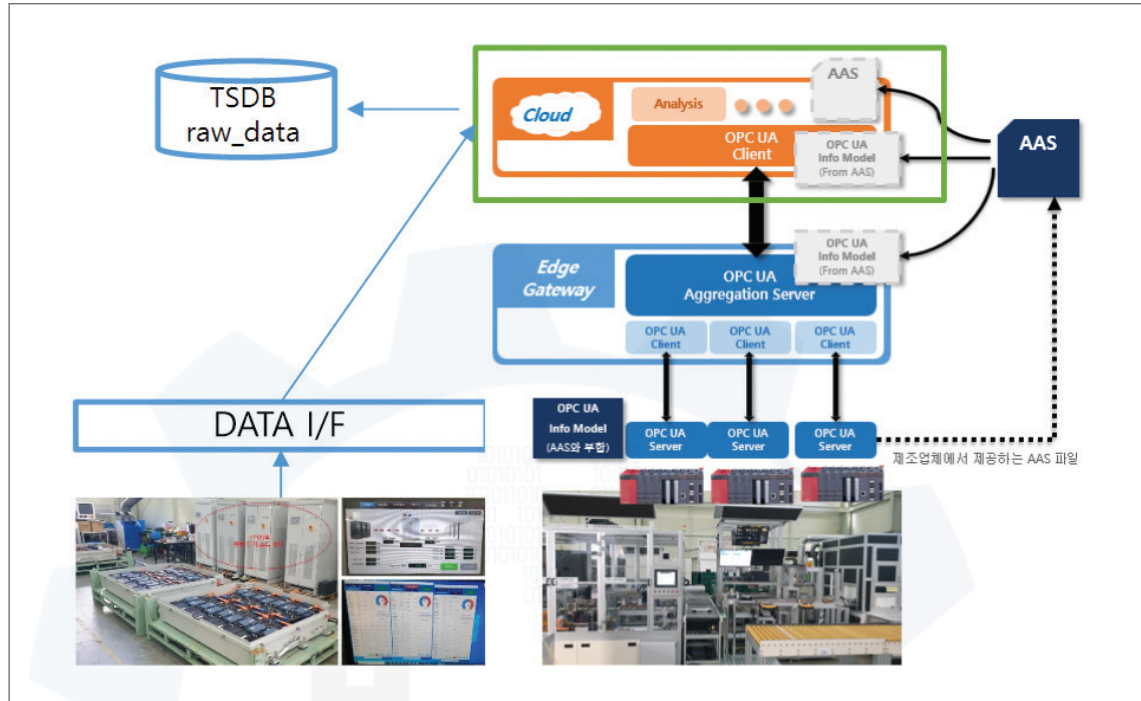
[그림 8] AAS 표준기반 제조데이터 수집/저장 체계 구축 가이드

- 현장 설비공정에 대한 정보 모델은 IEC 63278-1 AAS (Asset Administration Shell) 기술을 통해 모델링하고 이를 기반으로 AAS 표준기반 제조데이터 수집/저장 체계에 사용될 수 있는 형태로 변환되어 사용된다. 본 분석에 사용된 배터리팩 충·방전 시험 설비공정 AAS 파일은 오픈 소스 소프트웨어인 ‘AASX Package Explorer’를 통해 뷰/편집이 가능하다. AASX Package Explorer 설치는 ‘부록(AAS S/W 설치 가이드)’를 참고한다.



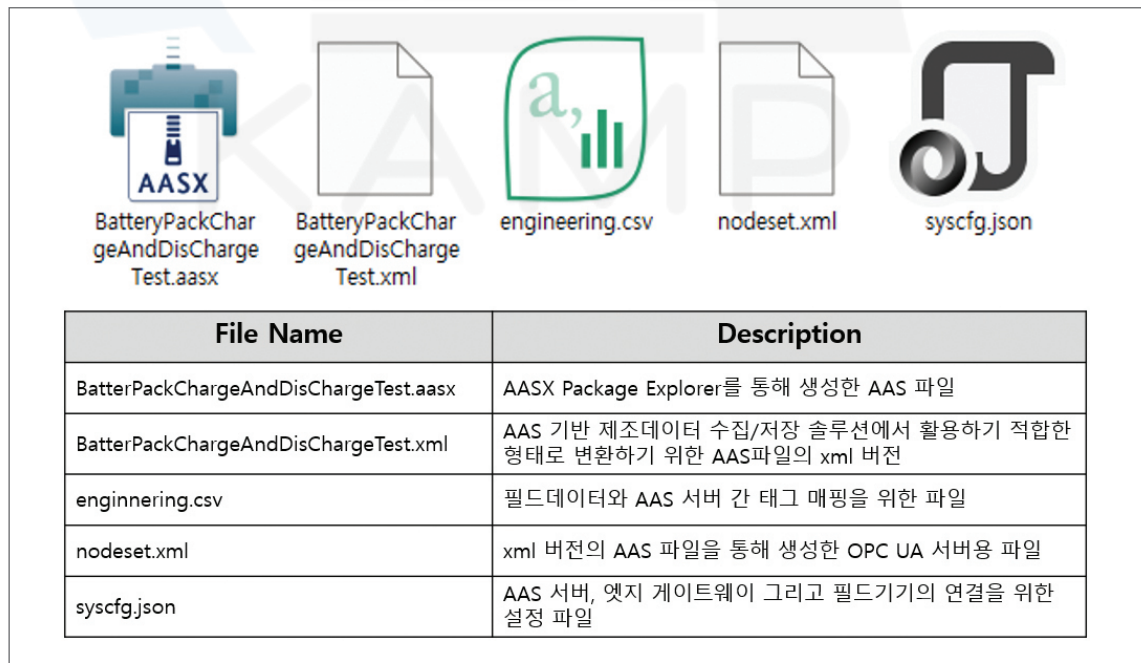
[그림 9] 배터리팩 충·방전 시험 설비공정 AAS 모델

- 현장 데이터는 설비공정에서 운용중인 PLC를 통해 엣지 게이트웨이로 수집하여 AAS 서버에서 TSDB에 저장하는 방법과 외부 서버 PC와 AAS 서버를 직접 연결하여 수집/저장하는 방법이 있다. 본 분석에서는 이러한 방식을 사용하여 RAW DATA를 수집하여 테스트/분석용 데이터를 확보하였다.



[그림 10] AAS 표준기반 제조데이터 수집/저장 체계 구성도

- 충·방전 시험기를 통해 수집하는 데이터는 AAS 파일 변환을 통해 생성되는 파일 중 하나인 'engineering.csv'에 태그형태로 매핑되어야 한다.



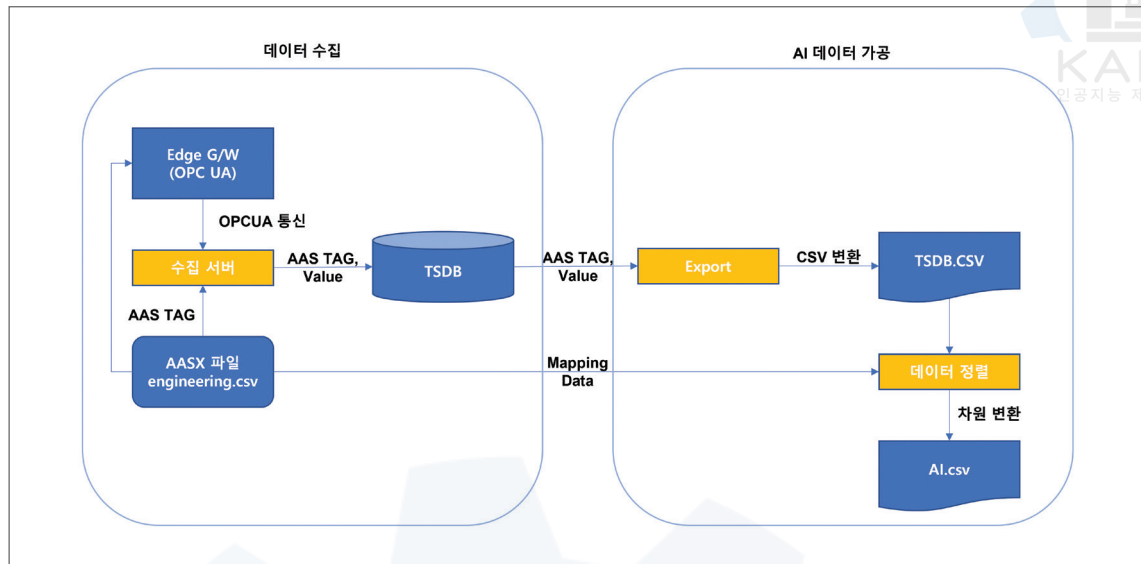
[그림 11] AAS 표준기반 제조데이터 수집/저장 체계 적용을 위한 AAS 파일 및 변환 파일

- engineering.csv 파일에서 A열은 AAS 모델을 통해 생성된 태그명, B열은 엣지 게이트웨이명, C열은 현장 데이터를 수집하는 필드장비명 D열은 현장에서 사용하는 OPC UA 태그명, E열은 ms 단위의 Sampling Interval, F열은 배열 정보 그리고 G열은 실제 필드에서 사용 중인 변수명이 들어간다. 단, 아래 그림과 같이 외부 공개하기 어려운 부분은 공백으로 표기하였다.

	A	B	C	D	E	F	G	H
30	ns=2;s=BatteryPackChargeAndDisChargeTest.OperationalData.ChargeTest.M01CV01				100	-1	M01CV01	
31	ns=2;s=BatteryPackChargeAndDisChargeTest.OperationalData.ChargeTest.M01CV02				100	-1	M01CV02	
32	ns=2;s=BatteryPackChargeAndDisChargeTest.OperationalData.ChargeTest.M01CV03				100	-1	M01CV03	
33	ns=2;s=BatteryPackChargeAndDisChargeTest.OperationalData.ChargeTest.M01CV04				100	-1	M01CV04	
34	ns=2;s=BatteryPackChargeAndDisChargeTest.OperationalData.ChargeTest.M01CV05				100	-1	M01CV05	
35	ns=2;s=BatteryPackChargeAndDisChargeTest.OperationalData.ChargeTest.M01CV06				100	-1	M01CV06	
36	ns=2;s=BatteryPackChargeAndDisChargeTest.OperationalData.ChargeTest.M01CV07				100	-1	M01CV07	
37	ns=2;s=BatteryPackChargeAndDisChargeTest.OperationalData.ChargeTest.M01CV08				100	-1	M01CV08	
38	ns=2;s=BatteryPackChargeAndDisChargeTest.OperationalData.ChargeTest.M01CV09				100	-1	M01CV09	
39	ns=2;s=BatteryPackChargeAndDisChargeTest.OperationalData.ChargeTest.M01CV10				100	-1	M01CV10	
40	ns=2;s=BatteryPackChargeAndDisChargeTest.OperationalData.ChargeTest.M01CV11				100	-1	M01CV11	
41	ns=2;s=BatteryPackChargeAndDisChargeTest.OperationalData.ChargeTest.M02CV01				100	-1	M02CV01	
42	ns=2;s=BatteryPackChargeAndDisChargeTest.OperationalData.ChargeTest.M02CV02				100	-1	M02CV02	
43	ns=2;s=BatteryPackChargeAndDisChargeTest.OperationalData.ChargeTest.M02CV03				100	-1	M02CV03	
44	ns=2;s=BatteryPackChargeAndDisChargeTest.OperationalData.ChargeTest.M02CV04				100	-1	M02CV04	
45	ns=2;s=BatteryPackChargeAndDisChargeTest.OperationalData.ChargeTest.M02CV05				100	-1	M02CV05	
46	ns=2;s=BatteryPackChargeAndDisChargeTest.OperationalData.ChargeTest.M02CV06				100	-1	M02CV06	
47	ns=2;s=BatteryPackChargeAndDisChargeTest.OperationalData.ChargeTest.M02CV07				100	-1	M02CV07	
48	ns=2;s=BatteryPackChargeAndDisChargeTest.OperationalData.ChargeTest.M02CV08				100	-1	M02CV08	
49	ns=2;s=BatteryPackChargeAndDisChargeTest.OperationalData.ChargeTest.M02CV09				100	-1	M02CV09	

[그림 12] AAS 변환 파일 중 engineering.csv 파일 구성

- 실제 설비공정에서 발생하는 데이터들은 의미 없는 값이나 누락이 발생할 수 있어 데이터 품질이 떨어지며, 이는 AAS 표준기반 제조데이터 수집/저장 체계에서도 영향을 받기 때문에 수집된 데이터만으로는 AI 분석에 적합하지 않다. 따라서, 데이터 가공 및 품질 전처리하는 데이터 분석에서 필수적인 단계이다.
- 본 분석에서 AAS 표준기반 제조데이터 수집/저장 체계는 작성한 AASX 파일을 기반으로 Edge Gateway에서 OPC UA 통신을 통해 수집 서버(AAS 서버)로 데이터를 전송한다. 수집 서버는 수신한 데이터에 대하여 AAS TAG에 맞추어 값을 TSDB에 저장하게 된다.



[그림 13] 데이터 수집 및 가공 구성도

- 수집된 데이터는 아래와 같이 명령어를 사용하여 TSDB로부터 CSV 파일을 추출한다.

```
curl --request -G http://localhost:8086/query?orgID=INFLUX_ORG_ID&database=MyDB&retention_policy=MyRP \
--header 'Authorization: Token INFLUX_TOKEN' \
--header 'Accept: application/csv' \
--header 'Content-type: application/json' \
--data-urlencode "q=SELECT used_percent FROM example-db.example-rp.example-measurement WHERE host=host1"
```

[그림 14] 수집 데이터에 대한 CSV 파일 추출 예시

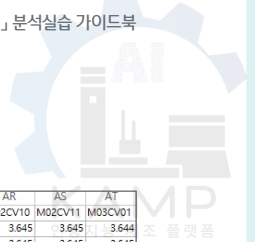
```
result,0,1970-01-01T00:00:00Z,2022-09-26T08:58:15,4498722672,2021-10-27T02:37:46.8472,0,ns=2;sb=BT_Pack_Assembly_System.Component.CCH_Unloader.OperationalData.PLC_IO_19.Fall,mos,device
result,0,1970-01-01T00:00:00Z,2022-09-26T08:58:15,4498722672,2021-10-27T02:37:51.1234982,0,ns=2;sb=BT_Pack_Assembly_System.Component.CCH_Unloader.OperationalData.PLC_IO_19.Fall,mos,device
result,0,1970-01-01T00:00:00Z,2022-09-26T08:58:15,4498722672,2021-11-02T06:06:29.8567752,0,ns=2;sb=BT_Pack_Assembly_System.Component.CCH_Unloader.OperationalData.PLC_IO_19.Fall,mos,device
result,0,1970-01-01T00:00:00Z,2022-09-26T08:58:15,4498722672,2021-11-02T18:37:07.2125452,0,ns=2;sb=BT_Pack_Assembly_System.Component.CCH_Unloader.OperationalData.PLC_IO_19.Fall,mos,device
result,0,1970-01-01T00:00:00Z,2022-09-26T08:58:15,4498722672,2021-11-03T07:28:00.5011492,0,ns=2;sb=BT_Pack_Assembly_System.Component.CCH_Unloader.OperationalData.PLC_IO_19.Fall,mos,device
result,0,1970-01-01T00:00:00Z,2022-09-26T08:58:15,4498722672,2021-11-03T09:56:06.569632,0,ns=2;sb=BT_Pack_Assembly_System.Component.CCH_Unloader.OperationalData.PLC_IO_19.Fall,mos,device
result,0,1970-01-01T00:00:00Z,2022-09-26T08:58:15,4498722672,2021-11-10T01:08:22.2183082,0,ns=2;sb=BT_Pack_Assembly_System.Component.CCH_Unloader.OperationalData.PLC_IO_19.Fall,mos,device
result,0,1970-01-01T00:00:00Z,2022-09-26T08:58:15,4498722672,2021-11-10T04:57:51.7041242,0,ns=2;sb=BT_Pack_Assembly_System.Component.CCH_Unloader.OperationalData.PLC_IO_19.Fall,mos,device
result,0,1970-01-01T00:00:00Z,2022-09-26T08:58:15,4498722672,2021-11-19T07:23:17.0771632,0,ns=2;sb=BT_Pack_Assembly_System.Component.CCH_Unloader.OperationalData.PLC_IO_19.Fall,mos,device
result,0,1970-01-01T00:00:00Z,2022-09-26T08:58:15,4498722672,2021-11-19T07:35:03.4622442,0,ns=2;sb=BT_Pack_Assembly_System.Component.CCH_Unloader.OperationalData.PLC_IO_19.Fall,mos,device
```

[그림 15] 추출한 데이터 예시

- 추출한 CSV 파일을 engineering.csv 파일 내 AAS 태그와 매핑된 변수명들을 매칭하여 데이터를 정렬한다. 이렇게 데이터가 모두 정렬되면 아래와 같이 데이터셋 생성을 위한 Raw 데이터를 획득할 수 있다.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	
1	Date	Time	SerialNum	Voltage	Current	RSOCmin	RSOCmax	RSOCavg	USOCmin	USOCmax	USOCavg	SOH	Power	ChgPmax	DchgPmax	ChgImax	DchgImax	Vmin	Vmax	DV	Tmin	Tmax
2	2021-05-27	16:12:44	166	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
3	2021-05-27	16:12:45	166	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
4	2021-05-27	16:12:46	166	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
5	2021-05-27	16:12:47	166	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
6	2021-05-27	16:12:48	166	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
7	2021-05-27	16:12:49	166	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

[그림 16] 데이터셋 생성을 위한 Raw 데이터 획득



2) 데이터 유형/구조

	X	Y	Z	AA	AB	AC	AD	AE	AF	AG	AH	AI	AJ	AK	AL	AM	AN	AO	AP	AQ	AR	AS	AT
1	M01CV01	M01CV02	M01CV03	M01CV04	M01CV05	M01CV06	M01CV07	M01CV08	M01CV09	M01CV10	M01CV11	M02CV01	M02CV02	M02CV03	M02CV04	M02CV05	M02CV06	M02CV07	M02CV08	M02CV09	M02CV10	M02CV11	M03CV01
2	3.646	3.645	3.645	3.645	3.645	3.644	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.644	3.644	3.645	3.645	3.644
3	3.646	3.645	3.645	3.645	3.645	3.644	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.644	3.644	3.645	3.645	3.645
4	3.646	3.645	3.645	3.645	3.645	3.644	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.644	3.644	3.645	3.645	3.645
5	3.646	3.645	3.645	3.645	3.645	3.644	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.644	3.644	3.645	3.645	3.645
6	3.646	3.645	3.645	3.645	3.645	3.644	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.644	3.644	3.645	3.645	3.644
7	3.646	3.645	3.645	3.645	3.645	3.644	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.644	3.644	3.645	3.645	3.644
8	3.646	3.645	3.645	3.645	3.645	3.644	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.644	3.644	3.645	3.645	3.644
9	3.646	3.645	3.645	3.645	3.645	3.644	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.644	3.644	3.645	3.645	3.644
10	3.646	3.645	3.645	3.645	3.645	3.644	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.644	3.644	3.645	3.645	3.644
11	3.646	3.645	3.645	3.645	3.645	3.644	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.644	3.644	3.645	3.645	3.644
12	3.646	3.645	3.645	3.645	3.645	3.644	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.644	3.644	3.645	3.645	3.644
13	3.646	3.645	3.645	3.645	3.645	3.644	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.644	3.644	3.645	3.645	3.644
14	3.646	3.645	3.645	3.645	3.645	3.644	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.644	3.644	3.645	3.645	3.644
15	3.645	3.645	3.645	3.645	3.645	3.644	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.644	3.644	3.645	3.645	3.644
16	3.646	3.645	3.645	3.645	3.645	3.644	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.644	3.644	3.645	3.645	3.644
17	3.646	3.645	3.645	3.645	3.645	3.644	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.644	3.644	3.645	3.645	3.644
18	3.646	3.645	3.645	3.645	3.645	3.644	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.644	3.644	3.645	3.645	3.644
19	3.645	3.645	3.645	3.645	3.645	3.644	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.644	3.644	3.645	3.645	3.644
20	3.646	3.645	3.645	3.645	3.645	3.644	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.644	3.644	3.645	3.645	3.644
21	3.646	3.645	3.645	3.645	3.645	3.644	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.644	3.644	3.645	3.645	3.644
22	3.646	3.645	3.645	3.645	3.645	3.644	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.644	3.644	3.645	3.645	3.644
23	3.646	3.645	3.645	3.645	3.645	3.644	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.644	3.644	3.645	3.645	3.644
24	3.646	3.645	3.645	3.645	3.645	3.644	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.644	3.644	3.645	3.645	3.644
25	3.646	3.645	3.645	3.645	3.645	3.644	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.644	3.644	3.645	3.645	3.644
26	3.646	3.645	3.645	3.645	3.645	3.644	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.644	3.644	3.645	3.645	3.644
27	3.646	3.645	3.645	3.645	3.645	3.644	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.644	3.644	3.645	3.645	3.644
28	3.646	3.645	3.645	3.645	3.645	3.644	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.644	3.644	3.645	3.645	3.644
29	3.646	3.645	3.645	3.645	3.645	3.644	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.644	3.644	3.645	3.645	3.644
30	3.646	3.645	3.645	3.645	3.645	3.644	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.644	3.644	3.645	3.645	3.644
31	3.646	3.645	3.645	3.645	3.645	3.644	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.644	3.644	3.645	3.645	3.644
32	3.646	3.645	3.645	3.645	3.645	3.644	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.644	3.644	3.645	3.645	3.644
33	3.646	3.645	3.645	3.645	3.645	3.644	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.644	3.644	3.645	3.645	3.644
34	3.645	3.645	3.645	3.645	3.645	3.644	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.645	3.644	3.644	3.645	3.645	3.644

[그림 17] 학습 데이터 중 1000_chg.csv 예시

• 데이터셋 구조

- 본 분석에서는 분석 실습과 관련 없는 항목들을 제외하고 배터리팩 충·방전 시험 설비로부터 수집되는 변수 208개(배터리셀 전압 176개, 배터리모듈 온도 32개)로 구성되어 있다.

• 데이터 개수 : 총 116,939,945개

- 학습용(Training) 데이터 : 111,489,609개
- 테스트용(Test) 데이터 : 5,450,336개



• 제조AI데이터셋 주요 변수 정의

변수 유형	변수명	설명
배터리팩 번호	SerialNumber	배터리팩 시리얼 번호
BMS 배터리팩 전압 (V)	Voltage	BMS에서 수집된 배터리팩 전압
BMS 배터리팩 전류 (A)	Current	BMS에서 수집된 배터리팩 전류
BMS 충전 상태(%)	RSOCmin	Real SOC Min
	RSOCmax	Real SOC Max
	RSOCavg	Real SOC avg
	USOCmin	User SOC min
	USOCmax	User SOC max
	USOCavg	User SOC avg
BMS 성능 상태(%)	SOH	배터리 잔존수명
BMS 전력(kW)	Power	BMS 전력
BMS 충·방전 최대 허용 전력(kW)	ChgPmax	BMS 충전중 최대 허용 전력
	DchgPmax	BMS 방전중 최대 허용 전력
BMS 충·방전 최대 허용 전류(kW)	ChgImax	BMS 충전중 최대 허용 전력
	DchgImax	BMS 방전중 최대 허용 전력
BMS 배터리셀 최소전압(mV)	Vmin	BMS 구성 배터리셀 중 최소전압값
BMS 배터리셀 최대전압(mV)	Vmax	BMS 구성 배터리셀 중 최대전압값
BMS 배터리셀 전압차 (mV)	DV	BMS 구성 배터리셀 중 최대전압과 최소전압의 차에 수식을 적용한 값
BMS 최소 배터리셀 온도 (°C)	Tmin	BMS 구성 배터리셀 중 최소 온도값
	Tmax	BMS 구성 배터리셀 중 최대 온도값
	Tavg	BMS 구성 배터리셀 중 평균 온도값
배터리셀 전압 (V)	MXXCV01	M01CV01~M01CV11 M02CV01~M02CV11 M03CV01~M03CV11 ... M15CV01~M15CV11 M16CV01~M16CV11 16개의 배터리모듈 176개의 배터리셀로 구성
	MXXCV02	
	MXXCV03	
	MXXCV04	
	MXXCV05	
	MXXCV06	
	MXXCV07	
	MXXCV08	
	MXXCV09	
	MXXCV10	
	MXXCV11	
배터리모듈 온도 ()	MXXT01	M01T01~M16T01 M01T02~M16T02
	MXXT02	총 32개의 배터리모듈 온도

[표 1] 데이터셋 주요변수



• 독립변수 / 종속변수

- 독립변수란, 다른 변수에 영향을 받지 않는 변수로, 입력값이나 원인을 나타낸다.
- 종속변수란, 독립변수의 변화에 따라 어떻게 변화하는지를 알고 싶은 변수로, 결과물이나 효과를 나타낸다.

구분	명칭
독립변수	MXXCV01
	MXXCV02
	MXXCV03
	MXXCV04
	MXXCV05
	MXXCV06
	MXXCV07
	MXXCV08
	MXXCV09
	MXXCV10
	MXXCV11
	MXXT01
	MXXT02
종속변수	없음

[표 2] 독립변수/종속변수 구성

- MXX는 모듈번호를 의미하며, 01 ~ 16번까지 존재한다.
- 독립변수 : 총 208개
- 종속변수 : 없음



• 주요 변수 기술 통계

구분	개수	평균	표준편차	최소값	중앙값	최대값
SerialNumber	504340	1163.578	831.3405	166	1021	5283
Voltage	504341	651.3336	147.5555	0	688.1	721.5
Current	504340	-2.03557	44.70008	-121.5	0	137.2
RSOCmin	504340	63.33219	28.09874	0	68.61	89.86
RSOCmax	504340	65.95474	27.58747	0	75.63	92
RSOCavg	504340	65.35261	27.54878	0	74.95	90.19
USOCmin	504340	68.90671	32.26046	0	75	100
USOCmax	504340	71.73438	31.62006	0	83	100
USOCavg	504340	71.32304	31.77569	0	82	100
SOH	504340	0	0	0	0	0
Power	504340	-1.63076	29.65823	-78.97	0	97.13
ChgPmax	504340	40.95806	27.36628	0	46.46	73.59
DchgPmax	504340	127.2945	33.73983	0	137.62	144.31
Chglmax	504340	61.47534	42.27841	0	69.5	107
Dchglmax	504340	185.8957	47.89484	0	200	200
Vmin	504340	3.731265	0.749892	0	3.905	4.09
Vmax	504340	3.748606	0.754731	0	3.925	4.1337
DV	504340	17.34109	9.262424	0	16	53.1
Tmin	504340	31.959	7.59029	-40	34	40
Tmax	504340	33.41509	7.963132	-40	35	43
Tavg	504340	32.72559	7.792263	-40	34	41
M01CV01	504338	3.855411	0.196372	3.3762	3.882	4.123
M01CV02	504338	3.856151	0.196685	3.3779	3.883	4.125
M01CV03	504338	3.855805	0.196659	3.3766	3.883	4.122
M01CV04	504338	3.85595	0.196727	3.3818	3.882	4.123
M01CV05	504338	3.855877	0.196594	3.369	3.883	4.121
M01CV06	504338	3.855603	0.196364	3.3732	3.883	4.122
M01CV07	504338	3.855616	0.196706	3.3699	3.882	4.127
M01CV08	504338	3.856675	0.197444	3.3757	3.883	4.124
M01CV09	504338	3.855351	0.196178	3.3688	3.882	4.143
M01CV10	504338	3.855839	0.197041	3.372	3.882	4.144
M01CV11	504338	3.855618	0.196735	3.3641	3.882	4.139
M02CV01	504338	3.855661	0.197061	3.1028	3.882	4.138

구분	개수	평균	표준편차	최소값	중앙값	최대값
M02CV02	504338	3.857137	0.197544	3.3723	3.883	4.2709
M02CV03	504338	3.856164	0.196614	3.3739	3.882	4.144
M02CV04	504338	3.856339	0.197048	3.3767	3.882	4.144
M02CV05	504338	3.856055	0.196472	3.3794	3.882	4.143
M02CV06	504338	3.85544	0.196269	3.38	3.882	4.142
M02CV07	504338	3.855587	0.196388	3.379	3.882	4.142
M02CV08	504338	3.855562	0.196166	3.3753	3.882	4.147
M02CV09	504338	3.855718	0.196438	3.3752	3.882	4.147
M02CV10	504338	3.855962	0.196757	3.3741	3.882	4.143
M02CV11	504338	3.856166	0.19688	3.3731	3.882	4.15
M03CV01	504338	3.856727	0.197434	3.3751	3.882	4.135
M03CV02	504338	3.857159	0.19773	3.3762	3.883	4.131
M03CV03	504338	3.85611	0.196808	3.3751	3.882	4.127
M03CV04	504338	3.856892	0.197499	3.3729	3.882	4.129
M03CV05	504338	3.855444	0.196259	3.3765	3.881	4.119
M03CV06	504338	3.857299	0.197858	3.3803	3.883	4.126
M03CV07	504338	3.855828	0.196743	3.3803	3.882	4.121
M03CV08	504338	3.857038	0.197417	3.3774	3.883	4.129
M03CV09	504338	3.85598	0.196543	3.3808	3.882	4.13
M03CV10	504338	3.856692	0.197358	3.3714	3.882	4.146
M03CV11	504338	3.855548	0.196544	3.3649	3.882	4.149
M04CV01	504338	3.856862	0.197609	3.3563	3.883	4.13
M04CV02	504338	3.855445	0.196228	3.3729	3.882	4.123
M04CV03	504338	3.856789	0.197058	3.3747	3.883	4.126
M04CV04	504338	3.855891	0.196384	3.3771	3.8825	4.135
M04CV05	504338	3.856573	0.197188	3.3773	3.883	4.14
M04CV06	504338	3.854655	0.195659	3.3779	3.882	4.121
M04CV07	504338	3.856254	0.19692	3.3778	3.883	4.119
M04CV08	504338	3.855414	0.195678	3.3888	3.883	4.121
M04CV09	504338	3.857017	0.197237	3.3897	3.883	4.122
M04CV10	504338	3.854991	0.195893	3.3826	3.882	4.12
M04CV11	504338	3.85699	0.197433	3.377	3.884	4.127
M05CV01	504338	3.855296	0.196078	3.3686	3.883	4.147
M05CV02	504338	3.855467	0.196072	3.3711	3.883	4.142



구분	개수	평균	표준편차	최소값	중앙값	최대값
M05CV03	504338	3.855512	0.196047	3.3794	3.883	4.14
M05CV04	504338	3.855786	0.196147	3.3579	3.883	4.145
M05CV05	504338	3.855012	0.195622	3.3788	3.883	4.143
M05CV06	504338	3.855775	0.19575	3.3809	3.883	4.149
M05CV07	504338	3.855209	0.195877	3.3745	3.883	4.143
M05CV08	504338	3.8558	0.196077	3.3788	3.884	4.151
M05CV09	504338	3.855671	0.196214	3.3745	3.883	4.143
M05CV10	504338	3.855879	0.196243	3.3739	3.883	4.144
M05CV11	504338	3.855189	0.196018	3.3628	3.883	4.141
M06CV01	504338	3.855395	0.196063	3.3638	3.883	4.145
M06CV02	504338	3.855969	0.196246	3.3722	3.883	4.141
M06CV03	504338	3.855747	0.19604	3.3741	3.883	4.135
M06CV04	504338	3.855198	0.195651	3.3754	3.883	4.131
M06CV05	504338	3.85558	0.195978	3.3726	3.883	4.124
M06CV06	504338	3.855178	0.195849	3.3763	3.883	4.122
M06CV07	504338	3.855749	0.195992	3.3733	3.883	4.122
M06CV08	504338	3.855665	0.196004	3.3809	3.883	4.12
M06CV09	504338	3.85544	0.195798	3.3847	3.883	4.123
M06CV10	504338	3.855604	0.196183	3.3853	3.883	4.121
M06CV11	504338	3.855688	0.196009	3.3774	3.883	4.123
M07CV01	504338	3.855436	0.196512	3.3778	3.883	4.145
M07CV02	504338	3.8557	0.196353	3.3796	3.883	4.144
M07CV03	504338	3.855103	0.196064	3.3754	3.882	4.135
M07CV04	504338	3.85587	0.196577	3.3764	3.883	4.139
M07CV05	504338	3.855657	0.196405	3.3803	3.883	4.136
M07CV06	504338	3.855834	0.196422	3.3776	3.883	4.133
M07CV07	504338	3.854971	0.195967	3.3747	3.883	4.137
M07CV08	504338	3.855605	0.19626	3.3772	3.883	4.139
M07CV09	504338	3.855806	0.196732	3.3725	3.882	4.138
M07CV10	504338	3.855556	0.196683	3.3733	3.883	4.135
M07CV11	504338	3.855288	0.196257	3.3916	3.883	4.141
M08CV01	504338	3.855441	0.196919	3.3663	3.882	4.151
M08CV02	504338	3.85536	0.196475	3.3568	3.882	4.147
M08CV03	504338	3.855158	0.19626	3.3546	3.882	4.149

구분	개수	평균	표준편차	최소값	중앙값	최대값
M08CV04	504338	3.855083	0.196055	3.3716	3.881	4.124
M08CV05	504338	3.855336	0.196486	3.3836	3.881	4.135
M08CV06	504338	3.854725	0.196044	3.3741	3.881	4.136
M08CV07	504338	3.854817	0.195931	3.3767	3.882	4.129
M08CV08	504338	3.854983	0.196193	3.3732	3.881	4.132
M08CV09	504338	3.85528	0.1964	3.3715	3.882	4.135
M08CV10	504338	3.85538	0.196577	3.3692	3.881	4.126
M08CV11	504338	3.854634	0.196404	3.3574	3.881	4.145
M09CV01	504338	3.855678	0.19657	3.3933	3.883	4.126
M09CV02	504338	3.855144	0.195917	3.3772	3.882	4.118
M09CV03	504338	3.855216	0.196165	3.37	3.882	4.119
M09CV04	504338	3.855171	0.195878	3.3743	3.882	4.1212
M09CV05	504338	3.85552	0.196386	3.3824	3.882	4.122
M09CV06	504338	3.855524	0.196166	3.3797	3.88275	4.125
M09CV07	504338	3.855385	0.196227	3.3756	3.883	4.1337
M09CV08	504338	3.855505	0.196292	3.3791	3.882	4.132
M09CV09	504338	3.854793	0.195643	3.3723	3.882	4.139
M09CV10	504338	3.854777	0.196091	3.37	3.881	4.136
M09CV11	504338	3.855558	0.196704	3.3691	3.882	4.14
M10CV01	504338	3.85468	0.196184	3.3741	3.881	4.146
M10CV02	504338	3.854771	0.195777	3.377	3.881	4.142
M10CV03	504338	3.854816	0.196138	3.3859	3.881	4.134
M10CV04	504338	3.854415	0.195665	3.3835	3.881	4.141
M10CV05	504338	3.854847	0.196074	3.3848	3.881	4.14
M10CV06	504338	3.854828	0.195812	3.3797	3.88185	4.144
M10CV07	504338	3.854629	0.195989	3.3785	3.881	4.138
M10CV08	504338	3.854607	0.195686	3.3829	3.881	4.14
M10CV09	504338	3.854756	0.196078	3.3835	3.881	4.136
M10CV10	504338	3.854502	0.195424	3.3717	3.882	4.141
M10CV11	504338	3.854941	0.196613	3.3723	3.881	4.14
M11CV01	504338	3.855591	0.196521	3.3742	3.883	4.133
M11CV02	504338	3.855754	0.196498	3.3709	3.882	4.135
M11CV03	504338	3.855269	0.196321	3.3769	3.882	4.131
M11CV04	504338	3.855156	0.196174	3.3847	3.882	4.135

구분	개수	평균	표준편차	최소값	중앙값	최대값
M11CV05	504338	3.854863	0.196227	3.372	3.882	4.133
M11CV06	504338	3.854982	0.196077	3.3735	3.882	4.131
M11CV07	504338	3.855329	0.196414	3.3772	3.882	4.134
M11CV08	504338	3.855165	0.196241	3.3727	3.882	4.133
M11CV09	504338	3.855256	0.1962	3.3789	3.882	4.137
M11CV10	504338	3.855563	0.196934	3.3706	3.883	4.137
M11CV11	504338	3.855463	0.196915	3.3594	3.883	4.14
M12CV01	504338	3.85529	0.196245	3.361	3.883	4.154
M12CV02	504338	3.855318	0.196172	3.3866	3.882	4.147
M12CV03	504338	3.854507	0.195571	3.3729	3.882	4.144
M12CV04	504338	3.854371	0.195402	3.3769	3.882	4.147
M12CV05	504338	3.854446	0.195428	3.3844	3.882	4.148
M12CV06	504338	3.854508	0.195523	3.384	3.882	4.147
M12CV07	504337	3.854614	0.195561	3.3788	3.882	4.142
M12CV08	504337	3.854839	0.195646	3.3684	3.883	4.147
M12CV09	504337	3.854894	0.19594	3.3752	3.882	4.139
M12CV10	504337	3.855033	0.196043	3.3741	3.882	4.138
M12CV11	504337	3.855002	0.196007	3.3663	3.883	4.145
M13CV01	504336	3.856804	0.197554	3.3751	3.88375	4.137
M13CV02	504336	3.856759	0.197295	3.3774	3.884	4.13
M13CV03	504336	3.856615	0.197257	3.3795	3.883	4.133
M13CV04	504336	3.857319	0.197614	3.3807	3.8845	4.137
M13CV05	504336	3.856619	0.197317	3.3795	3.883	4.132
M13CV06	504336	3.857266	0.197861	3.3733	3.884	4.133
M13CV07	504336	3.856522	0.197419	3.3673	3.8839	4.133
M13CV08	504336	3.856494	0.197558	3.3757	3.883	4.135
M13CV09	504336	3.856895	0.197475	3.3756	3.884	4.137
M13CV10	504336	3.856853	0.197611	3.368	3.884	4.14
M13CV11	504336	3.856939	0.197752	3.3615	3.884	4.135
M14CV01	504336	3.85602	0.197226	3.3655	3.883	4.148
M14CV02	504336	3.855706	0.196774	3.3817	3.883	4.146
M14CV03	504336	3.855347	0.196537	3.3703	3.882	4.147
M14CV04	504336	3.855789	0.196751	3.3701	3.883	4.146
M14CV05	504336	3.855384	0.19639	3.3803	3.882	4.139

구분	개수	평균	표준편차	최소값	중앙값	최대값
M14CV06	504336	3.855958	0.196993	3.3782	3.883	4.14
M14CV07	504336	3.854995	0.196159	3.3805	3.882	4.144
M14CV08	504336	3.856055	0.196897	3.3855	3.883	4.149
M14CV09	504336	3.855604	0.196898	3.3823	3.882	4.138
M14CV10	504336	3.855903	0.196686	3.3773	3.882	4.148
M14CV11	504336	3.855074	0.196541	3.3757	3.882	4.146
M15CV01	504336	3.856425	0.196853	3.372	3.884	4.134
M15CV02	504336	3.856286	0.196504	3.3838	3.8835	4.138
M15CV03	504336	3.85591	0.196401	3.3835	3.883	4.138
M15CV04	504336	3.856078	0.196239	3.379	3.883	4.14
M15CV05	504336	3.856493	0.196807	3.3941	3.884	4.132
M15CV06	504336	3.856073	0.19638	3.3829	3.884	4.136
M15CV07	504336	3.855562	0.196293	3.3722	3.883	4.138
M15CV08	504336	3.855976	0.196414	3.3771	3.883	4.136
M15CV09	504336	3.855618	0.196353	3.3878	3.882	4.134
M15CV10	504336	3.855648	0.196374	3.3805	3.882	4.136
M15CV11	504336	3.855857	0.196853	3.3656	3.883	4.139
M16CV01	504336	3.855141	0.196645	3.365	3.881	4.133
M16CV02	504336	3.855343	0.196669	3.3683	3.882	4.123
M16CV03	504336	3.855056	0.196272	3.3704	3.881	4.123
M16CV04	504336	3.854839	0.196469	3.3656	3.881	4.125
M16CV05	504336	3.854861	0.195951	3.377	3.881	4.12
M16CV06	504336	3.854927	0.196177	3.3745	3.881	4.119
M16CV07	504336	3.854473	0.195866	3.3722	3.881	4.122
M16CV08	504336	3.855389	0.196395	3.3732	3.882	4.12
M16CV09	504336	3.854621	0.195871	3.3736	3.881	4.132
M16CV10	504336	3.855077	0.196385	3.3703	3.881	4.132
M16CV11	504336	3.885065	0.794905	3.3674	3.884	52
M01T01	504338	33.91367	3.217412	14.1	34	42
M01T02	504338	33.93561	3.179291	14.3	34.2	41.7
M02T01	504338	33.97351	3.255448	14.4	34	42.8
M02T02	504338	34.12053	3.207208	14.9	34.3	42.5
M03T01	504338	34.04363	3.229472	15.1	34.2	42.5
M03T02	504338	34.10924	3.226424	14.8	34.3	42.6

구분	개수	평균	표준편차	최소값	중앙값	최대값
M04T01	504338	33.97564	3.277877	14.9	34.2	42.6
M04T02	504338	34.12465	3.256756	14.6	34.4	42.9
M05T01	504338	33.97372	3.231252	14.6	34.2	42.5
M05T02	504338	34.07805	3.284648	14.9	34.3	42.7
M06T01	504338	33.95445	3.229737	14.6	34.1	42.7
M06T02	504338	34.0927	3.271765	14.8	34.3	42.4
M07T01	504338	33.81943	3.124904	14.4	34	41.8
M07T02	504338	33.88452	3.181308	14.6	34.1	42.6
M08T01	504338	33.76917	3.213863	14.4	33.9	41.9
M08T02	504338	33.86051	3.149109	14.2	34	42.3
M09T01	504338	33.92993	3.298465	14.8	34	42.5
M09T02	504338	34.00643	3.211243	14.7	34.2	42.5
M10T01	504338	33.93349	3.237497	15	34.1	42.5
M10T02	504338	34.17	3.272225	14.8	34.5	43
M11T01	504338	34.00441	3.219091	14.6	34.1	42.3
M11T02	504338	34.08285	3.247191	14.7	34.2	42.1
M12T01	504337	34.0477	3.250102	15	34.1	42.2
M12T02	504337	34.19012	3.301618	15.3	34.4	42.6
M13T01	504336	34.00357	3.233446	15.1	34.2	42.3
M13T02	504336	34.07897	3.218409	15.2	34.4	42.2
M14T01	504336	33.9992	3.249952	15	34.1	42.4
M14T02	504336	34.01701	3.216188	15.3	34.2	42.3
M15T01	504336	33.97426	3.292704	14.7	34.1	42.2
M15T02	504336	34.02135	3.243106	14.4	34.2	42
M16T01	504336	33.76059	3.236125	14.1	33.9	42.3
M16T02	504336	33.90316	3.156175	14.1	34.1	43.7

[표 3] 데이터 변수 통계

- 본 데이터셋에 포함된 연속형 수치 데이터에 대하여 기초적인 기술 통계를 확인한다.

3) 데이터 품질 전처리

• 데이터 품질 전처리 목적

- 원시 데이터 (raw data)의 수집은 다양한 센서를 사용하여 배터리의 수명과 관련이 있는 데이터를 수집하여 저장함으로써 이루어진다.
- 실제 현장의 데이터는 의미 없는 값을 포함하거나 결측치, 혹은 이상 데이터가 발생하여 데이터 품질이 떨어진다.
- 품질이 낮은 데이터를 이용하면 좋은 결과를 얻을 수 없으므로 데이터의 전처리 (data pre-processing) 단계를 수행한다.
- 데이터의 품질을 측정하는 기준으로 다음 6가지 항목에 대하여 측정하여 데이터 품질 지수를 산출하고, 데이터의 전처리를 통하여 품질을 개선한다.

항목	항목 설명
완전성 (Completeness)	데이터의 필수 항목에 결측치가 없어야 함을 의미한다.
유일성 (Uniqueness)	데이터 엔티티 (data entity)가 관련 애플리케이션 내에서 고유하게 캡처, 표시 및 참조되어야 함을 나타낸다. 즉, 논리적으로 중복되는 엔티티가 없으며 데이터 세트 내의 각 엔티티를 고유하게 액세스하는 데 사용할 수 있는 고유 식별자가 있어야 함을 의미한다.
유효성 (Validity)	데이터 항목은 정해진 데이터의 유효범위 안에 있어야 한다.
일관성 (Consistency)	데이터의 구조, 유형 및 값은 형태가 일관되게 정의되고, 서로 일치해야 한다. 즉, 동일한 데이터 엔티티에 대하여 한 데이터셋의 데이터 값이 다른 데이터 세트의 값과 일관성이 있어야 함을 의미한다.
정확성 (Accuracy)	데이터가 모델링하려는 실제 개체를 정확하게 표현하는 정도를 나타낸다. 즉, 데이터 항목이 검증된 정확한 정보 (예: 참조 데이터)와 얼마나 일치하는지에 따라 측정된다. 정확성을 검증하기 위해서는 확증을 위한 참조 데이터 (다른 테이블의 유사한 확증 데이터 값)나 상관성이 존재하는 기존의 테이블의 값과 비교한다.
무결성 (Integrity)	데이터베이스 자료의 오류 없이 변화에 영향을 받지 않고 데이터의 유일성, 유효성, 일관성이 보호되어야 함을 의미한다.

[표 4] 데이터 품질지수

- 데이터 품질 지수에 관한 내용은 다음의 사이트를 참고하고, 구체적인 산출 방법은 4. 부록 [데이터 품질 전처리]를 참고한다.

◦ Data Quality Dimension 사이트:

<https://www.sciencedirect.com/topics/computer-science/data-quality-dimension>



• 데이터 품질 지수 세부 설명

(1) 완전성 품질지수

$$\text{완전성 품질지수} = \left(1 - \frac{\text{결측데이터의 개수}}{\text{전체 데이터의 개수}}\right) \times 100$$

- ① 결측 데이터의 값이 30% 이상인 레코드는 데이터의 완전성을 떨어뜨리기 때문에 각 열에 대한 결측 데이터값의 비율을 확인하여 삭제한다.
- ② 총 결측 데이터의 개수를 확인하기 위해 'isnull()' 함수를 사용하여 각 열에서의 결측 데이터 개수를 구한 뒤 'sum()' 함수를 이용하여 합산한다.
- ③ 위 식에 따라 완전성 품질지수를 계산한다.
- ④ 데이터의 완전성 (Completeness) 예시: 아래 테이블에서 Voltage에 null 값이 존재하므로 완전성에 위배된다.

	A	B	C	D	E	F	G	H	I	J	
1	Date	Time	SerialNum	Firmware	Voltage	ChgPmax	DchgPmax	ChgImax	DchgImax	Vmin	Vr
2	2022-05-27	14:29:14	6703	2.218	654.7	69.4	130.96	106	200	3.716	
3	2022-05-27	14:29:19	6703	2.218	655.2	69.45	131.03	106	200	3.718	
4	2022-05-27	14:29:24	6703	2.218		69.51	131.16	106	200	3.722	
5	2022-05-27	14:29:29	6703	2.218		69.54	131.22	106	200	3.724	
6	2022-05-27	14:29:34	6703	2.218	656.5	69.59	131.3	106	200	3.727	
7	2022-05-27	14:29:39	6703	2.218	656.7	69.61	131.34	106	200	3.728	
8	2022-05-27	14:29:44	6703	2.218	656.8	69.62	131.37	106	200	3.728	
9	2022-05-27	14:29:49	6703	2.218	656.9	69.63	131.37	106	200	3.728	
10	2022-05-27	14:29:54	6703	2.218	657.2	69.66	131.44	106	200	3.73	

(2) 유일성 품질지수

$$\text{유일성 품질지수} = \left(\frac{\text{유일한 데이터의 개수}}{\text{전체 데이터의 개수}}\right) \times 100$$

- ① 데이터에서 유일한 값을 갖는 열을 찾는다. 시계열 데이터는 주로 Timestamp이다.
- ② 유일한 값을 갖는 열에 대해 유일한 데이터 값을 확인하고, 그 개수를 구한다.
- ③ 위 식에 따라 유일성 품질지수를 계산한다.
- ④ 데이터의 유일성 (Uniqueness) 예시: 아래 테이블에서 시간이 5초 단위로 기록되어야 하는데, 같은 시간대가 중복해서 기록되었으므로 유일성에 위배된다.

	A	B	C	D	E	F	G	H	I	J	
29	2022-05-27	14:31:14	6703	2.218	659	69.86	131.82	106	200	3.741	
30	2022-05-27	14:31:19	6703	2.218	659.2	69.88	131.85	106	200	3.742	
31	2022-05-27	14:31:19	6703	2.218	659.3	69.88	131.86	106	200	3.743	
32	2022-05-27	14:31:29	6703	2.218	659.4	69.89	131.87	106	200	3.743	
33	2022-05-27	14:31:34	6703	2.218	659.4	69.9	131.89	106	200	3.743	
34	2022-05-27	14:31:39	6703	2.218	659.4	69.9	131.9	106	200	3.742	



(3) 유효성 품질지수

$$\text{유효성 품질지수} = \left(\frac{\text{유효성을 만족한 데이터의 개수}}{\text{전체 데이터의 개수}} \right) \times 100$$

- ① 데이터가 주어진 유효범위를 벗어나는지를 확인한다.
- ② 전체 데이터의 개수와 유효범위 안에 있는 데이터의 개수를 구한다.
- ③ 위 식에 따라 유효성 품질지수를 계산한다.
- ④ 데이터의 유효성(Validity) 예시: 아래 테이블에서 Voltage에 유효하지 않은 범위의 값이 존재하므로 유효성에 위배된다.

	A	B	C	D	E	F	G	H	I	J	
1	Date	Time	SerialNum	Firmware	Voltage	ChgPmax	DchgPmax	ChgImax	DchgImax	Vmin	Vr
2	2022-05-27	14:28:59	6703	2.218	652.9	69.2	130.57	106	200	3.706	
3	2022-05-27	14:29:04	6703	2.218	-222.2	69.3	130.76	106	200	3.711	
4	2022-05-27	14:29:09	6703	2.218	-222.2	69.35	130.86	106	200	3.714	
5	2022-05-27	14:29:14	6703	2.218	654.7	69.4	130.96	106	200	3.716	
6	2022-05-27	14:29:19	6703	2.218	655.2	69.45	131.03	106	200	3.718	

(4) 일관성 품질지수

$$\text{일관성 품질지수} = \left(\frac{\text{일관성을 만족한 데이터의 개수}}{\text{전체 데이터의 개수}} \right) \times 100$$

- ① 데이터 테이블 간 종속관계를 확인한다.
- ② 데이터의 타입이 정해진 형식에 맞는지 확인하고, 전체 데이터의 개수와 형식에 맞는 데이터의 개수를 구한다.
- ③ 위 식에 따라 일관성 품질지수를 계산한다.

(5) 정확성 품질지수

$$\text{정확성 품질지수} = 1 - \left(\frac{\text{정확성에 위배되는 데이터의 개수}}{\text{전체 데이터의 개수}} \right) \times 100$$

- ① 데이터 항목에 대하여 검증된 정확한 정보 (참조 데이터)가 있는지를 확인한다.
- ② 참조 데이터가 있는 경우, 데이터 항목 값과 참조 데이터의 값을 비교한다.
- ③ 이 차이가 0이 아니면 정확성을 위배하는 데이터가 존재함을 의미한다.

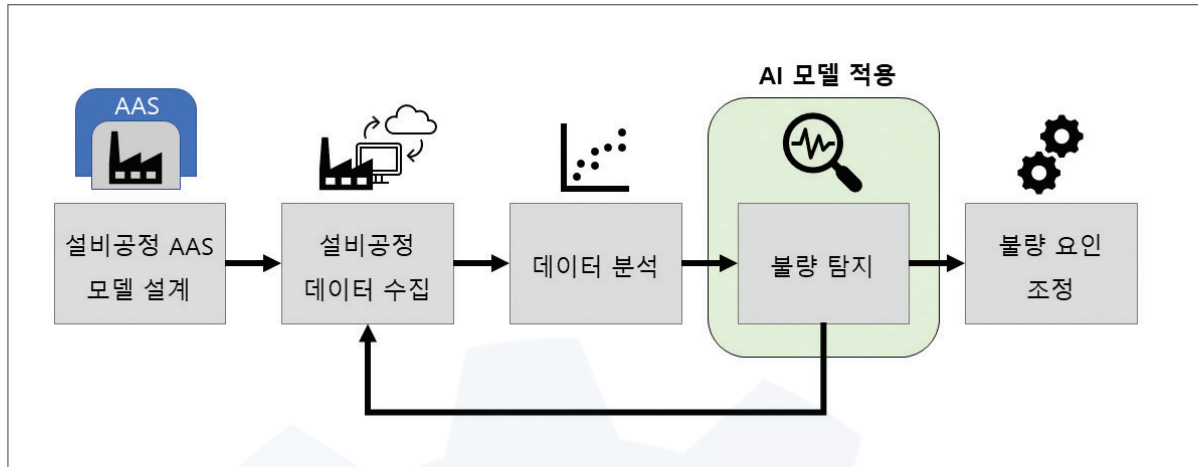
(6) 무결성 품질지수

$$\text{무결성 품질지수} = \left\{ 1 - \left(\frac{\text{유일성, 유효성, 일관성 중 100%가 아닌 지수}}{3} \right) \right\} \times 100$$

무결성 품질지수는 앞서 구한 '유일성', '유효성', '일관성' 지수를 기초로 위의 식으로 계산된다. 이 지수들이 100%이면 무결성 품질지수도 100%가 된다.

2.2 분석 모델 소개

1) 데이터 흐름 및 인공지능 모델 적용 흐름도



[그림 18] 데이터 흐름 및 인공지능 모델 적용 흐름도

2) AI 분석모델

• 해당 AI 방법론(알고리즘) 선정 이유

- 설비 데이터 분석 방법 : “PCA(Principal Component Analysis, 주성분분석) 및 MTadGAN(Multivariate Time Series Anomaly Detection by Generative Adversarial Network)”
 - 알고리즘 선정 이유: 배터리 충·방전 데이터는 약 170개 이상의 셀들의 전압과 온도 등으로 이루어져 있는데 이들 전압과 온도 값은 시간에 대해 서로 비슷한 함수로서 서로 간에 높은 상관관계를 가지고 있다. 이러한 다변량 시계열 데이터를 효율적으로 다루기 위해서는 차원을 낮추는 PCA 알고리즘이 유용하다. 또한 이러한 데이터에서의 이상(anomaly)은 흔히 전압값 등의 갑작스런 변화로 나타나는바 이러한 이상의 탐지에는 MTadGAN 같은 적대적 생성 모형이 효율적인 것으로 판단되어 이를 선정 하였다.
- (1) 주성분분석(PCA, Principal Component Analysis) 알고리즘은 많은 성분을 가지는 데이터의 차원을 낮추는 알고리즘으로 데이터의 분포를 계산하여 고차원에서 저차원으로 독립 변수의 개수를 낮추고 다음 단계의 AI 계산의 입력으로 하여 이상 탐지, 분류 등에 사용할 수 있도록 하는 알고리즘이다.
 - (2) MTadGAN(Multivariate Time Series Anomaly Detection by Generative Adversarial Network)은 다변량 시계열에 대하여 적대적 생성망의 방법을 적용하여 이상 탐지를 수행하는 알고리즘이다.
 - (3) 시계열 데이터에서의 이상(anomaly)이란 어떤 시계열 데이터 상에서 우리가 일반적으로 자연스럽게 일어나리라고 예측하는 패턴과 너무도 다른 패턴을 보이는 특정한 점이나 구간을 가리킨다. 여기에서 묵시적인 가정은 주어진 데이터가 어떤 특정한 규칙이나 법칙에 따라 생성

된다고 보는 것이다. 따라서 이상을 나타내는 점이나 구간은 데이터의 생성 규칙이나 법칙에 의해 생겨났다고 믿기 어려운 것을 의미한다. 따라서 이상 탐지(anomaly detection)는 흔히 주어진 데이터를 잘 기술하는 모형(Model)을 먼저 구성하고 이로부터 각각의 데이터 점들 중에 이 모형으로 설명되지 않고 거리가 먼 점이 포함된 구간을 찾아내게 된다.

- ① 이상값 데이터는 정상 데이터에 비해 매우 적게 발생하고 많은 분야 에서 이상 상태 데이터의 정답 레이블을 제공하기가 어렵다. 따라서 기계학습에 의한 이상 탐지에서 지도학습(supervised learning) 방식 보다는 비지도 학습(unsupervised learning) 방식이 주로 사용된다.
- ② 시계열 데이터에서의 이상 탐지는 입력 데이터의 특성에 따라서 한 가지 측정량을 다룰 경우 즉, 단변량(univariate) 데이터일 경우와 여러 가지 측정량을 동시에 다룰 경우 즉, 다변량(multivariate) 데이터의 경우로 나누어 볼 수 있고, 데이터 값이 연속적(continuous)인 변수 값일 경우와, 이와 달리 유한한 개수의 이산적(discrete)인 변수 값을 가지는 즉, 범주적(categorical)인 경우로 나누어 볼 수 있다. 여기서는 다변량 데이터에서의 이상 탐지를 위주로 다룬다. 이상 탐지의 출력의 관점에서는 개별 시점에 이상 여부를 부여하는 경우, 즉 점 이상(point anomaly)과, 연속적인 구간에 이상 여부를 부여하는 경우, 즉 구간 이상(subsequence anomaly)으로 나눌 수 있다.

- (4) 적대적 생성 신경망(Generative Adversarial Networks, GAN)(Goodfellow, et al 2014)을 이용한 시계열 데이터의 이상 탐지에서 적대적 생성 신경망은 이미지 생성, 이미지 변환, 그리고 비디오 예측 등에 성공적으로 활용되어 왔고 최근에는 이미지에서의 이상 탐지에서도 그 효율성을 보였다. 하지만 시계열 데이터에 대한 GAN방식의 연구는 매우 드물었다. 이는 시계열 데이터에 복잡한 시간적인 상관성(correlation)이 있어서 이에 생성적 모델을 적용하기가 어렵기 때문이다. 최근에 Li, et al 은 시계열 데이터에 기본적인 GAN방식을 적용하여 다변량 시계열 데이터의 분포를 표현하는 MAD-GAN 방법을 제안하였다. 또한, 최근에는 Wasserstein 로스 함수를 GAN에 적용하는 TadGAN (A. Geiger, et al., 2020) 알고리즘이 공개되었다. 그러나 공개된 알고리즘은 단변량 데이터에만 적용이 가능한 점이 단점이었다.

• 적용하고자하는 AI 분석 방법론(알고리즘)의 구체적 소개

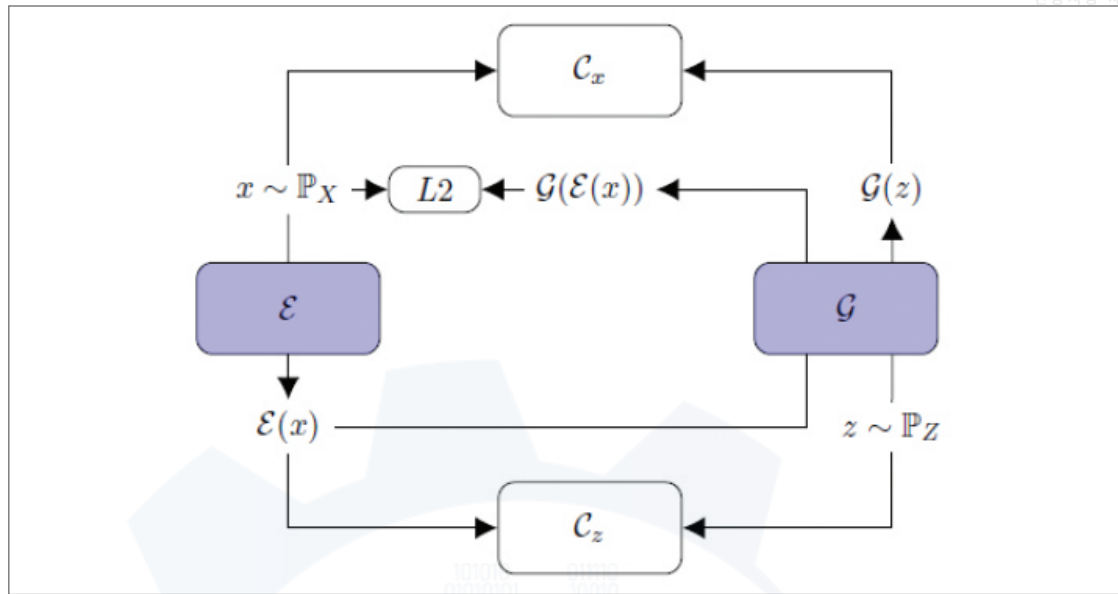
- PCA 알고리즘

- (1) PCA 알고리즘은 많은 측정 변수를 가지는 다변량 계의 여러 측정값 사이에 존재하는 관련성을 활용하여 저차원의 공간으로 투영-변환하여 변수의 개수를 줄임으로써 다음 단계의 이상탐지를 위한 AI 계산을 편리하게 수행할 수 있도록 해주는 알고리즘이다.

- MTadGAN 방법

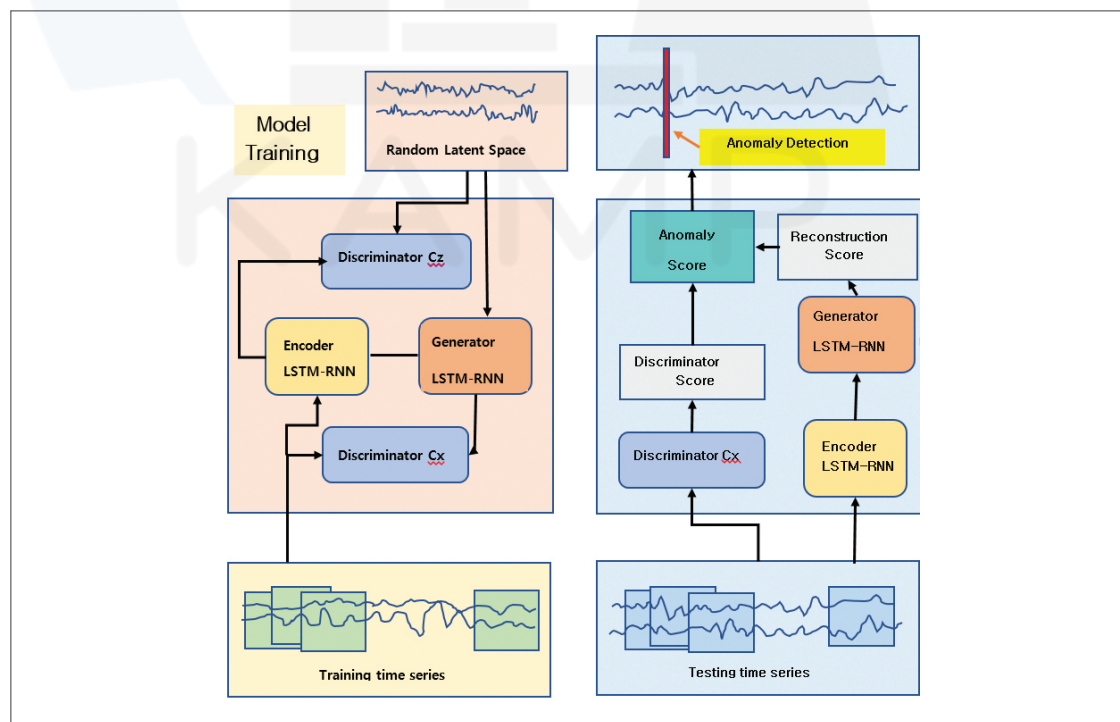
- (1) Wasserstein 로스 함수를 GAN에 적용하여 시계열 데이터의 이상을 탐지하는 TadGAN 방법으로부터 다변량의 계로 확장한 MTadGAN에 대해 설명한다.

- (2) MTadGAN의 모형 구조는 TadGAN과 동일한데 아래와 같이 그림 19의 도표로 나타낼 수 있다. 본 과제에서는 MTadGAN 알고리즘을 Tensorflow2로 구현하였다.



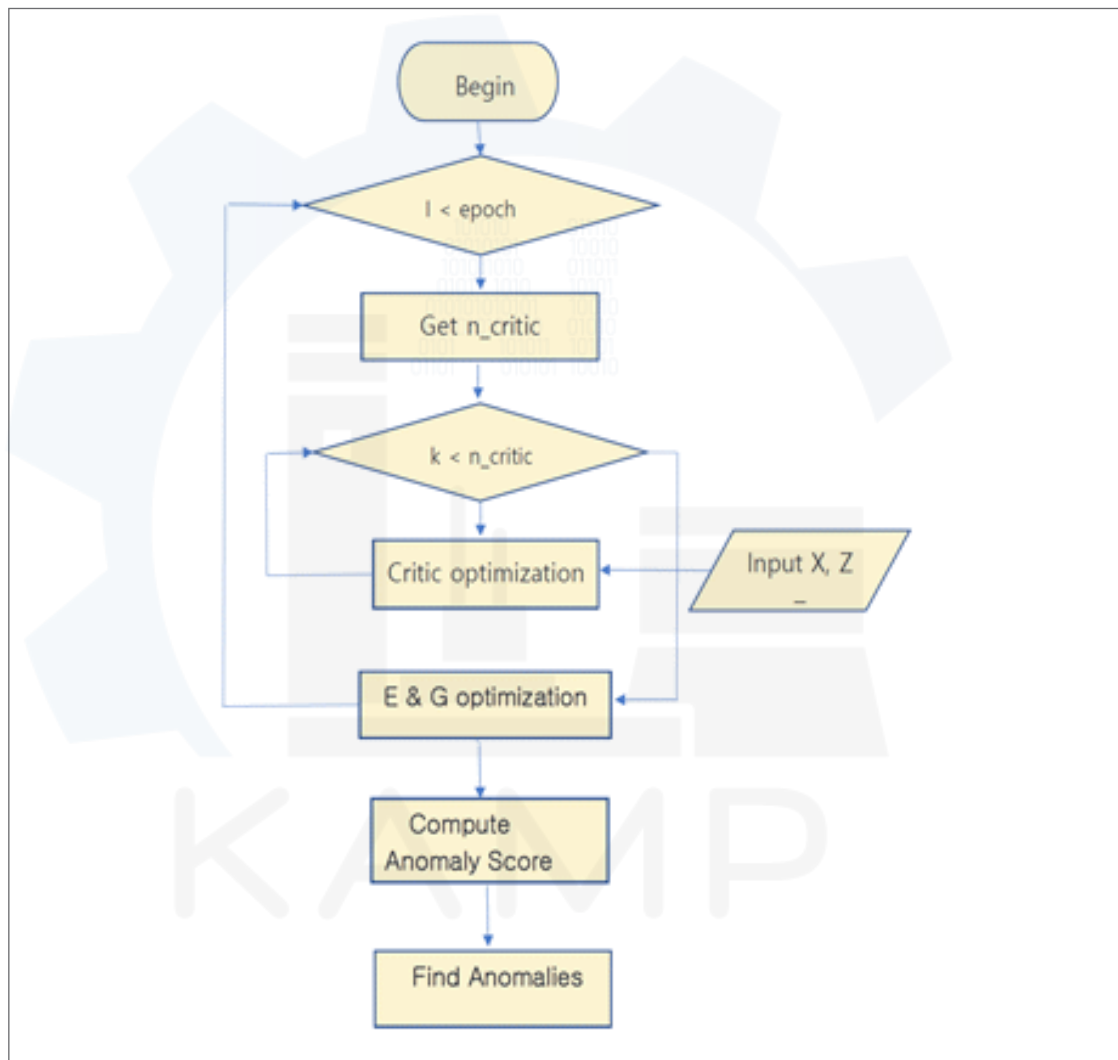
[그림 19] MTadGAN(또는 TadGAN)의 모형 구조

- (3) 생성자 네트워크 $E(x)$ 는 시계열 데이터 시퀀스를 잠재공간(latent space)로 변환하여 부호화(Encoder)하는 역할을 한다. 생성자 네트워크 $G(z)$ 는 잠재공간으로부터 변환하여 재구성되는 가상의 시계열을 생성하는 디코더(Decoder) 역할을 한다. 판별자(critic) C_x 는 실제의 데이터 X 에서 나오는 시계열 시퀀스들과 $G(z)$ 에서 생성된 가짜의 시계열 시퀀스들을 분별해 내는 함수이고 반면에 판별자 C_z 는 무작위로 택한 잠재공간 내의 표본 z 와 생성자 $E(x)$ 로 인코딩(encoding)된 표본을 분별하는 함수이다.



[그림 20] MTadGAN을 이용한 이상 탐지 모형 학습과 테스트 과정

- (4) 그림 20에서 왼쪽 블록들은 학습 과정을 보여주며 오른쪽은 이상 탐지 테스트 과정을 보여준다.
- (5) TADGAN 모형에서는 위의 그림 19에서와 같이 데이터 공간 X 와 잠재공간 Z 사이에 두 가지의 함수 즉, 인코더 함수 $E: X \rightarrow Z$, 그리고 생성자 함수 $G: Z \rightarrow X$ 를 트레이닝 하게 된다. X 는 실제 관측된 입력 데이터의 집합으로서 길이 t 인 연속된 시퀀스 데이터들의 모임을 나타낸다. 즉, $(x_i^{1,2,\dots,t})_{i=1}^N, x_i^{1,\dots,t} \in X$ 으로 표현할 수 있다. 여기서 $x_i^1, x_i^2 \dots$ 등은 특정 시각에 측정된 여러 측정치의 모임을 나타낸다. 또한 Z 는 잠재 공간(latent domain)으로서 표준 정규분포 $z \sim P_z = N(0,1)$ 를 따르는 무작위 잡음을 나타내는 벡터 z 를 Z 공간에서 추출하게 된다. 여기서는 편의상 시각 i 에 시작하여 길이가 t 인 시퀀스를 x_i 로 표시한다. 위의 두 함수를 결합하면 다음과 같이 입력 시계열 데이터를 재현할 수 있다.



[그림 21] MTadGAN 순서도

- (6) $x_i \rightarrow E(x_i) \rightarrow G(E(x_i)) \simeq \hat{x}_i$ 이 된다. 여기에서는 적대적 러닝을 이용하여 E 와 G 를 트레이닝 하고자 한다. 그림 19에 나타나 있듯이 두 함수 E 와 G 는 생성자(Generators)에 해당 하는데 E 는 시계열 데이터 시퀀스를 잠재 공간으로 보내는 인코더(Encoder)의 역할을 하고 반면에 G 는 잠재 공간을 변환하여 재구성된 시계열로 보내는 디코더(Decoder)의 역할을 함을 주목해야 한다. 또한 여기서 두 개의 적대적인 판별자(critics) 네트워크 C_x 와 C_z 를 도입

하는데 C_x 의 역할은 실제의 시계열 X 에서 나온 데이터 시퀀스와 $G(z)$ 에서 생성된 (가짜) 시퀀스를 구별해내는 일이고, 반면에 C_z 의 역할은 표준 정규분포에서 무작위로 택한 잠재공간 내의 표본 $z \sim P(z)$ 와 실제의 데이터 $x \sim P(x)$ 에서 인코딩(encoding)된 표본 $E(x)$ 를 구별해내는 것이다. 다시 말해서, 생성자 G 는 C_x 를 속여서 진짜처럼 보이는 시퀀스를 만들고자 하는 것이다. 따라서 최종적인 목표가 되는 손실(loss) 함수는 두 항의 합으로 이루어지는데, 그 첫째 항은 (1) 생성되는 시계열 시퀀스가 실제의 데이터 분포와 같아지도록 몰아가는 Wasserstein 손실 함수이고 다음으로는 (2) E 와 G 사이에 모순이 없도록 하는 이른바 순환-일관성 손실(cycle consistency losses)이다.

(7) 원래의 GAN 방식에서는 다음과 같은 표준적인 적대적 손실함수

$L = E_{x \sim P_x} [\log C_x(x)] + E_{z \sim P_z} [\log (1 - C_x(G(z)))]$ 를 적용하는데 여기에서 C_x 는 입력 시퀀스가 얼마나 실제 데이터에 가까운가를 0과 1 사이의 값으로 나타내는 함수이다. 하지만 이 손실함수의 방식은 이른바 모드-붕괴(mode collapse)의 문제점을 지니고 있다. 이것은 생성자 $G(z)$ 가 실제 (트레이닝) 데이터의 일부분만 학습한 채 판별자 C_x 를 속이는 데에 좋은 몇몇 일부 표본만을 계속 생성하려고 하는 것을 말한다. 더 이상 새로운 표본을 생성하기를 꺼리게 되고 따라서 목표로 하는 실제의 데이터 분포에 완벽하게 수렴하지를 못하게 되는 것이다. TadGAN에서는 이러한 한계를 극복하기 위하여 Wasserstein 손실함수를 적대적 손실함수로 GAN에 적용한다. 우선 P_X 를 실제 데이터 X 의 확률분포라고 하고 P_Z 를 잠재공간 Z 상에서 정의되는 표준정규분포라고 하자. 매핑 함수 $G: Z \rightarrow X$ 와 이에 대한 판별자 (네트워크) 함수 C_x 에 대해서 다음과 같은 최적화를 하게 된다.

$$\begin{aligned} & \min_G \max_{C_x \in C_X} V_X(C_x, G) \\ &= \min_G \max_{C_x \in C_X} E_{x \sim P_x} [C_x(x)] - E_{z \sim P_z} [C_x(G(z))] \end{aligned}$$

(8) $V_X(C_x, G) = E_{x \sim P_x} [C_x(x)] - E_{z \sim P_z} [C_x(G(z))]$ 이고 $C_x(x)$ 는 1-리프쉬츠 연속성을 만족하는 함수이다. K -리프쉬츠 연속성이란 양의 상수 K 그리고 임의의 두 점 x_1, x_2 에 대해서 $\|f(x_1) - f(x_2)\| \leq K \|x_1 - x_2\|$ 를 만족하는 것을 의미한다. 이러한 $K=1$ 리프쉬츠 연속성으로 인해서 트레이닝 과정에 그래디언트 기울기가 갑자기 커지는 위험이 줄어들고 안정적인 트레이닝이 가능해진다.

(9) 같은 방식으로 매핑 함수 $E: X \rightarrow Z$ 와 이에 대한 판별자 (네트워크) 함수 C_z 에 대해서도 동일하게 Wasserstein 손실함수를 다음과 같은 최적화를 하게 된다.

$$\begin{aligned} & \min_E \max_{C_z \in C_Z} V_Z(C_z, E) \\ &= \min_E \max_{C_z \in C_Z} E_{z \sim P_z} [C_z(z)] - E_{x \sim P_x} [C_z(E(x))] \end{aligned}$$

(10) $V_Z(C_z, E) = E_{z \sim P_z} [C_z(z)] - E_{x \sim P_x} [C_z(E(x))]$ 이다.

(11) 순환-일관성 손실(cycle consistency losses)을 두 생성자 E 와 G 에 적용한다. 이는 실제 데이터에서 추출된 입력 시퀀스와, 입력된 시퀀스가 E 로 인코딩 되고 그 다음에 G 로 재구성된 결과 사이의 차이 값에 $L2$ norm을 적용하여 최소화함으로써 생성자 네트워크 E 와 G 를 학습시킨다.

(12) 순환-일관성 손실함수는 $V_{L2}(E, G) = E_{x \sim P_x} [\|x - G(E(x))\|_2]$ 이다. 역방향의 일관성 손실함수 $E_{z \sim P_z} [\|z - E(G(z))\|_2]$ 를 추가할 수 있으나 성능을 더 높이지는 않는다고 한다.

(13) 위의 항들을 다 결합하면 최종적인 손실함수에 대한 MINMAX 문제를 얻게 된다.

$$L_{total} = \min_G \max_{C_x \in C_X, C_z \in C_Z} (V_X(C_x, G) + V_Z(C_z, E) + V_{L2}(E, G))$$

(14) TADGAN 모형의 전체 구조는 그림 20에 나타나 있다. 이 방식의 장점 중 하나는 판별자 네트워크 C_x 의 출력 스코어를 이상의 측도(anomaly measure)로 바로 활용할 수 있다는 점이다. 이것은 왜냐하면 C_x 가 바로 진짜 데이터 시퀀스와 가짜 사이를 판별하는 함수이기 때문이다.

(15) 다른 장점은 두 가지 생성자 네트워크 E 와 G 를 활용함으로써 원래의 시퀀스와 인코딩과 디코딩을 거쳐 재구성된 시퀀스 사이의 차이 값을 또 다른 이상 측도로 사용할 수 있다는 점이다.

(16) MTadGAN의 알고리즘 순서도는 그림 21과 같다. 주어진 시계열 데이터가 $X = (x_1, x_2, \dots, x_T)$ 라고 하자. x_i 는 시각 i 에서의 측정값들의 묶음 즉 일반적으로 d 개의 성분으로 이루어진 벡터를 나타낸다. 여기서는 설명의 편의상 하나의 실수로 표시되는 스칼라 값으로 간주한다.

(17) MTadGAN에서는 학습(TRAINING) 표본을 얻기 위해서 일정한 길이 t 의 슬라이딩 윈도우를 정하여 원래의 시계열 데이터를 서브 시퀀스의 집합 $X = (x_i^{1, \dots, t})_{i=1}^N$ 으로 만드는데 여기에서 $N = (T - t)$ 이 된다. 실제의 시계열 데이터에 있어서는 이상이 있는 점은 드물게 나타난다. 따라서 보통의 경우, 모든 학습 표본점이 정상이라고 가정한다. 또한 차원 k 인 잠재 공간에서 표준정규분포를 따르는 무작위 수를 추출하여 집합 $Z = (z_i^{1, \dots, k})_{i=1}^N$ 을 생성한다. 이 두 가지 X, Z 를 위의 GAN 모형에 입력하고 위에서 정의된 손실 함수를 이용하여 학습시킨다. 학습 후에 판별자의 출력 값과 재구성 에러 값을 이용하여 이상 점수(ANOMALY SCORE)를 계산할 수 있다.

• AI 분석 방법론(알고리즘) 구축 절차

- 변수 차원 축소 (PCA 알고리즘)

(1) 배터리 충전 및 방전 데이터에서 값이 없는 결측치는 표준적인 방법으로 값을 대체한다. 배터리 충전 방전 데이터의 경우 Voltage, 온도 등을 포함하여 약 260개의 다변량으로 이루어져 있고 이들 변수 사이에 높은 상관도가 있는 것으로 나타나 있어서 PCA 알고리즘을 써서 차원을 축소한다. 여기에서는 176개의 셀 각각의 전압과 32개 온도 데이터를 합하여 208개의 변수만을 추출하여 학습과 테스트를 실시한다.

- ① 배터리의 세 가지 모델(BS10, BS30, BS40) 각각에 대하여 충전과 방전 데이터로 분리한다. 현재의 배터리 데이터는 모두 이상(anomaly)이 없는 데이터로서 학습 데이터의 역할을 할 수 있고 이상 탐지를 테스트하기 위하여서는 별도로 제공된 4가지의 불량 데이터 중에서 세 가지를 택하여 MTadGAN으로 테스트한다.

(2) MTadGAN 학습 및 시각화

- ① MTadGAN 알고리즘을 활용하여 배터리 충전 및 배터리 방전 관련 Training 데이터에 대하여 학습하고 테스트 데이터에 대하여 테스트를 수행한다.

(3) 평가 및 해석

- ① Confusion Matrix를 통해 모델의 성능을 평가하며, 정확도, 정밀도, 재현율, F-score 등을 산출한다.

2.3 분석 체험

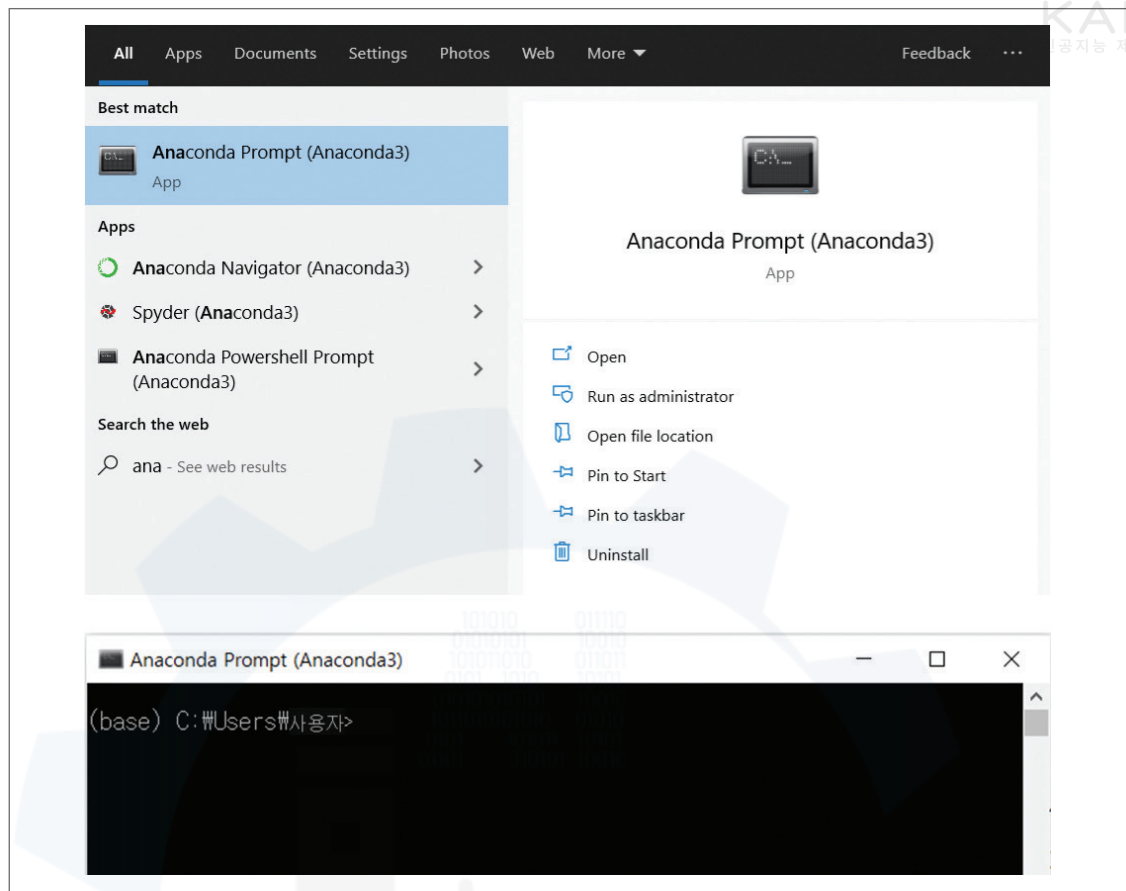
1) 필요 SW, 패키지 설치 방법 및 절차 가이드

▶ SW 설치는 '부록 3_분석환경 구축을 위한 설치 가이드' 참고

▶ 가상환경 구축 가이드

- 부록 3. [분석 환경 구축을 위한 설치 가이드]에서 설치한 아나콘다(Anaconda)에서 파이썬의 독립적인 가상의 작업 환경을 구축한다. 파이썬의 모듈은 다양한 버전이 존재하기 때문에 충돌이 일어나기 쉬우며, 이러한 충돌을 방지하기 위해 실습마다 독립적인 가상 환경 구축을 권장한다. 가상 환경을 사용하기 위해서는 환경을 구축하고 실행(활성화)해야하고, 실습이 끝난 이후에는 환경을 비활성화 시켜야 한다. 주피터 노트북에서 가상 환경을 실행하는 구체적인 순서 및 실행 방법은 아래와 같다.

① 아나콘다 프롬프트 실행



② 가상 환경 생성

```
# conda create -n 가상환경이름 python=버전
```

```
conda create -n KAMP python=3.8.8 jupyter
```

```
(base) C:\Users\User>conda create -n KAMP python=3.8.8 jupyter
Collecting package metadata (current_repodata.json): done
Solving environment: failed with repodata from current_repodata.json, will retry with next repodata source.
Collecting package metadata (repodata.json): done
Solving environment: done
```

```
==> WARNING: A newer version of conda exists. <==
current version: 4.12.0
latest version: 22.11.0
```

...

```
Proceed ([y]/n)? y
```

해당 문구가 뜨면 'y' 를 입력한다.

...

```
done
#
# To activate this environment, use
#
#     $ conda activate KAMP
#
# To deactivate an active environment, use
#
#     $ conda deactivate
#
(base) C:\Users\사용자>
```



③ 가상 환경 실행

가상 환경이 활성화되면 아래 그림처럼 명령 프롬프트 앞에 (가상 환경 이름)이 표시된다.

```
# conda activate 가상환경이름
```

```
conda activate KAMP
```

```
(base) C:\Users\사용자>conda activate KAMP
```

```
(KAMP) C:\Users\사용자>
```

④ 분석이 끝난 후에는 가상 환경을 종료한다.

```
conda deactivate
```

```
(KAMP) C:\Users\사용자>conda deactivate
```

```
(base) C:\Users\사용자>
```

⑤ 주피터 노트북에서 가상 환경 실행

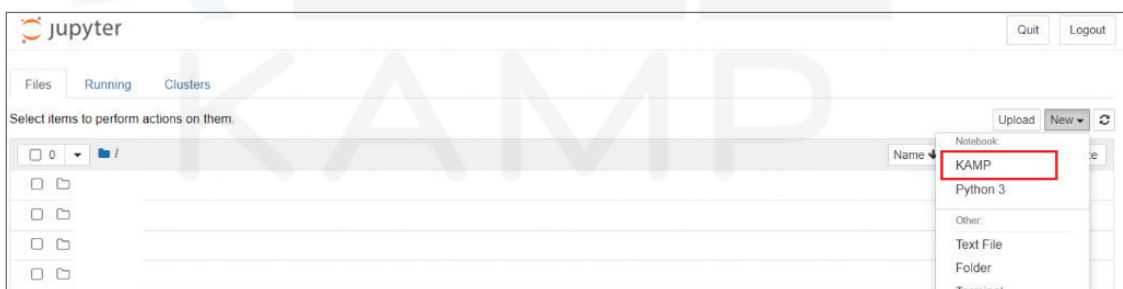
주피터 노트북에서 가상 환경을 실행하기 위해 가상 환경 활성화 상태에서 아래의 코드를 작성한다. 주피터 노트북 실행창에서 빨간 박스처럼 생성한 가상 환경이 있는 것을 확인할 수 있다. 해당 가상 환경을 선택 후 실습을 진행한다.

```
# python -m ipykernel install --user --name 가상환경이름 --display-name "가상환경이름"
```

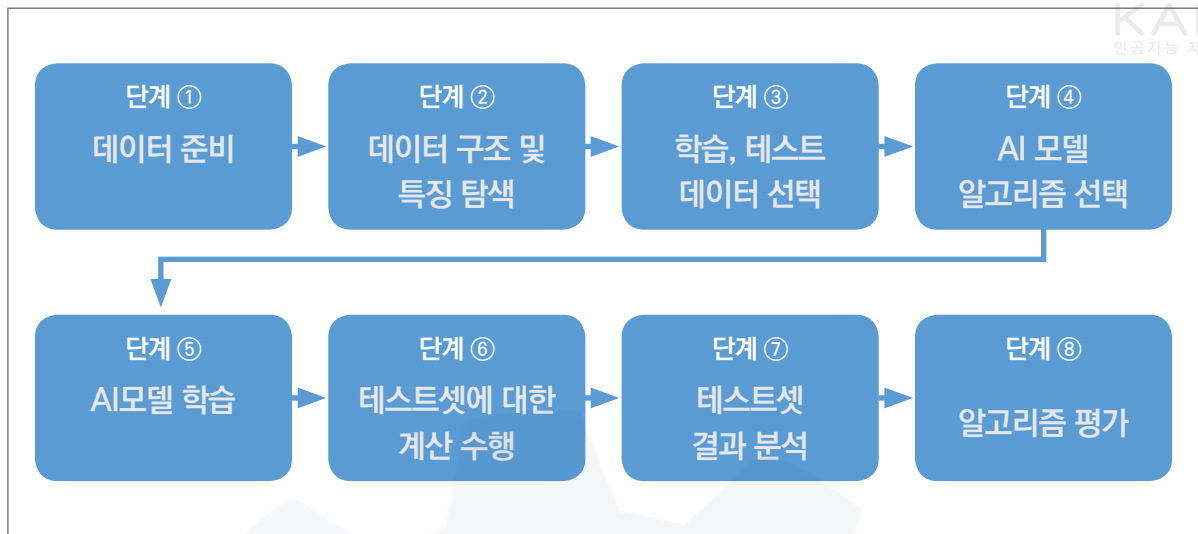
```
python -m ipykernel install --user --name KAMP --display-name "KAMP"
```

```
# jupyter notebook 실행
```

```
jupyter notebook
```



2) 분석 단계별 프로세스 - Flow Chart



[그림 22] 분석 단계별 프로세스

[단계 ①] 데이터 준비

①-1. 패키지 설치

- 현재 설치된 패키지 목록을 보기 위해서는 “!pip list”를 실행한다.

```
!pip list
```

Package	Version
absl-py	1.0.0
alabaster	0.7.12
anaconda-client	1.7.2
anaconda-navigator	2.0.3
anaconda-project	0.9.1
anyio	2.2.0
appdirs	1.4.4
argh	0.26.2
argon2-cffi	20.1.0
asn1crypto	1.4.0
astroid	2.5
astropy	4.2.1
astunparse	1.6.3
async-generator	1.10
atomicwrites	1.4.0
attrs	20.3.0
autopep8	1.5.6

[코드 1] 패키지 목록 확인

- 주피터 노트북 내에서 추가적으로 필요한 패키지 및 모듈은 “!pip install” 명령어를 사용하여 설치한다. 예를 들어 Python에서 시각화 도구로 많이 사용되는 matplotlib, 데이터 작업을 쉽고 직관적으로 할 수 있도록 해주는 pandas, 숫자 연산을 위한 Python 라이브러리인 numpy, 시각화를 위한 plotly를 설치하기 위해서는 다음 명령문을 실행한다.



```
!pip install matplotlib==3.5.1
!pip install pandas==1.4.2
!pip install numpy==1.19.5
!pip install plotly==5.10.0
!pip install scikit-learn==1.1.3
```

[코드 2] 패키지 설치

①-2. 라이브러리/패키지 불러오기

- 작업에 필요한 패키지 및 모듈을 import 하여 사용한다. 이때 별명(alias)을 주어 import 할 수 있으며, 친숙한 별명을 사용하면 편리하게 관련 패키지 및 모듈을 사용할 수 있다. numpy를 np, pandas를 pd 라는 별명으로 사용하고 싶은 경우, 다음 명령어를 사용한다. 또한, 라이브러리로부터 특정한 모듈만 import할 경우에는 다음과 같이 from~import 구문을 사용한다.

```
# "as name" indicates the name we will use to refer to this library
import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
# we will only import certain module from those libraries
from mpl_toolkits.mplot3d import Axes3D
from sklearn.decomposition import PCA
#from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import MinMaxScaler
from sklearn.covariance import EllipticEnvelope
from sklearn.ensemble import IsolationForest
from random import randrange
from datetime import datetime
import math
import plotly.graph_objects as go
import plotly.express as px
from plotly.subplots import make_subplots
warnings.simplefilter(action='ignore', category=FutureWarning)
```

[코드 3] 라이브러리 및 패키지 호출

- file_path에 분석 파일인 “1000_chg.csv”를 지정하고, pandas 라이브러리를 사용하여 csv 파일을 불러오며, Pandas DataFrame 형식으로 데이터를 사용한다. Pandas DataFrame은 2차원 테이블 형식으로 되어 있어 데이터를 직관적으로 쉽게 처리할 수 있다.

```
# the file path is the archive location of the file in our computer
file_path = './data/raw_data/train/1000_chg.csv'
data = pd.read_csv(file_path) # using pandas library (pd) to read the csv file.
# After reading the file, it will be used as a Pandas DataFrame.
# Pandas DataFrame is a special data structure from Pandas Library that is two-dimensional,
# size-mutable, potentially heterogeneous tabular data.
```

[코드 4] 파일 호출

①-3. 데이터 종류 및 개수 확인

- 데이터의 속성 (column) 이름과 각 속성의 처음 5개 행을 표시하기 위해서 head() 메소드를 사용하며, 이를 실행하면 다음의 내용이 표시된다.

```
# Showing the first five rows of our data
data.head()
```

	Date	Time	SerialNumber	Voltage	Current	RSOCmin	RSOCmax	RSOCavg	USOCmin	USOCmax	...	M12T01
0	2020-08-04	15:51:49	1000	641.3	0.0	33.43	34.29	33.84	33	34	...	31.6
1	2020-08-04	15:51:50	1000	641.3	0.0	33.43	34.29	33.84	33	34	...	31.6
2	2020-08-04	15:51:51	1000	641.3	0.0	33.43	34.29	33.84	33	34	...	31.6
3	2020-08-04	15:51:52	1000	641.3	0.0	33.43	34.29	33.84	33	34	...	31.6
4	2020-08-04	15:51:53	1000	641.3	0.0	33.43	34.29	33.84	33	34	...	31.6

5 rows × 231 columns

[코드 5] 데이터의 처음 5개의 행 확인

- 데이터의 속성을 확인하기 위하여 다음 구문을 사용하며, 전체 231개의 속성이 있음을 확인할 수 있고, 이 속성들 중 일부가 표시된다.

```
# There are 231 columns, and some of the names are shown in below.
data.columns
```

```
Index(['Date', 'Time', 'SerialNumber', 'Voltage', 'Current', 'RSOCmin',
       'RSOCmax', 'RSOCavg', 'USOCmin', 'USOCmax',
       ...,
       'M12T01', 'M12T02', 'M13T01', 'M13T02', 'M14T01', 'M14T02', 'M15T01',
       'M15T02', 'M16T01', 'M16T02'],
      dtype='object', length=231)
```

[코드 6] 데이터 속성 확인

- info() 메소드는 Data frame의 요약 정보를 보여준다. 표시되는 정보는 데이터 수가 6009 개이며, 231개의 속성이 있음을 나타낸다. 또한, 데이터의 유형, non-null value 및 메모리 사용 등의 정보가 표시된다.

```
# Concise summary of the dataframe. This method prints information about a DataFrame
# including the index dtype and columns, non-null values and memory usage.
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6009 entries, 0 to 6008
Columns: 231 entries, Date to M16T02
dtypes: float64(219), int64(10), object(2)
memory usage: 10.6+ MB
```

[코드 7] 데이터 프레임 요약 확인



- 데이터의 형태를 (rows, columns) 형식으로 보여준다. 데이터 수가 6009개이며, 231개의 속성이 있음을 알 수 있다.

```
data.shape
```

```
(6009, 231)
```

[코드 8] 데이터 형태 확인

①-4. 데이터 특성 파악 및 시각화

- 각 속성의 데이터 유형을 표시하기 위해서는 다음 구문을 사용한다.

```
data.dtypes
```

```
Date          object
Time          object
SerialNumber   int64
Voltage        float64
Current        float64
...
M14T02        float64
M15T01        float64
M15T02        float64
M16T01        float64
M16T02        float64
Length: 231, dtype: object
```

[코드 9] 속성 별 데이터 유형 확인

- 각 속성의 고유한 값 (unique value)의 수를 표시하기 위해서는 다음 구문을 사용한다.

```
data.nunique(axis = 0)
```

```
Date          1
Time          6009
SerialNumber   1
Voltage        712
Current        286
...
M14T02         46
M15T01         50
M15T02         43
M16T01         48
M16T02         40
Length: 231, dtype: int64
```

[코드 10] 속성 별 고유한 값의 수 확인

- Time Stamp 가 중복된 행을 표시하기 위해서는 다음 구문을 사용한다. 여기에서는 중복된 행이 없음을 알 수 있다.

```
data[data["Time"].isin(data["Time"][data["Time"].duplicated()])].sort_values("Time")
```

Date	Time	SerialNumber	Voltage	Current	RSOCmin	RSOCmax	RSOCavg	USOCmin	USOCmax	...	M12T01	M12T02	M
0 rows × 231 columns													

[코드 11] Time Stamp가 중복된 행 표시

- describe() 메소드는 데이터의 기초 통계를 요약하여 보여준다. 데이터의 각 속성에 대하여 데이터수(count), 평균(mean), 표준편차(std), 최소값(min), 25/50/75 분위수, 그리고 최대값(max) 등의 값을 표시한다.

```
data.describe()
```

	SerialNumber	Voltage	Current	RSOCmin	RSOCmax	RSOCavg	USOCmin	USOCmax
count	6009.0	6009.000000	6009.000000	6009.000000	6009.000000	6009.000000	6009.000000	6009.000000
mean	1000.0	690.951207	-35.552072	70.717998	71.577998	71.127998	77.457813	78.155933
std	0.0	24.973541	25.325789	19.037510	19.037510	19.037510	22.550164	22.239537
min	1000.0	641.300000	-57.300000	33.430000	34.290000	33.840000	33.000000	34.000000
25%	1000.0	666.400000	-55.400000	54.090000	54.950000	54.500000	58.000000	59.000000
50%	1000.0	698.100000	-54.100000	75.850000	76.710000	76.260000	83.000000	84.000000
75%	1000.0	714.800000	0.000000	89.600000	90.460000	90.010000	100.000000	100.000000
max	1000.0	719.000000	0.000000	89.600000	90.460000	90.010000	100.000000	100.000000
8 rows × 229 columns								

[코드 12] 데이터 기초 통계 요약 확인

- 히스토그램(Histogram)은 측정값이 존재하는 범위를 복수 개의 구간으로 나눈 후, 각 구간을 X축으로 하고 각 구간에 속하는 측정값의 출현 빈도수를 Y축으로 하여 배열한 그래프이다. 다음 그림은 속성 "Voltage", "Current", "RSOCavg", "USOCavg", "Power", "M16T02", "Vmax", "Tavg" 에 대하여 히스토그램을 행렬 형태로 나타낸 것이다.

```
# array_attributes = the names of the columns we would like the histograms for
# cols_number = indicate how many columns for the grid
def showHistograms(data, histogram_attributes, cols_number, height, width):

    rows_number = math.ceil(len(histogram_attributes) / cols_number)
    # rows number calculated automatically
    fig = make_subplots(rows = rows_number, cols = cols_number,
                        subplot_titles = (histogram_attributes))

    for i in range(len(histogram_attributes)):
        legend = histogram_attributes[i] + " count"
        # cols and rows start in 1, not in 0, thus we add one.
        row_number = i%cols_number + 1
        col_number = math.ceil((i+1) / cols_number)
        fig.append_trace(go.Histogram(name = legend, x = data[histogram_attributes[i]],
                                     row_number, col_number))
    fig.update_layout(height = height, width = width,
                      title_text = "Histograms of Selected Attributes")
    fig.show()
```

```
histogram_attributes = ["Voltage", "Current", "RSOCavg", "USOCavg", "Power", "M16T02",
                       "Vmax", "Tavg"]
# attributes we want in our histogram grid
showHistograms(data, histogram_attributes, 3, 900, 900)
```



[코드 13] 히스토그램

- 상관관계 히트 맵(Correlation Heat Map)에서 상관관계는 두 변수 간의 연관성 정도를 나타내는 통계적 척도이며, 여기에서는 -1에서 1 사이의 값을 생성하는 Pearson 상관관계를 사용한다. 값이 1이면 두 변수가 강한 양의 상관관계를 갖는 것이고, -1이면 변수는 음의 상관관계를 가진다. 그리고 값이 0이면 두 변수 사이에 상관관계가 없음을 나타낸다.
- 히트맵은 데이터의 패턴과 변화를 시각적으로 보여주는 데 사용된다. 시간에 따른 변화를 보여줄 수 있고, 색상 코딩 시스템을 사용하여 데이터의 다양한 값을 그래픽으로 표현할 수 있다. 즉, 색상으로 표현할 수 있는 다양한 정보를 일정한 이미지 위에 열분포 형태의 비주얼한 그래픽으로 출력하는 시각화 도구이다. 히트맵은 대량의 다차원 데이터를 시각화하는 데 적합하고 유사한 값이 있는 행의 클러스터가 유사한 색으로 표시되므로 이러한 행 클러스터를 식별하는 데 유용하다.
- 모든 변수 쌍 간의 상관관계수 값을 히트맵 그래프를 사용하여 시각화할 수 있다. 히트맵에서 셀의 색상이 노란색이면 양의 상관관계가 있음을 의미하고, 셀의 색상이 파란색이면 음의 상관관계가 있음을 나타낸다. 또한, 셀의 색상이 보라색이면 상관관계가 없거나 충분히 강하지 않음을 나타낸다. 다음 함수 plotCorrelationMatrixText()는 주어진 크기에 따라 상관관계 히트 맵을 그려주는 함수이다.

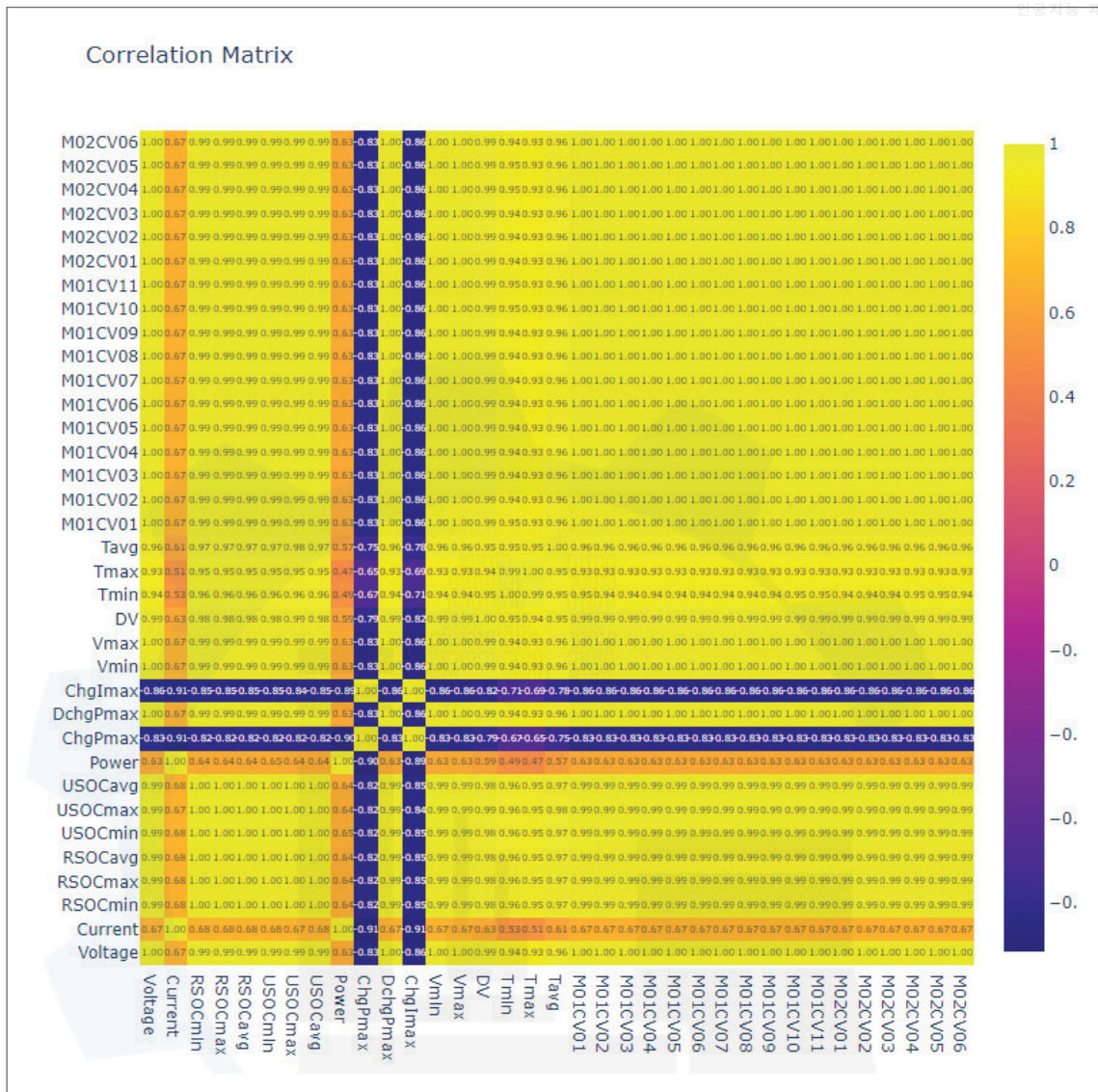
```
# Procedure for Drawing Correlation Heatmap with Coefficient Text Printed
# Input parameter: the dataframe name, the width for heatmap graph, and the number of column
# to visualize.
# Output: correaltion heatmap of the input dataframe with size (width X width)
def plotCorrelationMatrixText(df, graphSize, n):
    df = df[[col for col in df if df[col].nunique() > 1]] # keep columns where there are
                                                         # more than 1 unique values
    df = df.iloc[:, :n+1] #get the first n columns
    if df.shape[1] < 2: #check if there are more than one column in dataset.
        #If only one column in a dataset, print error message.
        print(f'No correlation plots shown: ')
        print(f'The number of non-NaN or constant columns ({df.shape[1]}) is less than 2')
        return
    corr = df.corr() #calculate pearson correlation between all attributes in dataset

    heatmap = go.Heatmap(
        z=corr,
        x=corr.columns,
        y=corr.columns,
        text=corr,
        texttemplate="%{text:.2f}",
        textfont={"size":7}
    )

    layout = go.Layout(
        title_text = "Correlation Matrix",
        width=graphSize,
        height=graphSize,
    )
    fig = go.Figure(data=[heatmap], layout=layout)
    fig.show()
```



```
plotCorrelationMatrixText(data, 800, 35) #create heatmap correlation for dataframe data2
#with 10 pixels width and for 35 first column
```



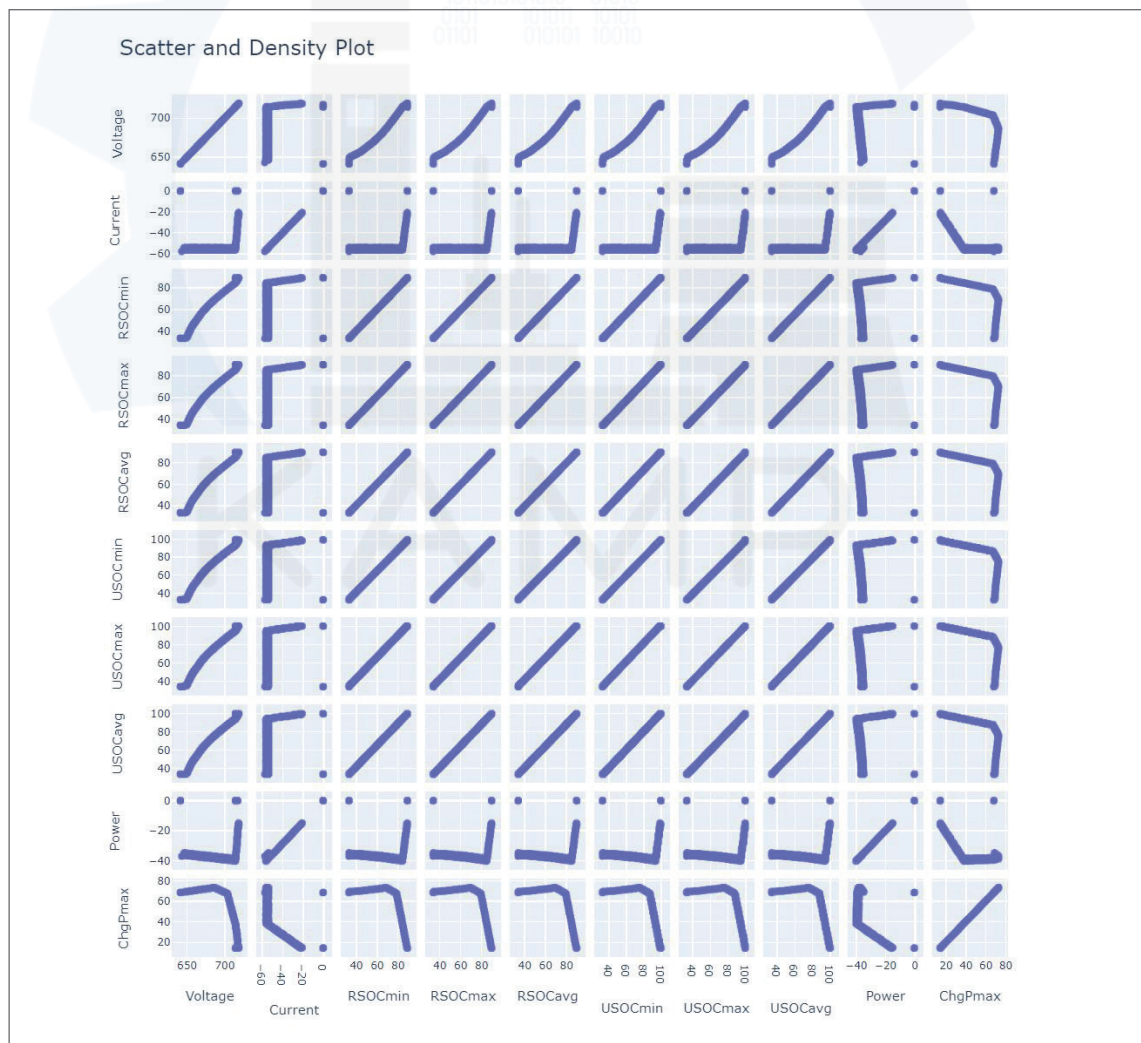
[코드 14] 상관관계 히트맵

- 산점도 행렬(Scatter Matrix)에서 산점도는 두 속성 간의 연관성을 보여주기 위하여 가로 축과 세로 축으로 형성된 2차원 공간에 데이터 포인트를 표시한다. 데이터 테이블의 각 행은 X축 및 Y축에 설정된 값에 따라 위치가 결정되는 마커(marker)로 표시되며, 마커가 산점도에서 직선에 가까운 경우 두 변수의 상관관계가 높고 균등하게 분산되는 경우 상관관계가 낮음을 나타낸다. 산점도 행렬은 데이터셋의 모든 속성 쌍 사이의 산점도를 시각화하여 보여주는 그래프이다.

```

# Procedure for Drawing Scatter Plot Matrix
# Input parameter: the dataframe name, the number of column to visualize, the size of graph,
#                 the text size of graph
# Output: Scatter matrix graph with specified size
def plotScatterMatrix(df, n, plotSize):
    df = df.select_dtypes(include = [np.number]) # keep only numerical columns
    df = df.dropna('columns') #drop column with missing data
    df = df[[col for col in df if df[col].nunique() > 1]]
            # keep columns where there are more than 1 unique values
    df = df.iloc[:, :n] #get the first n columns
    columnNames = list(df) #list of all attributes
    df = df[columnNames]
    layout = go.Layout(
        title_text = "Scatter matrix",
        width = plotSize,
        height = plotSize,)
    fig = px.scatter_matrix(df)
    fig.update_layout(
        title = '<span style="font-size: 16px;">Scatter and Density Plot</span>',
        width = plotSize, height=plotSize,
        font = dict(size = 8))
    fig.show()
  
```

```
plotScatterMatrix(data, 10, 800) #draw scatter matrix for dataframe dataset2 with size 800px
```



[코드 15] 산점도 행렬



①-5. 데이터 정제 (전처리)

- 결측치 처리(Missing Value Handling) : 데이터에 결측치로 인하여 중요한 의사결정이 달라질 수 있으며, 수집한 데이터에 결측치가 있으면 이를 해결하는 것이 중요하다. 결측치가 전체 데이터에 비해 아주 적다면 가장 좋은 방법은 결측치를 제거하는 것이다. 결측치 제거는 결측치가 존재하는 전체 행을 삭제하는 목록 삭제 방식과 결측치 자체만 삭제하고 다른 변수가 존재하는 경우에는 그대로 유지한 채 분석을 수행하는 페어와이즈 (pairwise) 삭제 방법이 있다. 목록 삭제는 페어와이즈 삭제에 비해 정보 손실이 크다. 그러나, 페어와이즈 삭제의 문제점은 항상 사용할 수 있는 것이 아니고, 샘플과 분석 방법에 따라 다르다.
- file_path에 분석 파일인 “1000_chg.csv”를 지정하고, pandas 라이브러리를 사용하여 csv 파일을 불러온다.

```
file_path = './data/raw_data/train/1000_chg.csv' # the file path is the archive location of the file in our computer
data = pd.read_csv(file_path) # using pandas library (pd) to read the csv file.
# After reading the file, it will be used as a Pandas DataFrame.
# Pandas DataFrame is a special data structure from Pandas Library that is two-dimensional,
# size-mutable, potentially heterogeneous tabular data.
```

[코드 16] 파일 호출

- 속성 별 null 개수를 확인하기 위하여 isna().sum() 메소드를 사용한다. 표시되는 정보는 다음과 같이 (column name, null 갯수) 형태이다.

```
# Displays the total number of null data of each column
data.isna().sum()
```

```
Date      0
Time      0
Serial Number  0
Voltage    0
Current    0
..
M14T02     0
M15T01     0
M15T02     0
M16T01     0
M16T02     0
Length: 231, dtype: int64
```

[코드 17] 속성 별 null 개수 확인

- null 값을 포함하고 있는 행을 표시하기 위해서는 다음 구문을 사용한다.



```
# Null 값을 포함하고 있는 행을 표시하기 위해서는 다음 구문을 사용한다.
null_data = data[data.isnull().any(axis=1)]
null_data
```

Date	Time	SerialNumber	Voltage	Current	RSOCmin	RSOCmax	RSOCavg	USOCmin	USOCmax	...	M12T01	M12T02	M13T01	M13T02	M14T01	M14T02
0 rows × 231 columns																

[코드 18] null 값을 포함하고 있는 행 확인

- 속성의 값이 오직 하나인 속성을 파악하여 제거한다. 하나의 값만을 가지는 속성은 전체 분석에 의미가 없으므로 이러한 속성은 제거하고 유효한 복수 개의 값을 가지는 속성만을 대상으로 분석을 진행한다.

```
# Function for removing the attribute with unique values below certain threshold
# Input parameter: the dataframe name (df) and threshold value for unique number (n)
# Output: dataframe where all of attributes have more than n unique values
def removeConstant(df, n):
    df = df[[col for col in df if df[col].nunique() > n]]
    return df
data1 = removeConstant(data, 1)
```

[코드 19] 속성의 값이 오직 하나인 속성 제거

- 속성의 값이 오직 하나인 속성을 제거한 후 남은 속성의 수는 227개로서, 4개의 속성이 제거되었음을 알 수 있다.

```
data1.shape
```

```
(6009, 227)
```

[코드 20] 속성 제거 후 데이터 형태 확인

- 본 분석 파일의 경우에는 모든 속성에서 null 값을 포함하고 있는 행이 없으므로 결측치가 존재하지 않음을 알 수 있다. 결측치가 존재하는 경우에는 결측치 처리를 해야 하며, 결측치 처리를 위한 `handleMissingValue()` 함수를 제공한다. 이 함수에서는 결측이 발생한 속성이 범주형인 경우는 그 속성의 대표값으로 결측치를 채우고, 수치형인 경우는 결측치가 발생한 시점의 이전 값과 이후 값을 고려하여 보간법을 적용하여 결측치를 채운다. 다음은 `handleMissingValue()` 함수의 정의이다.



```
# Function for handling missing value
# Input parameter: the dataframe
# Output: dataframe with all missing value already fill in
def handleMissingValue(data):
    df = data.copy()
    numeric_df = df.columns[df.dtypes != 'object']
    categorical_df = df.columns[df.dtypes == 'object']
    #handling missing value for categorical
    for i in categorical_df:
        df[i].fillna(df[i].mode()[0], inplace = True)

    df_flag_null = df.isnull()
    i, c = np.where(df_flag_null)
    #handling missing value for numerical
    for j in range(len(i)):
        #if missing value in the beginning
        if i[j] == 0:
            s = df.iloc[:, c[j]]
            id_s = s.notna().idxmax()
            val = df.iat[id_s, c[j]]
            df.iat[i[j], c[j]] = val
            #df.iloc[[i[j]], [c[j]]] = df.iloc[[id_s], [c[j]]]
            #print(type(val))
        #if missing value in the end
        elif i[j] == (len(df) - 1):
            s = df.iloc[:, c[j]]
            id_s = s.notna()[::-1].idxmax()
            val = df.iat[id_s, c[j]]
            df.iat[i[j], c[j]] = val
        #if missing value in the middle
        else:
            low = df.iat[i[j] - 1, c[j]]
            high = df.iat[i[j] + 1, c[j]]
            print(high)
            if(math.isnan(high)):
                val = low
            else:
                val = (low+high) / 2
            df.iat[i[j], c[j]] = val

    return df
```

[코드 21] 결측치 처리 함수 정의

- 위의 결측치 처리 함수 handleMissingValue()를 사용하여 결측치를 처리하고 처음 5행을 표시하는 코드는 다음과 같다.



```
data2 = handleMissingValue(data1)
data2.head(5)
```

	Time	Voltage	Current	RSOCmin	RSOCmax	RSOCavg	USOCmin	USOCmax	USOCavg	Power	...	M12T01	M12T02
0	15:51:49	641.3	0.0	33.43	34.29	33.84	33	34	34	0.0	...	31.6	31.7
1	15:51:50	641.3	0.0	33.43	34.29	33.84	33	34	34	0.0	...	31.6	31.7
2	15:51:51	641.3	0.0	33.43	34.29	33.84	33	34	34	0.0	...	31.6	31.7
3	15:51:52	641.3	0.0	33.43	34.29	33.84	33	34	34	0.0	...	31.6	31.7
4	15:51:53	641.3	0.0	33.43	34.29	33.84	33	34	34	0.0	...	31.6	31.7

5 rows × 227 columns

[코드 22] 결측치 처리 후 데이터의 처음 5개의 행 확인

- 이상치 처리(Outlier Handling): 이상치는 속성의 값이 일반적인 값과 편차가 큰 값을 말하며, 데이터셋이 이상치를 포함하면 분석의 정확도가 떨어진다. 따라서, 이상치를 검출하여 제거한 후 분석을 수행해야 한다. 이상치 검출 방법으로는 분산 (variance)을 사용하여 정규분포에서 97.5% 이상 또는 2.5%의 이하에 포함되는 값을 이상치로 판별하는 방법, 군집화 (clustering)를 시행하여 데이터를 여러 군집으로 구분한 후 어느 군집에도 속하지 않는 데이터를 이상치로 판별하는 방법, 그리고 사분범위 (inter-quartile range, IQR)을 이용한 방법 등 다양한 방법이 있다.
- 본 배터리팩 품질보증 평가 분석 실습에서는 데이터의 특성을 고려하여 IQR을 이용한 방법을 사용하여 이상치를 판별하고 제거한다. IQR은 $Q3 - Q1$ 으로 정의되며, 다음 `identify_outliers()` 함수는 IQR을 사용하여 이상치를 검출한다. 두 번째 인수인 상수 c 는 IQR의 c 배수를 의미하며 $c \times IQR$ 범위를 벗어난 값을 이상치로 간주한다.

```
# Function for identify outlier
# Input parameter: the dataframe and constant for IQR boundary
# (make sure all of the column in dataframe is numerical data types)
# Output: all of rows identified as outliers
def identify_outliers(df, c):
    #set the constant for IQR boundary
    constant = float(c)
    # calculate Q1 and Q3
    Q1 = df.quantile(0.25)
    Q3 = df.quantile(0.75)
    # calculate the IQR
    IQR = Q3 - Q1
    # filter the dataset with the IQR
    IQR_outliers = df[((df.lt(Q1 - constant * IQR)) | (df.gt(Q3 + constant * IQR))).any(axis=1)]
    IQR_outliers = pd.DataFrame(IQR_outliers)
    return IQR_outliers
```

[코드 23] 이상치 판별 함수의 정의

- 이상치가 검출되면 이상치를 포함하고 있는 데이터를 다음의 함수를 사용하여 제거한다.

```
# Function for removing outlier
# Input parameter: the dataframe and constant for IQR boundary
# (make sure all of the column in dataframe is numerical data types)
# Output: dataframe without outliers
def remove_outliers(df, c):
    #find outliers
    df = pd.DataFrame(df)
    outliers = identify_outliers(df, c)
    #remove outliers
    df_out = pd.DataFrame(outliers)
    df.drop(df_out.index, inplace = True)
    return df
```

[코드 24] 이상치 제거 함수 정의

- boolean 속성의 값은 숫자형 (0, 1)로 변환하고, identify_outliers() 함수의 두 번째 인수(c)를 '4'로 주어, 속성의 값이 4 x IQR의 범위를 벗어나면 이상치로 간주하여 이상치를 검출한다. identify_outliers() 함수의 실행 결과, 여기에서는 이상치가 검출되지 않았음을 확인할 수 있다.

```
# identify_outliers() 함수의 두 번째 인수를 '4'로 준다.
# 4 x IQR의 범위를 벗어나면 이상치로 간주하여 이상치를 검출한다.
# 이상치 검출 결과, 여기에서는 다음과 같이 이상치가 검출되지 않는다.
data_numeric = data2.copy()
a = identify_outliers(data_numeric[:, 4.0]) #get the list of index of outliers data
print(len(a))
```

```
0
```

[코드 25] 이상치 검출

- 다음 구문으로 이상치가 발생한 행을 확인할 수 있다. 이상치에 대해서는 필요에 따라 제거하거나 대체할 수 있다. 여기에서는 이상치가 발생하지 않았음을 나타낸다.

```
a.index
```

```
Int64Index([], dtype='int64')
```

[코드 26] 이상치가 검출된 행 확인

- 데이터 정규화(Data normalization) : 정규화는 기계 학습을 위한 데이터 전처리 단계에서 자주 사용되는 기술이다. 정규화의 목표는 값 범위 차이의 왜곡이나 정보 손실 없이 공통의 스케일을 사용하도록 데이터셋의 숫자 값을 변경하는 것이며, 기계 학습 알고리즘에서 데이터를 정확하게 모델링하려는 경우에도 정규화가 필요하다. 정규화는 속성값의 편차를 해소하여 모든 숫자 속성에 적용된 스케일 내에서 값을 유지하는 동시에 원본 데이터의 일반적인 분포와 비율을 유지하는 새 값을 만들어 준다. 여

기에서는 표준 정규화 후 데이터값의 범위를 [0, 1]로 최소-최대 정규화 (min-max normalization) 방법을 사용하여 정규화를 수행한다. 이에 대해서 scikit-learn에서 제공하는 StandardScaler()와 MinMaxScaler()를 사용한다. 배터리 충·방전 데이터에서는 여러 셀의 전압과 온도 데이터만으로 학습과 테스트를 진행하게 되는데 이 경우에는 많은 수의 변수를 AI 모형에 입력하기 전에 먼저 PCA 방식으로 차원을 축소하고 나서 스케일링을 나중에 적용하게 된다.

- 전압과 온도 데이터 값만을 추출하기 위해 각 속성의 이름을 확인한다.

```
# Column 필드 이름의 리스트를 확인한다.
columns_list2 = data2.columns
for i in range(len(columns_list2)):
    print(i, columns_list2[i])
```

```
9 Power
10 ChgPmax
11 DchgPmax
12 ChgImax
13 Vmin
14 Vmax
15 DV
16 Tmin
17 Tmax
```

[코드 27] 필요한 속성의 이름 확인

- 셀 전압 온도 데이터만을 추출해서 새로 데이터프레임을 만든다.

```
# Collect voltage and temperature data
df_train_0 = data2
columns_vt = columns_list2[18:226]
print(columns_vt)
df_train = df_train_0[columns_vt]
df_train.head(5)
```

```
Index(['Tavg', 'M01CV01', 'M01CV02', 'M01CV03', 'M01CV04', 'M01CV05',
      'M01CV06', 'M01CV07', 'M01CV08', 'M01CV09',
      ...,
      'M11T02', 'M12T01', 'M12T02', 'M13T01', 'M13T02', 'M14T01', 'M14T02',
      'M15T01', 'M15T02', 'M16T01'],
      dtype='object', length=208)
```

	Tavg	M01CV01	M01CV02	M01CV03	M01CV04	M01CV05	M01CV06	M01CV07	M01CV08	M01CV09	...	M11T02	M12T01
0	31	3.646	3.645	3.645	3.645	3.645	3.644	3.645	3.645	3.645	...	31.7	31.6
1	31	3.646	3.645	3.645	3.645	3.645	3.644	3.645	3.645	3.645	...	31.7	31.6
2	31	3.646	3.645	3.645	3.645	3.645	3.644	3.645	3.645	3.645	...	31.7	31.6
3	31	3.646	3.645	3.645	3.645	3.645	3.644	3.645	3.645	3.645	...	31.7	31.6
4	31	3.646	3.645	3.645	3.645	3.645	3.644	3.645	3.645	3.645	...	31.7	31.6

5 rows x 208 columns

[코드 28] 신규 데이터 프레임 생성



- data 폴더에 하위 폴더 “./data/preprocessed/train”를 생성하고 그 안에 전처리된 학습파일 “./data/preprocessed/train/1000_chg.csv”을 저장한다.

```
# Save the processed (train) data for later
df_train.to_csv('./data/preprocessed/train/1000_chg.csv', index = False)
```

[코드 29] 전처리한 데이터 저장

- 현재 폴더에서 테스트 파일 “Test05_NG_chg.csv”를 pandas로 읽는다.

```
# Test data 전처리 과정 시작: 원 데이터 불러오기
# the file path is the archive location of the file in our computer
file_path = './data/raw_data/test/Test05_NG_chg.csv'
df_test1 = pd.read_csv(file_path) # using pandas library (pd) to read the csv file.
```

[코드 30] 테스트 파일 Test05_NG_chg.csv 호출

- 테스트 파일에서 단일한 값을 가지는 속성을 제거한다.

```
df_test1 = removeConstant(df_test1, 1)
df_test1.shape
```

(5016, 212)

[코드 31] 속성 제거 후 데이터 형태 확인

- 결측치를 처리한다.

```
# 결측치를 처리한다.
test1 = handleMissingValue(df_test1)
test1.head(5)
```

	Time	Vmin	Vmax	DV	M01CV01	M01CV02	M01CV03	M01CV04	M01CV05	M01CV06	...	M12T01	M12T02	M13T01	M13T02	M14T01	M14T02	M15T01
0	9:35:12	3.6422	3.6462	4.0	3.6436	3.6446	3.6440	3.6441	3.6438	3.6436	...	24.9	24.7	24.7	24.8	25.1	24.9	24.9
1	9:35:13	3.6422	3.6462	4.0	3.6436	3.6445	3.6441	3.6441	3.6437	3.6437	...	24.9	24.8	24.7	24.8	25.1	24.9	24.9
2	9:35:14	3.6423	3.6461	3.8	3.6436	3.6444	3.6440	3.6439	3.6436	3.6435	...	24.9	24.7	24.7	24.8	25.1	24.9	24.9
3	9:35:15	3.6421	3.6461	4.0	3.6435	3.6445	3.6440	3.6440	3.6436	3.6435	...	24.9	24.7	24.8	24.8	25.1	24.9	24.9
4	9:35:16	3.6422	3.6461	3.9	3.6434	3.6444	3.6440	3.6438	3.6436	3.6435	...	24.9	24.8	24.7	24.8	25.1	24.9	24.9

5 rows × 212 columns

[코드 32] 결측치 처리

- 셀 전압과 온도 데이터의 열의 인덱스를 확인한다.



```
# 셀 전압과 온도 데이터의 열의 인덱스 확인.
columns_test1 = test1.columns
for i in range(len(columns_test1)):
    print(i, columns_test1[i])
```

```
0 Time
1 Vmin
2 Vmax
3 DV
4 M01CV01
5 M01CV02
6 M01CV03
7 M01CV04
8 M01CV05
```

[코드 33] 셀 전압, 온도 데이터의 인덱스 확인

- 셀 전압과 온도 데이터를 추출한다.

```
# Collect voltage and temperature data
columns_vt = columns_test1[4:212]
print(columns_vt)
df_test1 = test1[columns_vt]
df_test1.head(5)
```

```
Index(['M01CV01', 'M01CV02', 'M01CV03', 'M01CV04', 'M01CV05', 'M01CV06',
      'M01CV07', 'M01CV08', 'M01CV09', 'M01CV10',
      ...,
      'M12T01', 'M12T02', 'M13T01', 'M13T02', 'M14T01', 'M14T02', 'M15T01',
      'M15T02', 'M16T01', 'M16T02'],
      dtype='object', length=208)
```

	M01CV01	M01CV02	M01CV03	M01CV04	M01CV05	M01CV06	M01CV07	M01CV08	M01CV09	M01CV10	...	M12T01	M1
0	3.6436	3.6446	3.6440	3.6441	3.6438	3.6436	3.6436	3.6438	3.6436	3.6436	...	24.9	
1	3.6436	3.6445	3.6441	3.6441	3.6437	3.6437	3.6436	3.6437	3.6435	3.6434	...	24.9	
2	3.6436	3.6444	3.6440	3.6439	3.6436	3.6435	3.6435	3.6437	3.6435	3.6434	...	24.9	
3	3.6435	3.6445	3.6440	3.6440	3.6436	3.6435	3.6434	3.6436	3.6435	3.6434	...	24.9	
4	3.6434	3.6444	3.6440	3.6438	3.6436	3.6435	3.6434	3.6436	3.6435	3.6433	...	24.9	

5 rows x 208 columns

[코드 34] 셀 전압, 온도 데이터 추출

- 전처리된 테스트 파일 “Test05_NG_chg.csv”을 하위폴더 “data/preprocessed/test”에 저장한다.

```
# Save the processed (train) data for later
df_test1.to_csv('./data/preprocessed/test/Test05_NG_chg.csv', index=False)
```

[코드 35] 전처리된 테스트 파일 저장

- 테스트 파일 “Test07_NG_dchg.csv”을 불러온다.

```
# Next, 용량 불량 테스트 데이터
# the file path is the archive location of the file in our computer
file_path = './data/raw_data/test/Test07_NG_dchg.csv'
df_test2 = pd.read_csv(file_path) # using pandas library (pd) to read the csv file.
```

[코드 36] 테스트 파일 Test07_NG_dchg.csv 호출



- 테스트 파일에서 단일한 값을 가지는 열을 제거한다.

```
df_test2 = removeConstant(df_test2, 1)
df_test2.shape
```

```
(4594, 209)
```

[코드 37] 속성 제거 후 데이터 형태 확인

- 결측치를 처리한다.

```
# 결측치를 처리한다.
test2 = handleMissingValue(df_test2)
test2.head(5)
```

	Time	M01CV01	M01CV02	M01CV03	M01CV04	M01CV05	M01CV06	M01CV07	M01CV08	M01CV09	...	M12T01	M12T02	M13T01	M13T02	M14T01	M
0	14:32:52	3.9545	3.955	3.9591	3.9535	3.9516	3.9538	3.9535	3.9545	3.9525	...	25.7	25.8	25.6	25.8	25.8	
1	14:32:53	3.9544	3.955	3.9592	3.9535	3.9515	3.9539	3.9534	3.9546	3.9525	...	25.7	25.8	25.6	25.8	25.8	
2	14:32:54	3.9545	3.955	3.9592	3.9535	3.9515	3.9539	3.9535	3.9547	3.9525	...	25.7	25.8	25.6	25.8	25.8	
3	14:32:55	3.9545	3.955	3.9591	3.9535	3.9515	3.9539	3.9535	3.9546	3.9526	...	25.7	25.8	25.6	25.8	25.8	
4	14:32:56	3.9545	3.955	3.9591	3.9533	3.9516	3.9539	3.9534	3.9547	3.9525	...	25.7	25.8	25.6	25.8	25.8	

5 rows × 209 columns

[코드 38] 결측치 처리 후 데이터의 처음 5개의 행 확인

- 셀 전압과 온도 데이터의 열의 인덱스를 확인한다.

```
# 셀 전압과 온도 데이터의 열의 인덱스 확인.
columns_test2 = test2.columns
for i in range(len(columns_test2)):
    print(i, columns_test2[i])
```

```
0 Time
1 M01CV01
2 M01CV02
3 M01CV03
4 M01CV04
5 M01CV05
6 M01CV06
```

[코드 39] 셀 전압, 온도 데이터의 인덱스 확인

- 셀 전압과 온도 데이터를 추출한다.

```
# Collect voltage and temperature data
columns_vt = columns_test2[1:209]
print(columns_vt)
df_test2 = test2[columns_vt]
df_test2.head(5)
```

```
Index(['M01CV01', 'M01CV02', 'M01CV03', 'M01CV04', 'M01CV05', 'M01CV06',
       'M01CV07', 'M01CV08', 'M01CV09', 'M01CV10',
       ...,
       'M12T01', 'M12T02', 'M13T01', 'M13T02', 'M14T01', 'M14T02', 'M15T01',
       'M15T02', 'M16T01', 'M16T02'],
      dtype='object', length=208)
```

	M01CV01	M01CV02	M01CV03	M01CV04	M01CV05	M01CV06	M01CV07	M01CV08	M01CV09	M01CV10	...	M12T01	M12T02
0	3.9545	3.955	3.9591	3.9535	3.9516	3.9538	3.9535	3.9545	3.9525	3.9546	...	25.7	25.7
1	3.9544	3.955	3.9592	3.9535	3.9515	3.9539	3.9534	3.9546	3.9525	3.9547	...	25.7	25.7
2	3.9545	3.955	3.9592	3.9535	3.9515	3.9539	3.9535	3.9547	3.9525	3.9547	...	25.7	25.7
3	3.9545	3.955	3.9591	3.9535	3.9515	3.9539	3.9535	3.9546	3.9526	3.9546	...	25.7	25.7
4	3.9545	3.955	3.9591	3.9533	3.9516	3.9539	3.9534	3.9547	3.9525	3.9547	...	25.7	25.7

5 rows × 208 columns

[코드 40] 셀 전압, 온도 데이터 추출

- 전처리된 테스트 파일 “Test07_NG_dchg.csv”을 하위폴더 “data/preprocessed/test”에 저장한다.

```
# Save the processed (train) data for later
df_test2.to_csv('./data/preprocessed/test/Test07_NG_dchg.csv', index = False)
```

[코드 41] 전처리된 테스트 파일 저장

- 테스트 파일 “Test08_NG_chg.csv”을 불러온다.

```
# Next, 용량 불량 테스트 데이터
# the file path is the archive location of the file in our computer
file_path = './data/raw_data/test/Test08_NG_chg.csv'
df_test3 = pd.read_csv(file_path) # using pandas library (pd) to read the csv file.
```

[코드 42] 테스트 파일 Test08_NG_chg.csv 호출

- 세 번째 테스트 파일에서 단일한 값을 가지는 열을 제거한다.

```
df_test3 = removeConstant(df_test3, 1)
df_test3.shape
```

```
(2396, 209)
```

[코드 43] 속성 제거 후 데이터 형태 확인

- 결측치를 처리한다.



```
# 결측치를 처리한다.
test3 = handleMissingValue(df_test3)
test3.head(5)
```

	Time	M01CV01	M01CV02	M01CV03	M01CV04	M01CV05	M01CV06	M01CV07	M01CV08	M01CV09	...	M12T01	M12T02	M13T01	M13T02	M14T01	M14T02
0	8:00:48	3.6374	3.6375	3.6374	3.6367	3.6366	3.6366	3.6361	3.6366	3.6359	...	23.1	23.1	23.0	22.9	22.9	22.9
1	8:00:49	3.6374	3.6375	3.6374	3.6365	3.6366	3.6367	3.6362	3.6366	3.6359	...	23.1	23.1	23.0	22.9	22.9	22.9
2	8:00:50	3.6374	3.6375	3.6374	3.6366	3.6365	3.6366	3.6362	3.6366	3.6359	...	23.1	23.1	23.0	22.9	22.9	22.9
3	8:00:51	3.6373	3.6375	3.6373	3.6367	3.6365	3.6367	3.6361	3.6366	3.6359	...	23.1	23.1	23.0	22.9	22.9	22.9
4	8:00:52	3.6374	3.6376	3.6374	3.6367	3.6365	3.6366	3.6361	3.6366	3.6359	...	23.1	23.1	23.0	22.9	22.9	22.9

5 rows × 209 columns

[코드 44] 결측치 처리

- 셀 전압과 온도 데이터의 열의 인덱스를 확인한다.

```
# 셀 전압과 온도 데이터의 열의 인덱스 확인.
columns_test3 = test3.columns
for i in range(len(columns_test3)):
    print(i, columns_test3[i])
```

```
0 Time
1 M01CV01
2 M01CV02
3 M01CV03
4 M01CV04
5 M01CV05
6 M01CV06
```

[코드 45] 셀 전압, 온도 데이터의 인덱스 확인

- 셀 전압과 온도 데이터를 추출한다.

```
# Collect voltage and temperature data
columns_vt = columns_test3[1:209]
print(columns_vt)
df_test3 = test3[columns_vt]
df_test3.head(5)
```

```
Index(['M01CV01', 'M01CV02', 'M01CV03', 'M01CV04', 'M01CV05', 'M01CV06',
       'M01CV07', 'M01CV08', 'M01CV09', 'M01CV10',
       ...,
       'M12T01', 'M12T02', 'M13T01', 'M13T02', 'M14T01', 'M14T02', 'M15T01',
       'M15T02', 'M16T01', 'M16T02'],
      dtype='object', length=208)
```

	M01CV01	M01CV02	M01CV03	M01CV04	M01CV05	M01CV06	M01CV07	M01CV08	M01CV09	M01CV10	...	M12T01	M12T02
0	3.6374	3.6375	3.6374	3.6367	3.6366	3.6366	3.6361	3.6366	3.6359	3.6366	...	23.1	23.1
1	3.6374	3.6375	3.6374	3.6365	3.6366	3.6367	3.6362	3.6366	3.6359	3.6366	...	23.1	23.1
2	3.6374	3.6375	3.6374	3.6366	3.6365	3.6366	3.6362	3.6366	3.6359	3.6365	...	23.1	23.1
3	3.6373	3.6375	3.6373	3.6367	3.6365	3.6367	3.6361	3.6366	3.6359	3.6366	...	23.1	23.1
4	3.6374	3.6376	3.6374	3.6367	3.6365	3.6366	3.6361	3.6366	3.6359	3.6366	...	23.1	23.1

5 rows × 208 columns

[코드 46] 셀 전압, 온도 데이터 추출



- 전처리된 테스트 파일 “Test08_NG_chg.csv”을 하위폴더 “data/preprocessed/test”에 저장한다.

```
# Save the processed (train) data for later
df_test3.to_csv('./data/preprocessed/test/Test08_NG_chg.csv', index = False)
```

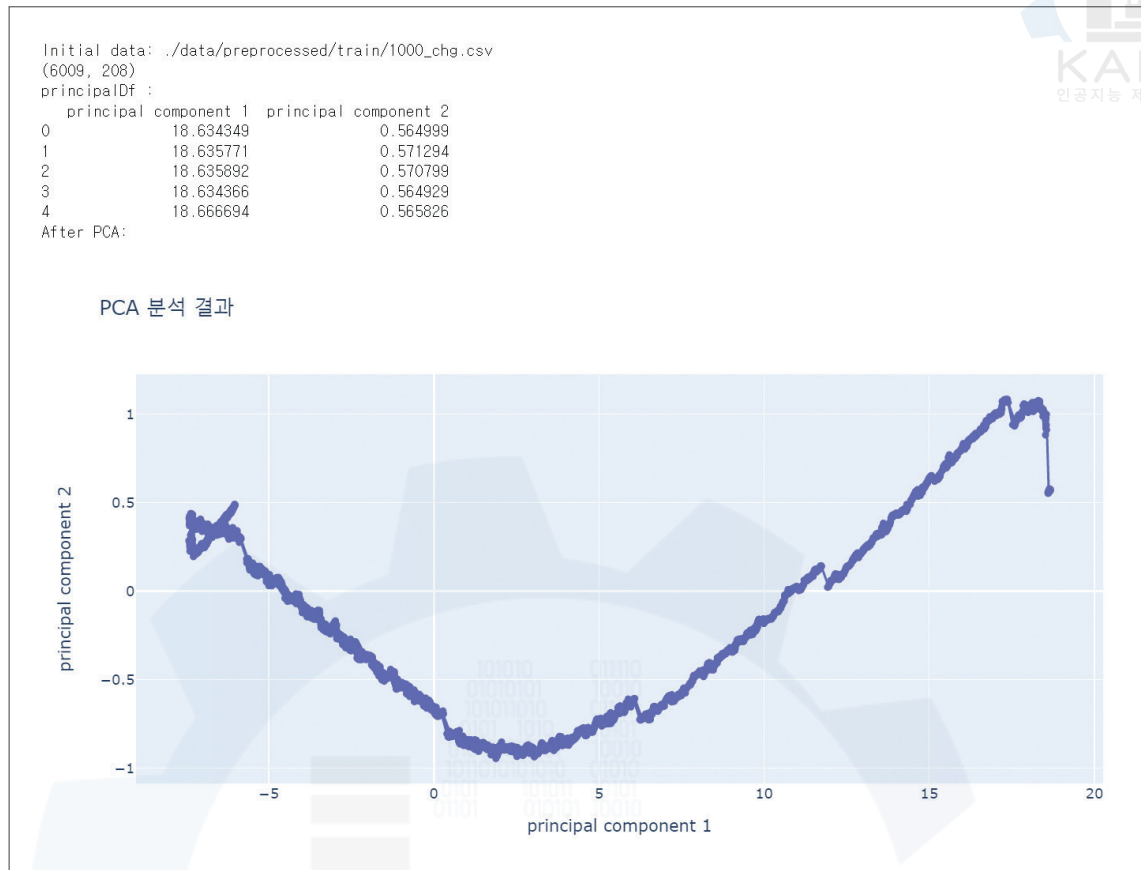
[코드 47] 전처리된 테스트 파일 저장

[단계 ②] 데이터 구조 및 특징 탐색

②-1. 데이터 구조 및 특징 탐색

- 데이터의 구조적 특징을 확인하기 위하여 주성분 분석(PCA)을 수행한다.
- 다음의 PCA 코드로 샘플 학습 데이터 “1000_chg.csv”에 적용한 결과는 다음 아래 그림에 나타나 있다. 결과를 보면 몇몇 데이터 점을 제외하고는 성분 1과 성분 2 사이에 대략 일정한 규칙적인 함수 관계를 유지하고 있음을 알 수 있다. 즉, 데이터의 복잡성이 낮고, 데이터에 잡음이 많지 않음을 알 수 있다.

```
# PCA
import numpy as np
import pandas as pd
from datetime import datetime
from sklearn.decomposition import PCA
# plot utilities
import matplotlib.pyplot as plt
import plotly.graph_objects as go
# PCA 차원 축소 테스트
df_pca = pd.read_csv('./data/preprocessed/train/1000_chg.csv')
print("Initial data: ./data/preprocessed/train/1000_chg.csv")
print(df_pca.shape)
max_column = df_pca.shape[0]
max_row = df_pca.shape[1]
x = df_pca
pca = PCA(n_components=2)
principalComponents = pca.fit_transform(x)
principalDf = pd.DataFrame(data = principalComponents,
                           columns = ['principal component 1', 'principal component 2'])
print("principalDf : ")
print(principalDf.head(5))
print("After PCA:")
pca_1 = np.array(principalDf['principal component 1'])
pca_2 = np.array(principalDf['principal component 2'])
# Create traces
fig = go.Figure()
fig.add_trace(go.Scatter(x = pca_1,
                        y = pca_2,
                        mode = 'lines+markers',
                        name = 'PCA'))
fig.update_layout(title = 'PCA 분석 결과',
                  xaxis_title = 'principal component 1',
                  yaxis_title = 'principal component 2')
fig.show()
```



[코드 48] 주성분 분석(PCA) 결과

[단계 ③] 학습, 테스트 데이터 선택

③-1. 학습, 테스트 데이터 선택

- 학습 데이터로는 길이가 적당한 정상적인 데이터로서 배터리 충전 데이터 중 하나인 “1000_chg.csv”을 택한다.
- 이상 감지 테스트를 위하여 불량이 있는 테스트 데이터를 선택하여 MTadGAN 테스트를 수행한다. 본 가이드북에서 사용될 테스트 데이터 파일은 “Test07_NG_dchg.csv”인데 이 데이터는 센서와이어 불량으로 인해 전압 이상을 보인다.

[단계 ④] AI모델 알고리즘 선택

④-1. AI모델 알고리즘 선택

- AI모델 알고리즘으로는 기존의 TadGAN 방법을 다변량 데이터로 확장한 MTadGAN 알고리즘을 주된 알고리즘으로 택하고 데이터의 차원을 축소하기 위하여 PCA 알고리즘을 초기 데이터 처리와 결합한다. MTadGAN 알고리즘을 사용하기 위해서는 Tensorflow 패키지를 설치하여야한다.
- Tensorflow 및 분석에 필요한 패키지를 설치한다.



```
!pip install tensorflow==2.5.0
!pip install graphviz==0.20.1
!pip install pydot==1.4.2
!pip install pydotplus==2.0.2
!pip install pyts==0.12.0
```

```
Requirement already satisfied: tensorflow==2.5.0 in c:\users\user\anaconda3\envs\kamp\lib\site-packages (2.5.0)
Requirement already satisfied: flatbuffers~=1.12.0 in c:\users\user\anaconda3\envs\kamp\lib\site-packages (from tensorflow==2.5.0) (1.12)
Requirement already satisfied: astunparse~=1.6.3 in c:\users\user\anaconda3\envs\kamp\lib\site-packages (from tensorflow==2.5.0) (1.6.3)
Requirement already satisfied: keras-preprocessing~=1.1.2 in c:\users\user\anaconda3\envs\kamp\lib\site-packages (from tensorflow==2.5.0) (1.1.2)
Requirement already satisfied: google-pasta~=0.2 in c:\users\user\anaconda3\envs\kamp\lib\site-packages (from tensorflow==2.5.0) (0.2.0)
Requirement already satisfied: typing-extensions~=3.7.4 in c:\users\user\anaconda3\envs\kamp\lib\site-packages (from tensorflow==2.5.0) (3.7.4.3)
Requirement already satisfied: opt-einsum~=3.3.0 in c:\users\user\anaconda3\envs\kamp\lib\site-packages (from tensorflow==2.5.0) (3.3.0)
Requirement already satisfied: tensorflow-estimator<2.6.0,>=2.5.0rc0 in c:\users\user\anaconda3\envs\kamp\lib\site-packages (from tensorflow==2.5.0) (2.5.0)
```

[코드 49] 패키지 설치

- 데이터에 대한 학습과 테스트에 필요한 라이브러리를 호출한다. 대표적인 라이브러리로는 tensorflow 이외에 데이터를 편리하게 다루기 위한 라이브러리로서 pandas, numpy가 있고 그래픽 도구로서는 plotly, matplotlib 등이 있으며 다양한 기계학습 기타 통계 계산 도구로서 sklearn(scikit-learn)이 있다.
- pandas: 이차원의 데이터를 이름이 부여된 칼럼별로 테이블 형식으로 저장하고 편리하게 조작할 수 있도록 해준다.
- numpy: 다차원의 배열로 이루어진 수치 데이터를 편리하게 다룰 수 있도록 한다.



```
import os
import sys
import numpy as np
import pandas as pd
# we will only import certain module from those libraries
from mpl_toolkits.mplot3d import Axes3D
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import MinMaxScaler
from sklearn.decomposition import PCA
from sklearn.covariance import EllipticEnvelope
from sklearn.ensemble import IsolationForest
from sklearn.impute import SimpleImputer
from datetime import datetime
import plotly.graph_objects as go
import plotly.express as px
import math
from plotly.subplots import make_subplots
import matplotlib.pyplot as plt
from random import randrange
import argparse
import collections
import tensorflow as tf
import logging
from tensorflow.keras import backend as K
from tensorflow.keras.utils import plot_model
from tensorflow.keras.layers import Input
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Bidirectional, LSTM, Flatten, Dense, Reshape, UpSampling1D, TimeDistributed
from tensorflow.keras.layers import Activation, Conv1D, LeakyReLU, Dropout, Add, Layer
from tensorflow.compat.v1.keras.layers import CuDNNLSTM
from tensorflow.keras.optimizers import Adam
import pydot
import pydotplus
from pydotplus import graphviz
from scipy import stats
from scipy import integrate
from scipy.optimize import fmin
from pyts.metrics import dtw
from pandas.plotting import register_matplotlib_converters
```

[코드 50] 라이브러리 호출

[단계 ⑤] AI모델 학습

⑤-1. AI모델 학습

- 먼저 MTadGAN을 이용하기 위해 필요한 파라미터를 설정한다. 중요한 파라미터로는 win_size, features_dim이 있는데 win_size 파라미터는 시계열 데이터에서 MTadGAN 모형으로 입력하는 데이터의 시간 윈도우의 크기를 나타낸다. 본 가이드북에서는 win_size=10으로 설정한다. 한편 features_dim 파라미터는 MTadGAN 모형에서 받아들이는 데이터 피쳐의 차원수를 말하는데 여기서는 features_dim=3으로 설정한다. 따라서 원래 데이터의 개수(차원수)는 약 200 쯤 되므로 PCA 알고리즘을 써서 이를 3 차원으로 축소한다. 이에 더하여 Dictionary 자료형 변수로 params를 정의하여 여러 가지 파라미터를 정의하는데 예를 들면 params['epochs']은 MTadGAN 모형 학습(training)에서의 반복 횟수를 나타내고 params['learning_rate']는 MTadGAN 모형 학습에서의 학습률을 나타내며 params['latent_dim']는 입력 데이터 벡터를 부호화(encoding) 할 때의 숨은(잠재) 공간의 차원을 표시한다.

```
#
# hyper parameters
#
win_size = 10
features_dim = 3
feat_dim = features_dim
params = {}
params['plot_network'] = True
params['epochs'] = 30
params['batch_size'] = 64
params['n_critic'] = 5
params['learning_rate'] = 0.00005
params['latent_dim'] = 20
params['shape'] = [win_size, features_dim]
params['encoder_input_shape'] = [win_size, features_dim]
params['encoder_reshape_shape'] = [20, 1]
params['generator_input_shape'] = [20, 1]
params['generator_reshape_shape'] = [win_size, 1]
params['critic_x_input_shape'] = [win_size, features_dim]
params['critic_z_input_shape'] = [20, 1]
print("win_size = %d, features_dim = %d" % (win_size, features_dim))
```

```
win_size = 10, features_dim = 3
```

[코드 51] 하이퍼파라미터 설정

- 학습 데이터 파일 “1000_chg.csv”을 지정하여 args.signal_file이라는 변수를 저장하며 기타 변수를 정한다. 이상값의 파일이 있으면 args.anomaly_file이라는 변수에 저장한다. args.mode = ‘train’은 학습 모드를 나타낸다.



```
arguments=collections.namedtuple('Args',
    'signal_file timest_form anomaly_file mode aggregate_interval regate_interval')

args=arguments(signal_file = './data/preprocessed/train/1000_chg.csv',
    timest_form = 0,
    anomaly_file = "",
    mode = 'train',
    aggregate_interval = 1,
    regate_interval = 1)

print("arguments")
print("-----")
print("args.timest_form :", args.timest_form)
print("args.signal_file :", args.signal_file)
print("args.anomaly_file :", args.anomaly_file)
print("args.mode :", args.mode)
print("args.aggregate_interval :", args.aggregate_interval)
print("-----\n")
```

```
arguments
-----
args.timest_form : 0
args.signal_file : ./data/preprocessed/train/1000_chg.csv
args.anomaly_file :
args.mode : train
args.aggregate_interval : 1
-----
```

[코드 52] 학습 데이터 파일 및 기타 변수 설정

- 학습 데이터에 대한 차분 함수 `diff_smooth_df()`를 정의한다. 이 함수는 입력 데이터 값을 일정한 시차(`diffs_n>1`)로 차이값을 구하거나 인접한 시점의 데이터 값들을 결합하여 데이터를 스무드하게 만드는 역할을 한다. 아래 코드에서는 `diffs_n=0` 등으로 지정하므로 차분함수가 원래 데이터를 그대로 두게 된다.



```

diffs_n = 0 # take differences
lags_n = 0 # include lags in the feature vector
smooth_n = 0 # smooth the latest values to be included in the feature vector
print("diffs_n: ", diffs_n)
#
def diff_smooth_df(df, lags_n, diffs_n, smooth_n, diffs_abs=False, abs_features=False):
    """Given a pandas dataframe preprocess it to take differences, add smoothing, and lags as specified.
    """
    if diffs_n >= 1:
        # take differences
        df = df.diff(diffs_n).dropna()
        # abs diffs if defined
        if diffs_abs == True:
            df = abs(df)
    if smooth_n >= 2:
        # apply a rolling average to smooth out the data a bit
        df = df.rolling(smooth_n).mean().dropna()
    if lags_n >= 1:
        # for each dimension add a new columns for each of lags_n lags of the differenced and smoothed values for that dimension
        df_columns_new = [f'{col}_lag{n}' for n in range(lags_n+1) for col in df.columns]
        df = pd.concat([df.shift(n) for n in range(lags_n+1)], axis=1).dropna()
        df.columns = df_columns_new
    # sort columns to have lagged values next to each other for clarity when looking at the feature vectors
    df = df.reindex(sorted(df.columns), axis=1)
    # abs all features if specified
    if abs_features == True:
        df = abs(df)
    return df
print("import diff_smooth_df() : Done ! ")

```

```

diffs_n: 0
import diff_smooth_df : Done !

```

[코드 53] 데이터 차분 함수 정의

- PCA 알고리즘을 적용하여 학습 데이터의 차원을 3차원으로 축소한다.



```
##### PCA reduction of data #####
signal_path = args.signal_file
df_train_0 = pd.read_csv(signal_path)
data_0 = df_train_0 # copy
# preprocess or 'featurize' the training data
data_1 = diff_smooth_df(data_0, lags_n, diffs_n, smooth_n)
pca = PCA(n_components=features_dim)
data = pca.fit_transform(data_1) # Dimensional reduction of Train Data by PCA
df_1 = []
for i in range(len(data)):
    row = [i+1] # attach index of integers
    for jj in range(features_dim):
        row.append(data[i][jj])
    df_1.append(row)
df = pd.DataFrame(df_1)
columns_new = ['date']
for i in range(1, features_dim+1):
    pca_i = 'pca_%s' % str(i)
    columns_new.append(pca_i)
df.columns = columns_new
print("After PCA reduction df.shape = ", df.shape)
print("-----")
print(df.head(5))
print("-----")
print("\n")
```

After PCA reduction df.shape = (6009, 4)

	date	pca_1	pca_2	pca_3
0	1	18.488237	0.590387	-0.223252
1	2	18.489667	0.596742	-0.252020
2	3	18.489788	0.596241	-0.252126
3	4	18.488254	0.590316	-0.223271
4	5	18.520762	0.590422	-0.187038

[코드 54] 주성분 분석(PCA)로 차원 축소 후 결과 확인

- 입력 학습 데이터의 시간-집합화(time-aggregate) 과정을 수행하는 함수 `time_segments_aggregate()`를 정의한다. 이는 위에서 정의한 변수 `args.aggregate_interval`의 값이 1보다 클 때 그 구간 안의 데이터를 평균하거나하여 데이터를 조정하는 함수이다. 여기에서는 `args.aggregate_interval=1`로 설정한다.



```
def time_segments_aggregate(X, interval, time_column, method=['mean']):
    """Aggregate values over given time span.
    Args:
        X (ndarray or pandas.DataFrame): N-dimensional sequence of values.
        interval (int): Integer denoting time span to compute aggregation of.
        time_column (int): Column of X that contains time values.
        method (str or list):
            Optional. String describing aggregation method or list of strings describing
            multiple aggregation methods. If not given, 'mean' is used.
    Returns:
        ndarray, ndarray:
            * Sequence of aggregated values, one column for each aggregation method.
            * Sequence of index values (first index of each aggregated segment).
    """
    if isinstance(X, np.ndarray):
        X = pd.DataFrame(X)
    X = X.sort_values(time_column).set_index(time_column)
    if isinstance(method, str):
        method = [method]
    start_ts = X.index.values[0]
    max_ts = X.index.values[-1]
    values = list()
    index = list()
    while start_ts <= max_ts:
        end_ts = start_ts + interval
        subset = X.loc[start_ts:end_ts-1]
        aggregated = [
            getattr(subset, agg)(skipna=True).values
            for agg in method
        ]
        values.append(np.concatenate(aggregated))
        index.append(start_ts)
        start_ts = end_ts
    return np.asarray(values), np.asarray(index)
```

[코드 55] 시간-집합화 함수 정의

- time_segments_aggregate() 함수를 작용하고 이들 값을 [-1,+1] 사이의 값으로 정규화한다.

```

#
# aggregate time segments
#
X, index = time_segments_aggregate(df, interval=args.aggregate_interval,
                                   time_column='date')
print("signal data (after time_segments_aggregate) = ", X.shape)
print("-----")
print(X[:5])
print("-----")
print("\n")
#
# apply min_max scaler
#
X = SimpleImputer().fit_transform(X)
X = MinMaxScaler(feature_range=(-1, 1)).fit_transform(X)
print("X (after MinMaxScaler) = ", X.shape)
print("-----")
print(X[:5])
print("-----")
print("\n")
X_norm = X

```

```

signal data (after time_segments_aggregate) = (6009, 3)

```

```

-----
[[18.6343489  0.56499908 0.26141038]
 [18.63577092 0.57129376 0.26365645]
 [18.63589171 0.57079932 0.26356652]
 [18.63436617 0.56492876 0.26139721]
 [18.6669403  0.56582562 0.25559404]]

```

```

X (after MinMaxScaler) = (6009, 3)

```

```

-----
[[0.99721634 0.48867576 0.26892606]
 [0.99732535 0.49487412 0.27233804]
 [0.99733461 0.49438724 0.27220142]
 [0.99721766 0.48860651 0.26890604]
 [0.99969584 0.48948965 0.2600905 ]]

```

[코드 56] 시간 집합화 및 데이터 정규화

- 학습 데이터를 AI 알고리즘이 요구하는 입력 윈도우 길이(win_size)에 맞게 묶는 함수인 `rolling_window_sequences()` 함수를 정의한다. 즉, 만일 win_size=10이면 연속한 10개의 데이터 값을 벡터로 묶어서 입력값으로 사용한다.



```
def rolling_window_sequences(X, index, window_size, target_size, step_size,
                             target_column, drop=None, drop_windows=False):
    """Create rolling window sequences out of time series data.
    The function creates an array of input sequences and an array of target sequences
    by rolling over the input sequence with a specified window. Optionally, certain
    values can be dropped from the sequences.
    Args:
        X (ndarray): N-dimensional sequence to iterate over.
        index (ndarray): Array containing the index values of X.
        window_size (int): Length of the input sequences.
        target_size (int): Length of the target sequences.
        step_size (int):
            Indicating the number of steps to move the window forward each round.
        target_column (int): Indicating which column of X is the target.
        drop (ndarray or None or str or float or bool):
            Optional. Array of boolean values indicating which values of X are invalid,
            or value indicating which value should be dropped. If not given,
            'None' is used.
        drop_windows (bool): Optional. Indicates whether the dropping
            functionality should be enabled. If not given, 'False' is used.
    Returns:
        ndarray, ndarray, ndarray, ndarray:
            * input sequences.
            * target sequences.
            * first index value of each input sequence.
            * first index value of each target sequence.
    """
    out_X = list()
    out_y = list()
    X_index = list()
    y_index = list()
    target = X[:, target_column]
    if drop_windows:
        if hasattr(drop, '__len__') and (not isinstance(drop, str)):
            if len(drop) != len(X):
                raise Exception('Arrays `drop` and `X` must be of the same length.')
        else:
            if isinstance(drop, float) and np.isnan(drop):
                drop = np.isnan(X)
            else:
                drop = X == drop
    start = 0
    max_start = len(X) - window_size - target_size + 1
    while start < max_start:
        end = start + window_size
        if drop_windows:
            drop_window = drop[start:end+target_size]
            to_drop = np.where(drop_window)[0]
            if to_drop.size:
                start += to_drop[-1] + 1
                continue
        out_X.append(X[start:end])
        out_y.append(target[end:end+target_size])
        X_index.append(index[start])
        y_index.append(index[end])
        start = start + step_size
    return np.asarray(out_X), np.asarray(out_y), np.asarray(X_index), np.asarray(y_index)
```

[코드 57] 입력 윈도우 묶음 생성 함수 정의

- 학습 데이터에 `rolling_window_sequences()` 함수를 적용하여 MTadGAN 알고리즘에 입력할 수 있도록 만든다. 즉, MTadGAN에서는 시계열 데이터를 모형으로 입력할 때에 특정한 시점에 대하여 일정한 윈도우 길이(`win_size`)만큼 묶어서 입력하게 된다. `rolling_window_sequences()` 함수는 입력 윈도우 길이(`win_size`)에 맞게 모든 시점에 대하여 묶는 함수이다. 만일 `win_size=10`이면 모든 시점에 대하여 각각 근처의 연속한 10개의 데이터 값을 벡터로 묶어서 입력값으로 사용할 수 있게 한다.

```
# rolling window sequences
#
X, y, X_index, y_index=rolling_window_sequences(X, index,
        window_size=win_size, target_size=1, step_size=1, target_column=0)
print("X shape (after rolling_window_seq): {}".format(X.shape))
print("X : ")
print(X[:3, :5])
print("X index shape: {}".format(X_index.shape))
print("y shape: {}".format(y.shape))
print("y : ")
print(y[:5])
print("y index shape: {}".format(y_index.shape))
print("\n");
```

```
X shape (after rolling_window_seq): (5999, 10, 3)
X :
[[[0.99721634 0.48867576 0.26892606]
  [0.99732535 0.49487412 0.27233804]
  [0.99733461 0.49438724 0.27220142]
  [0.99721766 0.48860651 0.26890604]
  [0.99969584 0.48948965 0.2600905 ]]

  [[0.99732535 0.49487412 0.27233804]
  [0.99733461 0.49438724 0.27220142]
  [0.99721766 0.48860651 0.26890604]
  [0.99969584 0.48948965 0.2600905 ]
  [0.99729103 0.49165725 0.27192521]]

  [[0.99733461 0.49438724 0.27220142]
  [0.99721766 0.48860651 0.26890604]
  [0.99969584 0.48948965 0.2600905 ]
  [0.99729103 0.49165725 0.27192521]
  [0.99741076 0.49573991 0.26903184]]]
X index shape: (5999,)
y shape: (5999, 1)
y :
[[0.99860594]
 [0.99594234]
 [0.99452801]
 [1.         ]
 [0.9959476 ]]
y index shape: (5999,)
```

[코드 58] 학습 데이터의 윈도우 묶음

- GPU가 있는지 체크한다. GPU가 있을 시 GPU 정보가 나오며, GPU가 없을 시 아래와 같은 결과가 나오게 된다.



```
# Check gpu envrionmental
gpus = tf.config.experimental.list_physical_devices('GPU')
if gpus:
    try:
        for gpu in gpus:
            tf.config.experimental.set_memory_growth(gpu, True)
    except RuntimeError as e:
        print(e)
print(gpus)
```

```
[]
```

[코드 59] 설치된 GPU 목록 확인

- MTadGAN에 필요한 함수 중 하나로서 RandomWeightedAverage() 클래스를 정의하는데 이는 입력 데이터와 예측된 입력 데이터를 정해진 비율로 선형 결합하는 역할을 한다.

```
class RandomWeightedAverage(Layer):
    def __init__(self, batch_size):
        """initialize Layer
        Args:
            batch_size: 64
        """
        super().__init__()
        self.batch_size = batch_size
    def call(self, inputs, **kwargs):
        """calculate random weighted average
        Args:
            inputs[0] x: original input
            inputs[1] x_: predicted input
        """
        alpha = K.random_uniform((self.batch_size, 1, 1))
        return (alpha * inputs[0]) + ((1-alpha) * inputs[1])
print("RandomWeightedAverage() Class defined, Done! ")
```

```
RandomWeightedAverage() Class defined, Done!
```

[코드 60] Random Weighted Average 클래스 정의

- MTadGAN의 encoder 레이어를 생성하는 함수를 정의한다. 이는 입력 데이터를 숨은 공간으로 임베딩하는 기능을 한다.



```
def build_encoder_layer(input_shape, encoder_reshape_shape):
    """build encoder layer
    Args:
        encoder_input_shape: [10, 1]
        encoder_reshape_shape: [20, 1]
    Returns:
        encoder model
    """
    x = Input(shape=input_shape)
    model = tf.keras.models.Sequential([
        Bidirectional(LSTM(units=win_size, return_sequences=True)),
        Flatten(),
        Dense(20), # 20 = self.critic_z_input_shape[0]
        Reshape(target_shape=encoder_reshape_shape)]) # (20, 1)
    return Model(x, model(x))
print("build encoder layer defined, Done!")
```

```
build encoder layer defined, Done!
```

[코드 61] encoder 레이어 생성 함수 정의

- MTadGAN의 generator 레이어를 생성하는 함수를 정의한다. 이는 숨은 공간으로부터 가상의 데이터를 생성하는 역할을 한다.

```
def build_generator_layer(input_shape, generator_reshape_shape):
    # input_shape = (20, 1) / generator_reshape_shape = (50, 1)
    x = Input(shape=input_shape)
    model = tf.keras.models.Sequential([
        Flatten(),
        Dense(win_size), # 50 originally
        Reshape(target_shape=generator_reshape_shape), # (50, 1)
        Bidirectional(LSTM(units=64, return_sequences=True), merge_mode='concat'),
        Dropout(rate=0.2),
        #UpSampling1D(size=2),
        UpSampling1D(size=1),
        Bidirectional(LSTM(units=64, return_sequences=True), merge_mode='concat'),
        Dropout(rate=0.2),
        TimeDistributed(Dense(features_dim)), # features_dim >= 1, multiple features
        Activation(activation='tanh')) # (None, 10, 1)
    return Model(x, model(x))
print("build generator layer defined, Done!")
```

```
build generator layer defined, Done!
```

[코드 62] generator 레이어 생성 함수 정의

- MTadGAN의 판별자 critic_x 레이어를 만드는 함수를 정의한다. 여기에서 critic_x는 실제의 학습 데이터와 숨은 공간에서 생성된 가상의 데이터를 구별해내는 판별자 네트워크를 말한다. 여기에서 critic_x는 실제의 학습 데이터와 숨은 공간에서 생성된 가상의 데이터를 구별해내는 판별자 네트워크를 말한다. 즉, 그림 19에서 보듯이 실제의 데이터 X에서 나오는 시계열 시퀀스들과 생성자 G(z)에서 생성된 가짜의 시계열 시퀀스들을 분별해내는 함수이다.



```

if win_size >= 30:
    k_size = 5
else:
    k_size = 2

def build_critic_x_layer(input_shape):
    """build critic_x layer
    Args:
        critic_x_input_shape: [10, 1]
    Returns:
        critic_x model
    """
    x = Input(shape=input_shape)
    model = tf.keras.models.Sequential([
        Conv1D(filters=64, kernel_size=k_size),
        LeakyReLU(alpha=0.2),
        Dropout(rate=0.25),
        Conv1D(filters=64, kernel_size=k_size),
        LeakyReLU(alpha=0.2),
        Dropout(rate=0.25),
        Conv1D(filters=64, kernel_size=k_size),
        LeakyReLU(alpha=0.2),
        Dropout(rate=0.25),
        Conv1D(filters=64, kernel_size=k_size),
        LeakyReLU(alpha=0.2),
        Dropout(rate=0.25),
        Flatten(),
        Dense(units=1)])
    return Model(x, model(x))
print("build critic_x layer defined, Done!")

```

```
build critic_x layer defined, Done!
```

[코드 63] 판별자 x 레이어 생성 함수 정의

- MTadGAN의 판별자 critic_z 레이어를 만드는 함수를 정의한다. critic_z는 실제의 학습 데이터 샘플로부터 encoding 된 벡터와 그냥 숨은 공간에서 샘플된 벡터 사이를 구별해내는 역할을 한다. 구별해내는 판별자 네트워크를 말한다. 즉, 그림 19에서 보듯이 무작위로 택한 잠재공간 내의 표본 z 와 학습 데이터 샘플로부터 생성자 $E(x)$ 를 써서 인코딩(encoding)된 표본을 분별하는 함수(네트워크)이다. 즉 그림 19에서 보듯이 critic_z는 잠재공간(embedding space) 내에서의 판별자이고 critic_x는 이와 반대로 실제 데이터 공간에서의 판별자가 된다.



```
def build_critic_z_layer(input_shape):
    """build critic_z layer
    Args:
        critic_z_input_shape: [20, 1]
    Returns:
        critic_z model
    """
    x = Input(shape=input_shape)
    model = tf.keras.models.Sequential([
        Flatten(),
        Dense(units=100),
        LeakyReLU(alpha=0.2),
        Dropout(rate=0.2),
        Dense(units=100),
        LeakyReLU(alpha=0.2),
        Dropout(rate=0.2),
        Dense(units=1)])
    return Model(x, model(x))
print("build critic_z layer defined, Done!")
```

```
build critic_z layer defined, Done!
```

[코드 64] 판별자 z 레이어 생성 함수 정의

- MTadGAN의 Wasserstein 로스 함수를 정의한다. Wasserstein 로스 함수는 두 확률 분포 사이의 거리를 주는 함수로서 판별자 네트워크의 학습에 쓰이는 로스함수이다.

```
def wasserstein_loss(y_true, y_pred):
    """calculate wasserstein loss
    Args:
        y_true: real value
        y_pred: predic value
    Returns:
        loss: wasserstein loss
    """
    return K.mean(y_true*y_pred)
print("Wasserstein_loss defined, Done!")
```

```
Wasserstein_loss defined, Done!
```

[코드 65] Wasserstein Loss 함수 정의

- MTadGAN에서 필요로 하는 checkpoints 디렉터리, 기타 앞에서 정의한 네트워크 파라미터 값을 복사한다. checkpoints 디렉터리는 학습된 MTadGAN 모델을 저장하는 데에 쓰인다.

```
#
ckpt_dir = os.path.join(os.getcwd(), 'checkpoints')
os.makedirs(ckpt_dir, exist_ok=True)
network_dir = os.path.join(os.getcwd(), 'networks')
os.makedirs(network_dir, exist_ok=True)
# plot network
plot_network = params[plot_network]
# Training Parameters
batch_size = params[batch_size]
n_critics = params[n_critic]
epochs = params[epochs]
# Layer Parameters
shape = params[shape]
window_size = shape[0]
feat_dim = shape[1]
latent_dim = params[latent_dim]
encoder_input_shape = params[encoder_input_shape]
generator_input_shape = params[generator_input_shape]
critic_x_input_shape = params[critic_x_input_shape]
critic_z_input_shape = params[critic_z_input_shape]
encoder_reshape_shape = params[encoder_reshape_shape]
generator_reshape_shape = params[generator_reshape_shape]

print('init MTadGAN')
print("latent_dim= ", latent_dim)
print("shape= ", shape)
print("encoder_input_shape= ", encoder_input_shape)
print("generator_input_shape= ", generator_input_shape)
print("critic_x_input_shape= ", critic_x_input_shape)
print("critic_z_input_shape= ", critic_z_input_shape)
print("encoder_reshape_shape= ", encoder_reshape_shape)
print("generator_reshape_shape= ", generator_reshape_shape)

init MTADGAN
latent_dim= 20
shape= [10, 3]
encoder_input_shape= [10, 3]
generator_input_shape= [20, 1]
critic_x_input_shape= [10, 3]
critic_z_input_shape= [20, 1]
encoder_reshape_shape= [20, 1]
generator_reshape_shape= [10, 1]
```

[코드 66] Checkpoints 디렉터리, 기타 네트워크 파라미터 값 복사

- MTadGAN의 encoder, generator 및 판별자(critic_x, critic_z) 네트워크의 인스턴스를 정의한다. 동시에 학습 계산을 위한 최적화 알고리즘으로 Adam 함수를 도입한다. 학습률의 값은 learning_rate = 0.0005로 정의한다.



```
learning_rate = 0.0005
encoder = build_encoder_layer(input_shape=encoder_input_shape,
                              encoder_reshape_shape=encoder_reshape_shape)
generator = build_generator_layer(input_shape=generator_input_shape,
                                  generator_reshape_shape=generator_reshape_shape)
critic_x = build_critic_x_layer(input_shape=critic_x_input_shape)
critic_z = build_critic_z_layer(input_shape=critic_z_input_shape)
optimizer = tf.keras.optimizers.Adam(learning_rate)
print("encoder generator critic_x critic_z and optimizer instance defined: Done!")
```

```
encoder generator critic_x critic_z and optimizer instance defined: Done!
```

[코드 67 네트워크 인스턴스 정의 및 Adam 함수 도입]

- 판별자 네트워크와 encoder-generator 네트워크의 입출력 구조를 명시한다.

```
z = Input(shape=(latent_dim, 1))
x = Input(shape=shape)
x_ = generator(z)
z_ = encoder(x)
fake_x = critic_x(x_)
valid_x = critic_x(x)
interpolated_x = RandomWeightedAverage(batch_size)([x, x_])
critic_x_model = Model(inputs=[x, z], outputs=[valid_x, fake_x, interpolated_x])
fake_z = critic_z(z_)
valid_z = critic_z(z)
interpolated_z = RandomWeightedAverage(batch_size)([z, z_])
critic_z_model = Model(inputs=[x, z], outputs=[valid_z, fake_z, interpolated_z])
z_gen = Input(shape=(latent_dim, 1))
x_gen_ = generator(z_gen)
x_gen = Input(shape=shape)
z_gen_ = encoder(x_gen)
x_gen_rec = generator(z_gen_)
fake_gen_x = critic_x(x_gen_)
fake_gen_z = critic_z(z_gen_)
encoder_generator_model = Model([x_gen, z_gen], [fake_gen_x, fake_gen_z, x_gen_rec])
```

[코드 68] 판별자 네트워크와 encoder-generator 네트워크 입출력 구조 명시

- 이미 critic_x 모형, critic_z 모형 그리고 encoder-generator 모형의 세 가지가 학습되어 저장되어 있으면 이를 불러오고, 네트워크 구조를 그래픽 파일로 저장하며 입출력 구조를 출력한다.



```

if os.path.isfile(os.path.join(ckpt_dir, 'critic_x_model.h5')):
    print("load critic_x weights")
    critic_x_model.load_weights(os.path.join(ckpt_dir, 'critic_x_model.h5'))
if os.path.isfile(os.path.join(ckpt_dir, 'critic_z_model.h5')):
    print("load critic_z weights")
    critic_z_model.load_weights(os.path.join(ckpt_dir, 'critic_z_model.h5'))
if os.path.isfile(os.path.join(ckpt_dir, 'encoder_generator_model.h5')):
    print("load encoder_generator weights")
    encoder_generator_model.load_weights(os.path.join(
        ckpt_dir,
        'encoder_generator_model.h5'))
critic_x_model.summary()
critic_z_model.summary()
encoder_generator_model.summary()

if plot_network:
    plot_model(critic_x_model,
               to_file=os.path.join(network_dir, 'critic_x_model_tf2.png'),
               show_shapes=True,
               expand_nested=True)
    plot_model(critic_z_model,
               to_file=os.path.join(network_dir, 'critic_z_model_tf2.png'),
               show_shapes=True,
               expand_nested=True)
    plot_model(encoder_generator_model,
               to_file=os.path.join(network_dir, 'enc_gen_model_tf2.png'),
               show_shapes=True,
               expand_nested=True)
print("critic_x_model critic_z_model and encode_generator_model defined: Done!")

```

Model: "model_25"			
Layer (type)	Output Shape	Param #	Connected to
=====			
input_29 (InputLayer)	[(None, 20, 1)]	0	
input_30 (InputLayer)	[(None, 10, 3)]	0	
model_22 (Functional)	(None, 10, 3)	133205	input_29[0][0]
model_23 (Functional)	(None, 1)	25601	model_22[0][0] input_30[0][0]
random_weighted_average_6 (Rand	(64, 10, 3)	0	input_30[0][0] model_22[0][0]
=====			
Total params: 158,806			
Trainable params: 158,806			
Non-trainable params: 0			



Model: "model_26"			
Layer (type)	Output Shape	Param #	Connected to
input_30 (InputLayer)	[(None, 10, 3)]	0	
input_29 (InputLayer)	[(None, 20, 1)]	0	
model_21 (Functional)	(None, 20, 1)	5140	input_30[0][0]
model_24 (Functional)	(None, 1)	12301	model_21[0][0] input_29[0][0]
random_weighted_average_7 (Rand (64, 20, 1))		0	input_29[0][0] model_21[0][0]
Total params: 17,441 Trainable params: 17,441 Non-trainable params: 0			
Model: "model_27"			
Layer (type)	Output Shape	Param #	Connected to
input_31 (InputLayer)	[(None, 20, 1)]	0	
input_32 (InputLayer)	[(None, 10, 3)]	0	
model_22 (Functional)	(None, 10, 3)	133205	input_31[0][0] model_21[1][0]
model_21 (Functional)	(None, 20, 1)	5140	input_32[0][0]
model_23 (Functional)	(None, 1)	25601	model_22[1][0]
model_24 (Functional)	(None, 1)	12301	model_21[1][0]
Total params: 176,247 Trainable params: 176,247 Non-trainable params: 0			
critic_x_model critic_z_model and encode_generator_model defined: Done!			

[코드 69] 기존 학습 모형 체크 및 네트워크 입출력 구조 표시

- critic_x 네트워크를 학습하는 과정을 수행하는 함수를 정의한다.
로스함수를 정의하고 gradient 계산함수, 그리고 Adam 최적화 함수를 포함한다.



```
@tf.function
def critic_x_train_on_batch(x, z, valid, fake, delta):
    with tf.GradientTape() as tape:
        (valid_x, fake_x, interpolated) = critic_x_model(inputs=[x, z], training=True)
        with tf.GradientTape() as gp_tape:
            gp_tape.watch(interpolated)
            pred = critic_x(interpolated, training=True)
            grads = gp_tape.gradient(pred, interpolated)[0]
            grads = tf.square(grads)
            ddx = tf.sqrt(1e-8 + tf.reduce_sum(grads, axis=np.arange(1, len(grads.shape))))
            gp_loss = tf.reduce_mean((ddx-1.0)**2)
            loss = tf.reduce_mean(wasserstein_loss(valid, valid_x))
            loss += tf.reduce_mean(wasserstein_loss(fake, fake_x))
            loss += gp_loss*10.0
        gradients = tape.gradient(loss, critic_x_model.trainable_weights)
        optimizer.apply_gradients(zip(gradients, critic_x_model.trainable_weights))
    return loss
print("critic_x_train_on_batch defined: Done!")
```

```
critic_x_train_on_batch defined: Done!
```

[코드 70] critic_x에 대한 학습 과정 정의

- critic_z 네트워크를 학습하는 과정을 수행하는 함수를 정의한다. 로스함수를 정의하고 gradient 계산함수, 그리고 Adam 최적화 함수를 포함한다.

```
@tf.function
def critic_z_train_on_batch(x, z, valid, fake, delta):
    with tf.GradientTape() as tape:
        (valid_z, fake_z, interpolated) = critic_z_model(inputs=[x, z], training=True)
        with tf.GradientTape() as gp_tape:
            gp_tape.watch(interpolated)
            pred = critic_z(interpolated, training=True)
            grads = gp_tape.gradient(pred, interpolated)[0]
            grads = tf.square(grads)
            ddx = tf.sqrt(1e-8 + tf.reduce_sum(grads, axis=np.arange(1, len(grads.shape))))
            gp_loss = tf.reduce_mean((ddx-1.0)**2)
            loss = tf.reduce_mean(wasserstein_loss(valid, valid_z))
            loss += tf.reduce_mean(wasserstein_loss(fake, fake_z))
            loss += gp_loss*10.0
        gradients = tape.gradient(loss, critic_z_model.trainable_weights)
        optimizer.apply_gradients(zip(gradients, critic_z_model.trainable_weights))
    return loss
print("critic_z_train_on_batch defined: Done!")
```

```
critic_z_train_on_batch defined: Done!
```

[코드 71] critic_z 네트워크에 대한 로스 함수 및 gradient 정의

- encoder-generator 네트워크를 학습하는 과정을 수행하는 함수를 정의한다. 로스 함수를 정의하고 gradient 함수, 그리고 Adam 최적화 함수를 포함한다.



```
@tf.function
def enc_gen_train_on_batch(x, z, valid):
    with tf.GradientTape() as tape:
        (fake_gen_x, fake_gen_z, x_gen_rec) = encoder_generator_model(inputs=[x, z], training=True)
        x = tf.squeeze(x)
        x_gen_rec = tf.squeeze(x_gen_rec)
        loss = tf.reduce_mean(wasserstein_loss(valid, fake_gen_x))
        loss += tf.reduce_mean(wasserstein_loss(valid, fake_gen_z))
        loss += tf.keras.losses.MSE(x, x_gen_rec)*10
        loss = tf.reduce_mean(loss)
        gradients = tape.gradient(loss, encoder_generator_model.trainable_weights)
        optimizer.apply_gradients(zip(gradients, encoder_generator_model.trainable_weights))
    return loss
print("enc_gen_train_on_batch defined: Done!")
```

```
enc_gen_train_on_batch defined: Done!
```

[코드 72] encoder-generator 네트워크에 대한 로스 함수 및 gradient 정의

- 학습률 Learning rate = 0.0005, 반복 횟수 epochs=30으로 학습을 진행하며 각 epoch 끝날 때마다 로스 함수들을 출력한다. 학습 부분은 로스 함수의 최적화 과정에서 초기 네트워크 상태를 무작위 상태로부터 시작하거나 또는 기존의 네트워크의 모형을 불러와서 시작하거나 하므로 그 시작 상태에 따라 출력 결과가 실행 시마다 조금씩 달라진다. 즉, 최종적으로 학습된 네트워크 모형이 달라질 수 있는 것이다. 따라서 이렇게 조금씩 달라진 네트워크 모형을 써서 수행하는 테스트 결과 또한 실행 시마다 조금씩 그 결과가 달라진다.



```
# Train
X = X.reshape((-1, shape[0], feat_dim)) # feat_dim = features dimension
X_ = np.copy(X)
fake = np.ones((batch_size, 1), dtype=np.float32)
valid = -np.ones((batch_size, 1), dtype=np.float32)
delta = np.ones((batch_size, 1), dtype=np.float32)
for epoch in range(1, epochs+1):
    np.random.shuffle(X_)
    epoch_g_loss = []
    epoch_cx_loss = []
    epoch_cz_loss = []
    minibatches_size = batch_size * n_critics
    num_minibatches = int(X_.shape[0] // minibatches_size)
    for i in range(num_minibatches):
        minibatch = X_[i * minibatches_size:(i+1) * minibatches_size]
        # train critics
        critic_x.trainable = True
        critic_z.trainable = True
        generator.trainable = False
        encoder.trainable = False
        for j in range(n_critics):
            x = minibatch[j * batch_size:(j+1) * batch_size]
            z = np.random.normal(size=(batch_size, latent_dim, 1))
            epoch_cx_loss.append(critic_x_train_on_batch(x, z, valid, fake, delta))
            epoch_cz_loss.append(critic_z_train_on_batch(x, z, valid, fake, delta))
        critic_x.trainable = False
        critic_z.trainable = False
        generator.trainable = True
        encoder.trainable = True
        # train encoder, generator
        epoch_g_loss.append(enc_gen_train_on_batch(x, z, valid))
    cx_loss = np.mean(np.array(epoch_cx_loss), axis=0)
    cz_loss = np.mean(np.array(epoch_cz_loss), axis=0)
    g_loss = np.mean(np.array(epoch_g_loss), axis=0)
    print('Epoch: {}/{}. [Dx loss: {}] [Dz loss: {}] [G loss: {}]'.format(epoch, epochs, cx_loss, cz_loss, g_loss))
critic_x_model.save_weights(os.path.join(ckpt_dir, 'critic_x_model.h5'), save_format='h5')
critic_z_model.save_weights(os.path.join(ckpt_dir, 'critic_z_model.h5'), save_format='h5')
encoder_generator_model.save_weights(os.path.join(ckpt_dir,
                                                    'encoder_generator_model.h5'),
                                     save_format='h5')
```

```
Epoch: 1/30, [Dx loss: 4.16777229309082] [Dz loss: 4.219563007354736] [G loss: 1.14092302323877]
Epoch: 2/30, [Dx loss: 1.2432202100753784] [Dz loss: 1.5060335397720337] [G loss: -2.2327628135681152]
Epoch: 3/30, [Dx loss: -0.2490355223417282] [Dz loss: -7.347346305847168] [G loss: 9.421393394470215]
Epoch: 4/30, [Dx loss: -2.686958074569702] [Dz loss: -11.091506004333496] [G loss: 14.346874237060547]
Epoch: 5/30, [Dx loss: 0.18424877524375916] [Dz loss: -5.175566673278809] [G loss: -4.64584493637085]
Epoch: 6/30, [Dx loss: -0.4449044167995453] [Dz loss: -5.620662212371826] [G loss: -4.304211616516113]
Epoch: 7/30, [Dx loss: -0.5848439331869507] [Dz loss: -7.014995574951172] [G loss: 7.853679180145264]
Epoch: 8/30, [Dx loss: 0.6649893622262573] [Dz loss: -8.238423347473145] [G loss: 2.1117773056030273]
Epoch: 9/30, [Dx loss: -0.2036726474761963] [Dz loss: -8.190013885498047] [G loss: -0.8092625737190247]
Epoch: 10/30, [Dx loss: -0.6961482763290405] [Dz loss: -8.470255851745605] [G loss: 3.244856119155884]
Epoch: 11/30, [Dx loss: 0.7447929382324219] [Dz loss: -8.766849517822266] [G loss: 4.3266730308532715]
Epoch: 12/30, [Dx loss: 0.3475151062011719] [Dz loss: -8.367478370666504] [G loss: 7.756348609924316]
Epoch: 13/30, [Dx loss: -1.3327996730804443] [Dz loss: -7.496187210083008] [G loss: 13.388547897338867]
Epoch: 14/30, [Dx loss: -0.7976640462875366] [Dz loss: -7.036890983581543] [G loss: 14.864120483398438]
Epoch: 15/30, [Dx loss: 0.14546538889408112] [Dz loss: -6.318492889404297] [G loss: 9.226099014282227]
Epoch: 16/30, [Dx loss: 0.6381199955940247] [Dz loss: -6.258098125457754] [G loss: 2.252939462661743]
Epoch: 17/30, [Dx loss: 0.13575565814971924] [Dz loss: -6.294454097747803] [G loss: 6.143173694610596]
Epoch: 18/30, [Dx loss: 0.4621911644935608] [Dz loss: -6.872226238250732] [G loss: 12.352283477783203]
Epoch: 19/30, [Dx loss: -0.3203960061073303] [Dz loss: -6.823994159698486] [G loss: 15.676807403564453]
Epoch: 20/30, [Dx loss: -1.2204418182373047] [Dz loss: -6.870025157928467] [G loss: 16.30488395690918]
Epoch: 21/30, [Dx loss: 0.21600930392742157] [Dz loss: -6.000976085662842] [G loss: 18.177541732788086]
Epoch: 22/30, [Dx loss: -0.22557127475738525] [Dz loss: -5.131932258605957] [G loss: 21.005769729614258]
Epoch: 23/30, [Dx loss: 0.05431414768099785] [Dz loss: -5.312427997589111] [G loss: 12.880874633789062]
Epoch: 24/30, [Dx loss: -2.135640859603882] [Dz loss: -5.6674299240112305] [G loss: 12.607781410217285]
Epoch: 25/30, [Dx loss: -1.6997179985046387] [Dz loss: -5.388360500335693] [G loss: 8.326112747192383]
Epoch: 26/30, [Dx loss: 1.0303093194961548] [Dz loss: -5.468316078186035] [G loss: 6.587855339050293]
Epoch: 27/30, [Dx loss: 0.24237880110740662] [Dz loss: -5.292451858520508] [G loss: 14.05638313293457]
Epoch: 28/30, [Dx loss: -0.5010596513748169] [Dz loss: -5.881876468658447] [G loss: 18.398862838745117]
Epoch: 29/30, [Dx loss: 1.4054419994354248] [Dz loss: -7.310657978057861] [G loss: 16.187345504760742]
Epoch: 30/30, [Dx loss: -0.9866009950637817] [Dz loss: -7.632802486419678] [G loss: 17.02708625793457]
```

[코드 73] 학습 진행 및 로스 함수 결과



[단계 ⑥] 테스트 데이터에 대한 계산 수행

⑥-1. 테스트 데이터에 대한 계산 수행

- 예측 함수 predict()를 정의한다. 이 함수는 입력 데이터에 encoder-generator 네트워크를 작용하여 재현된 데이터 y_hat을 출력하고 또한 critic_x 판별자 값을 출력한다.

```
def predict(X):
    """Predict values using the initialized object.
    Args:
        X (ndarray): N-dimensional array containing the input sequences for the model.
    Returns:
        ndarray:
            N-dimensional array containing the reconstructions for each input sequence.
        ndarray:
            N-dimensional array containing the critic scores for each input sequence.
    """
    X = X.reshape((-1, shape[0], feat_dim)) # feat_dim : feature dimension (modified from 1: yeesej)
    z_ = encoder.predict(X)
    y_hat = generator.predict(z_)
    critic = critic_x.predict(X)
    return y_hat, critic
print("predict() defined: Done!")
```

```
predict() defined: Done!
```

[코드 74] predict() 함수 정의

- 미리 준비된 테스트 데이터 파일 “./data/preprocessed/test/Test07_NG_dchg.csv”와 anomaly 파일인 “./data/preprocessed/test/Test07_NG_dchg_Label.csv”를 지정한다.

```
# Prediction test mode
# New test data file introduced
args=arguments(signal_file='./data/preprocessed/test/Test07_NG_dchg.csv',
               timest_form=0,
               anomaly_file='./data/preprocessed/test/Test07_NG_dchg_Label.csv',
               mode='predict',
               aggregate_interval=1,
               regate_interval=1)
print("arguments for prediction mode")
print("-----")
print("args.timest_form :", args.timest_form)
print("args.signal_file :", args.signal_file)
print("args.anomaly_file :", args.anomaly_file)
print("args.mode :", args.mode)
print("args.aggregate_interval :", args.aggregate_interval)
print("-----\n")
```

```
arguments for prediction mode
-----
args.timest_form : 0
args.signal_file : ./data/preprocessed/test/Test07_NG_dchg.csv
args.anomaly_file : ./data/preprocessed/test/Test07_NG_dchg_Label.csv
args.mode : predict
args.aggregate_interval : 1
-----
```

[코드 75] 테스트 데이터 파일 및 기타 변수 설정



- 테스트 데이터 “./data/preprocessed/test/Test07_NG_dchg.csv”를 불러들인다.
이를 pandas DataFrame에 저장한다.

```
file_path = args.signal_file
df_test1 = pd.read_csv(file_path) # using pandas library (pd) to read the csv file.
# After reading the file, it will be used as a Pandas DataFrame.
```

[코드 76] 테스트 데이터 불러들이기

- 학습 데이터의 경우와 마찬가지로 테스트 데이터에 대하여도 PCA를 써서 3차원으로 차원을 축소한다.

```
##### PCA reduction of data #####
data_0 = df_test1 # copy
# (lags_n=0, diffs_n=0, smooth_n=0)
data_1 = diff_smooth_df(data_0, lags_n, diffs_n, smooth_n)
pca = PCA(n_components=features_dim)
data = pca.fit_transform(data_1) # Dimensional reduction of Train Data by PCA
df_1 = []
for i in range(len(data)):
    row=[i+1] # attach index of integers
    for jj in range(features_dim):
        row.append(data[i][jj])
    df_1.append(row)
df = pd.DataFrame(df_1)
columns_new = ['date']
for i in range(1, features_dim+1):
    pca_i = 'pca_%s' % str(i)
    columns_new.append(pca_i)
df.columns=columns_new
print("After PCA reduction of test data df.shape = ", df.shape)
print("-----")
print(df.head(5))
print("-----")
print("\n")
```

After PCA reduction of test data df.shape = (4594, 4)

	date	pca_1	pca_2	pca_3
0	1	-0.412384	0.583294	0.027636
1	2	-0.406828	0.567897	0.026531
2	3	-0.368964	0.575908	0.015944
3	4	-0.430935	0.547518	0.026097
4	5	-0.425375	0.532121	0.025034

[코드 77] 주성분 분석(PCA) 차원 축소 후 결과

- MTadGAN 모형에 입력하기 위하여 테스트 데이터에 대한 집합화(aggregate)를 수행한다. time_segments_aggregate() 함수에 대해서는 [코드 55]를 참조한다.



```
# aggregate time segments
#
X, index = time_segments_aggregate(df, interval=args.aggregate_interval,
                                   time_column='date')
print("Test signal data (after time_segments_aggregate) = ", X.shape)
print("-----")
print(X[:5])
print("-----")
print("\n")
```

```
signal data (after time_segments_aggregate) = (4594, 3)
-----
[[-0.41238433  0.58329384  0.02763632]
 [-0.40682767  0.56789719  0.02653118]
 [-0.36896407  0.57590827  0.01594388]
 [-0.43093477  0.54751791  0.02609742]
 [-0.42537477  0.532121   0.02503421]]
-----
```

[코드 78] 테스트 데이터에 대한 시간-집합화

- 스케일링에 의한 정규화 과정을 수행한다. 테스트 데이터를 [-1,+1] 구간 안의 값으로 정규화한다.

```
#
# apply min_max scaler
#
X = SimpleImputer().fit_transform(X)
X = MinMaxScaler(feature_range=(-1, 1)).fit_transform(X)
print("Test X (after MinMaxScaler) = ", X.shape)
print("-----")
print(X[:5])
print("-----")
print("\n")
```

```
Test X (after MinMaxScaler) = (4594, 3)
-----
[[-0.98179729  1.         0.02048554]
 [-0.97807573  0.96581515  0.01806001]
 [-0.95271668  0.98360199 -0.0051768 ]
 [-0.9942214   0.9205675   0.017108  ]
 [-0.99049761  0.88638206  0.01477448]]
-----
```

[코드 79] 테스트 데이터 정규화

- MTadGAN의 구조에 맞게 입력할 수 있도록 테스트 데이터에 rolling_window_sequences() 함수를 작용하여 이웃하는 데이터 값으로 윈도우 길이(win_size=10)만큼씩 묶어서 저장한다.



```
# rolling window sequences
X, y, X_index, y_index = rolling_window_sequences(X, index, window_size= win_size,
                                                    target_size=1, step_size=1, target_column=0)
print("X shape (after rolling_window_seq): {}".format(X.shape))
print("X : ")
print(X[:3, :5])
print("X index shape: {}".format(X_index.shape))
print("y shape: {}".format(y.shape))
print("y : ")
print(y[:5])
print("y index shape: {}".format(y_index.shape))
print("\n");
```

```
X shape (after rolling_window_seq): (4584, 10, 3)
X :
[[[-9.81797293e-01  1.00000000e+00  2.04855453e-02]
 [-9.78075731e-01  9.65815151e-01  1.80600109e-02]
 [-9.52716680e-01  9.83601990e-01 -5.17680441e-03]
 [-9.94221404e-01  9.20567495e-01  1.71080059e-02]
 [-9.90497609e-01  8.86382057e-01  1.47744871e-02]]

 [[-9.78075731e-01  9.65815151e-01  1.80600109e-02]
 [-9.52716680e-01  9.83601990e-01 -5.17680441e-03]
 [-9.94221404e-01  9.20567495e-01  1.71080059e-02]
 [-9.90497609e-01  8.86382057e-01  1.47744871e-02]
 [-9.60544141e-01  9.71167281e-01 -9.41994990e-04]]

 [[-9.52716680e-01  9.83601990e-01 -5.17680441e-03]
 [-9.94221404e-01  9.20567495e-01  1.71080059e-02]
 [-9.90497609e-01  8.86382057e-01  1.47744871e-02]
 [-9.60544141e-01  9.71167281e-01 -9.41994990e-04]
 [-9.79474891e-01  9.25992926e-01  1.28564550e-02]]]
X index shape: (4584,)
y shape: (4584, 1)
y :
[[-0.97946451]
 [-0.99049122]
 [-0.99048524]
 [-0.97807193]
 [-0.97945812]]
y index shape: (4584,)
```

[코드 80] 테스트 데이터의 윈도우 묶음

- MTadGAN에서의 Anomaly 클래스 및 이상탐지 관련 속성 함수들을 정의한다. 여기서는 Anomaly 클래스에서 주로 쓰이는 주요한 속성함수에 대하여 간략하게 설명한다. 각 함수의 입력 변수는 생략한다.

_find_threshold() : 에러 값들로부터 최적의 threshold 값을 찾는다.

_fixed_threshold() : 입력 k 변수의 값에 따라 에러의 threshold를 구한다.

_compute_scores() : Anomaly score를 계산한다.

_find_window_sequences() : 연속된 이상 값들의 sequence를 찾는다.

find_anomalies() : 이상 값을 가지는 연속된 에러값의 sequence를 찾는다.

_compute_critic_score() : 판별자 score에서 얻는 이상 값의 배열을 계산한다.

_reconstruction_errors() : 데이터 재현 에러 값을 계산한다. 이는 입력 데이터와 $\hat{y}$ 사이의 차이값으로부터 구해진다.

score_anomalies() : 판별자 score와 재현 에러의 결합으로 최종 이상 score를 얻는다.



```
# anomalies.py (1)
```

```
"""
```

Time Series anomaly detection functions.

Some of the implementation is inspired by the paper

<https://arxiv.org/pdf/1802.04431.pdf>

```
"""
```

```
class Anomaly(object):
```

```
    def __init__(self):
```

```
        pass
```

```
    def _deltas(self, errors, epsilon, mean, std):
```

```
        """Compute mean and std deltas.
```

```
        delta_mean = mean(errors) - mean(all errors below epsilon)
```

```
        delta_std = std(errors) - std(all errors below epsilon)
```

```
        Args:
```

```
            errors (ndarray): Array of errors.
```

```
            epsilon (ndarray): Threshold value.
```

```
            mean (float): Mean of errors.
```

```
            std (float): Standard deviation of errors.
```

```
        Returns:
```

```
            float, float:
```

```
                * delta_mean.
```

```
                * delta_std.
```

```
        """
```

```
        below = errors[errors <= epsilon]
```

```
        if not len(below):
```

```
            return 0, 0
```

```
        return mean - below.mean(), std - below.std()
```

```
# anomalies.py (2)
```

```
def _count_above(self, errors, epsilon):
```

```
    """Count number of errors and continuous sequences above epsilon.
```

Continuous sequences are counted by shifting and counting the number of positions where there was a change and the original value was true, which means that a sequence started at that position.

```
    Args:
```

```
        errors (ndarray): Array of errors.
```

```
        epsilon (ndarray): Threshold value.
```

```
    Returns:
```

```
        int, int:
```

```
            * Number of errors above epsilon.
```

```
            * Number of continuous sequences above epsilon.
```

```
    """
```

```
    above = errors > epsilon
```

```
    total_above = len(errors[above])
```

```
    above = pd.Series(above)
```

```
    shift = above.shift(1)
```

```
    change = above != shift
```

```
    total_consecutive = sum(above & change)
```

```
    return total_above, total_consecutive
```

```
# anomalies.py (3)
```

```
def _z_cost(self, z, errors, mean, std):
```

```
    """Compute how bad a z value is.
```

The original formula is::

```
    (delta_mean/mean) + (delta_std/std)
```

```
    -----
    number of errors above + (number of sequences above)^2
```

which computes the \"goodness\" of 'z', meaning that the higher the value the better the 'z'.



In this case, we return this value inverted (we make it negative), to convert it into a cost function, as later on we will use `scipy.fmin` to minimize it.

Args:

`z` (ndarray): Value for which a cost score is calculated.

`errors` (ndarray): Array of errors.

`mean` (float): Mean of errors.

`std` (float): Standard deviation of errors.

Returns float: Cost of `z`.

"""

`epsilon = mean + z * std`

`delta_mean, delta_std = self._deltas(errors, epsilon, mean, std)`

`above, consecutive = self._count_above(errors, epsilon)`

`numerator = -(delta_mean / mean + delta_std / std)`

`denominator = above + consecutive ** 2`

`if denominator == 0:`

`return np.inf`

`return numerator / denominator`

anomalies.py (4)

`def _find_threshold(self, errors, z_range):`

"""Find the ideal threshold.

The ideal threshold is the one that minimizes the `z_cost` function.

`Scipy.fmin` is used to find the minimum, using the values from `z_range` as starting points.

Args:

`errors` (ndarray): Array of errors.

`z_range` (list):

List of two values denoting the range out of which the start points for the `scipy.fmin` function are chosen.

Returns: float: Calculated threshold value.

"""

`mean = errors.mean()`

`std = errors.std()`

`min_z, max_z = z_range`

`best_z = min_z`

`best_cost = np.inf`

`for z in range(min_z, max_z):`

`best = fmin(self._z_cost, z, args=(errors, mean, std), full_output=True,`

`disp=False)`

`z, cost = best[0:2]`

`if cost < best_cost:`

`best_z = z[0]`

`return mean + best_z * std`

`def _fixed_threshold(self, errors, k=3.0):`

"""Calculate the threshold.

The fixed threshold is defined as `k` standard deviations away from the mean.

Args:

`errors` (ndarray): Array of errors.

Returns:

float: Calculated threshold value.

"""

`mean = errors.mean()`

`std = errors.std()`

`return mean + k * std`

anomalies.py (5-1)

`def _find_sequences(self, errors, epsilon, anomaly_padding):`

"""Find sequences of values that are above `epsilon`.

This is done following this steps:



```

* create a boolean mask that indicates which values are above epsilon.
* mark certain range of errors around True values with a True as well.
* shift this mask by one place, filling the empty gap with a False.
* compare the shifted mask with the original one to see if there are
  changes.
* Consider a sequence start any point which was true and has changed.
* Consider a sequence end any point which was false and has changed.
Args:
  errors (ndarray): Array of errors.
  epsilon (float): Threshold value. All errors above epsilon are considered an
  anomaly.
  anomaly_padding (int):
    Number of errors before and after a found anomaly that are added to
    the anomalous sequence.
Returns:
  ndarray, float:
    * Array containing start, end of each found anomalous sequence.
    * Maximum error value that was not considered an anomaly.
"""
above = pd.Series(errors > epsilon)
index_above = np.argmax(above.values)
for idx in index_above.flatten():
    above[max(0, idx - anomaly_padding):min(idx + anomaly_padding + 1,
        len(above))] = True
shift = above.shift(1).fillna(False)
change = above != shift
if above.all():
    max_below = 0
else:
    max_below = max(errors[~above])
index = above.index
starts = index[above & change].tolist()
ends = (index[~above & change]-1).tolist()
if len(ends) == len(starts)-1:
    ends.append(len(above)-1)
return np.array([starts, ends]).T, max_below

# anomalies.py (5-2)
def _get_max_errors(self, errors, sequences, max_below):
    """Get the maximum error for each anomalous sequence.
    Also add a row with the max error which was not considered anomalous.
    Table containing a "max_error" column with the maximum error of each
    sequence and the columns "start" and "stop" with the corresponding start
    and stop indexes, sorted descendingly by the maximum error.
    Args:
        errors (ndarray): Array of errors.
        sequences (ndarray): Array containing start, end of anomalous sequences
        max_below (float): Maximum error value that was not considered an anomaly.
    Returns:
        pandas.DataFrame: DataFrame object containing columns "start", "stop" and "max_error".
    """
    max_errors = [{
        'max_error': max_below,
        'start': -1,
        'stop': -1
    }]
    for sequence in sequences:
        start, stop = sequence
        sequence_errors = errors[start: stop + 1]

```



```

max_errors.append({
    'start': start,
    'stop': stop,
    'max_error': max(sequence_errors)
})
max_errors = pd.DataFrame(max_errors).sort_values('max_error', ascending=False)
return max_errors.reset_index(drop=True)

```

anomalies.py (6)

```
def _prune_anomalies(self, max_errors, min_percent):
```

"""Prune anomalies to mitigate false positives.

This is done by following these steps:

- * Shift the errors 1 negative step to compare each value with the next one.
- * Drop the last row, which we do not want to compare.
- * Calculate the percentage increase for each row.
- * Find rows which are below "min_percent".
- * Find the index of the latest of such rows.
- * Get the values of all the sequences above that index.

Args:

max_errors (pandas.DataFrame): DataFrame object containing columns "start", "stop" and "max_error".

min_percent (float):

Percentage of separation the anomalies need to meet between themselves and the highest non-anomalous error in the window sequence.

Returns:

ndarray: Array containing start, end, max_error of the pruned anomalies.

"""

```

next_error = max_errors['max_error'].shift(-1).iloc[:-1]
max_error = max_errors['max_error'].iloc[:-1]
increase = (max_error - next_error) / max_error
too_small = increase < min_percent
if too_small.all():
    last_index = -1
else:
    last_index = max_error[~too_small].index[-1]
return max_errors[['start', 'stop', 'max_error']].iloc[0: last_index+1].values

```

anomalies.py (7)

```
def _compute_scores(self, pruned_anomalies, errors, threshold, window_start):
```

"""Compute the score of the anomalies.

Calculate the score of the anomalies proportional to the maximum error in the sequence and add window_start timestamp to make the index absolute.

Args:

pruned_anomalies (ndarray): Array of anomalies containing the start, end and max_error for all anomalies in the window.

errors (ndarray): Array of errors.

threshold (float): Threshold value.

window_start (int): Index of the first error value in the window.

Returns:

list: List of anomalies containing start-index, end-index, score for each anomaly.

"""

```

anomalies = list()
denominator = errors.mean() + errors.std()
for row in pruned_anomalies:
    max_error = row[2]
    score = (max_error - threshold) / denominator
    anomalies.append([row[0] + window_start, row[1] + window_start, score])
return anomalies

```

anomalies.py (8)

```
def _merge_sequences(self, sequences):
```



```

"""Merge consecutive and overlapping sequences.
We iterate over a list of start, end, score triples and merge together
overlapping or consecutive sequences.
The score of a merged sequence is the average of the single scores,
weighted by the length of the corresponding sequences.
Args:
    sequences (list): List of anomalies, containing start-index, end-index, score for each anomaly.
Returns:
    ndarray: Array containing start-index, end-index, score for each anomaly after merging.
"""

if len(sequences) == 0:
    return np.array([])
sorted_sequences = sorted(sequences, key=lambda entry: entry[0])
new_sequences = [sorted_sequences[0]]
score = [sorted_sequences[0][2]]
weights = [sorted_sequences[0][1] - sorted_sequences[0][0]]
for sequence in sorted_sequences[1:]:
    prev_sequence = new_sequences[-1]
    if sequence[0] <= prev_sequence[1] + 1:
        score.append(sequence[2])
        weights.append(sequence[1] - sequence[0])
        weighted_average = np.average(score, weights=weights)
        new_sequences[-1] = (prev_sequence[0],
                             max(prev_sequence[1],
                                 sequence[1]), weighted_average)
    else:
        score = [sequence[2]]
        weights = [sequence[1] - sequence[0]]
        new_sequences.append(sequence)
return np.array(new_sequences)

# anomalies.py (9)
def find_window_sequences(self, window, z_range, anomaly_padding, min_percent, window_start, fixed_threshold):
    """Find sequences of values that are anomalous.
    We first find the threshold for the window, then find all sequences above that
    threshold.
    After that, we get the max errors of the sequences and prune the anomalies.
    Lastly, the
    score of the anomalies is computed.
    Args:
        window (ndarray): Array of errors in the window that is analyzed.
        z_range (list):
            List of two values denoting the range out of which the start points for the
            dynamic find_threshold function are chosen.
        anomaly_padding (int):
            Number of errors before and after a found anomaly that are added to
            the anomalous sequence.
        min_percent (float):
            Percentage of separation the anomalies need to meet between
            themselves and the
            highest non-anomalous error in the window sequence.
        window_start (int): Index of the first error value in the window.
        fixed_threshold (bool): Indicates whether to use fixed threshold or dynamic
        threshold.
    Returns:
        ndarray:
            Array containing the start-index, end-index, score for each anomalous
            sequence that was found in the window.
    """

```



```

if fixed_threshold: threshold =self._fixed_threshold(window)
else:
    threshold =self._find_threshold(window, z_range)
window_sequences, max_below =self._find_sequences(window, threshold,
anomaly_padding)
max_errors =self._get_max_errors(window, window_sequences, max_below)
pruned_anomalies =self._prune_anomalies(max_errors, min_percent)
window_sequences =self._compute_scores(pruned_anomalies, window, threshold,
window_start)
return window_sequences

```

anomalies.py (10-1)

```

def find_anomalies(self, errors, index, z_range=(0, 10), window_size=None,
window_size_portion=None, window_step_size=None,
window_step_size_portion=None, min_percent=0.1,
anomaly_padding=50, lower_threshold=False,
fixed_threshold=True):

```

"""Find sequences of error values that are anomalous.

We first define the window of errors, that we want to analyze. We then find the anomalous sequences in that window and store the start/stop index pairs that correspond to each

sequence, along with its score. Optionally, we can flip the error sequence around the mean and apply the same procedure, allowing us to find unusually low error sequences.

We then move the window and repeat the procedure.

Lastly, we combine overlapping or consecutive sequences.

Args:

errors (ndarray): Array of errors.

index (ndarray): Array of indices of the errors.

z_range (list):

Optional. List of two values denoting the range out of which the start points for the scipy.fmin function are chosen. If not given, (0, 10) is used.

window_size (int):

Optional. Size of the window for which a threshold is calculated. If not given, 'None' is used, which finds one threshold for the entire sequence of errors.

window_size_portion (float):

Optional. Specify the size of the window to be a portion of the sequence of errors.

If not given, 'None' is used, and window size is used as is.

window_step_size (int): Optional. Number of steps the window

is moved before another threshold is calculated for the new window.

window_step_size_portion (float):

Optional. Specify the number of steps to be a portion of the window size. If not given, 'None' is used, and window step size is used as is.

anomalies.py (10-2)

min_percent (float):

Optional. Percentage of separation the anomalies need to meet between themselves and the highest non-anomalous error in the window sequence. If not given, 0.1 is used.

anomaly_padding (int):

Optional. Number of errors before and after a found anomaly that are added to the anomalous sequence. If not given, 50 is used.

lower_threshold (bool):

Optional. Indicates whether to apply a lower threshold to find unusually low errors. If not given, 'False' is used.

fixed_threshold (bool):

Optional. Indicates whether to use fixed threshold or dynamic threshold. If not given, 'False' is used.

Returns:

ndarray: Array containing start-index, end-index, score for each anomalous sequence that was found.

"""



```

window_size = window_size or len(errors)
if window_size_portion:
    window_size = np.ceil(len(errors) * window_size_portion).astype('int')
window_step_size = window_step_size or window_size
if window_step_size_portion:
    window_step_size = np.ceil(window_size * window_step_size_portion).astype('int')
window_start = 0
window_end = 0
sequences = list()
# anomalies.py (10-3)
while window_end < len(errors):
    window_end = window_start + window_size
    window = errors[window_start:window_end]
    window_sequences = self._find_window_sequences(window,
                                                    z_range,
                                                    anomaly_padding,
                                                    min_percent,
                                                    window_start,
                                                    fixed_threshold)

    sequences.extend(window_sequences)
    if lower_threshold:
        # Flip errors sequence around mean
        mean = window.mean()
        inverted_window = mean - (window - mean)
        inverted_window_sequences = self._find_window_sequences(inverted_window,
                                                                z_range,
                                                                anomaly_padding,
                                                                min_percent,
                                                                window_start,
                                                                fixed_threshold)

        sequences.extend(inverted_window_sequences)
    window_start = window_start + window_step_size
    sequences = self._merge_sequences(sequences)
    anomalies = list()
    for start, stop, score in sequences:
        print("start", start)
        print("stop", stop)
        print("score", score)
        anomalies.append([index[int(start)], index[int(stop)], score])
    return anomalies

# anomalies.py (11)
def _compute_critic_score(self, critics, smooth_window):
    """Compute an array of anomaly scores.
    Args:
        critics (ndarray): Critic values.
        smooth_window (int): Smooth window that will be applied to compute smooth errors.
    Returns:
        ndarray: Array of anomaly scores.
    """
    critics = np.asarray(critics)
    l_quantile = np.quantile(critics, 0.25)
    u_quantile = np.quantile(critics, 0.75)
    in_range = np.logical_and(critics >= l_quantile, critics <= u_quantile)
    critic_mean = np.mean(critics[in_range])
    critic_std = np.std(critics)
    z_scores = np.absolute((np.asarray(critics) - critic_mean) / critic_std) + 1
    z_scores = pd.Series(z_scores).rolling(smooth_window, center=True,
                                          min_periods=smooth_window // 2).mean().values

```



```

    return z_scores
# anomalies.py (12)
def _regression_errors(y, y_hat, smoothing_window=0.01, smooth=True):
    """Compute an array of absolute errors comparing predictions and expected output.
    If smooth is True, apply EWMA to the resulting array of errors.
    Args:
        y (ndarray): Ground truth.
        y_hat (ndarray): Predicted values.
        smoothing_window (float):
            Optional. Size of the smoothing window, expressed as a proportion of the total
            length of y. If not given, 0.01 is used.
        smooth (bool):
            Optional. Indicates whether the returned errors should be smoothed with EWMA.
            If not given, 'True' is used.
    Returns:
        ndarray: Array of errors.
    """
    errors = np.abs(y-y_hat)[:, 0]
    if not smooth:
        return errors
    smoothing_window = int(smoothing_window*len(y))
    return pd.Series(errors).ewm(span=smoothing_window).mean().values
# anomalies.py (13)
def regression_errors(y, y_hat, smoothing_window=0.01, smooth=True):
    """Compute an array of absolute errors comparing predictions and expected output.
    Args:
        y (ndarray): Ground truth.
        y_hat (ndarray): Predicted values.
        smoothing_window (float):
            Optional. Size of the smoothing window, expressed as a proportion of the total
            length of y. If not given, 0.01 is used.
        smooth (bool):
            Optional. Indicates whether the returned errors should be smoothed with EWMA.
            If not given, 'True' is used.
    Returns:
        ndarray: Array of errors.
    """
    errors = np.abs(y - y_hat)[:, 0]
    if not smooth:
        return errors
    smoothing_window = int(smoothing_window * len(y))
    return pd.Series(errors).ewm(span=smoothing_window).mean().values
# anomalies.py (14)
def _point_wise_error(self, y, y_hat):
    """Compute point-wise error between predicted and expected values.
    Args:
        y (ndarray): Ground truth.
        y_hat (ndarray): Predicted values.
    Returns:
        ndarray: An array of smoothed point-wise error.
    """
    y_abs = abs(y-y_hat)
    y_abs_sum = np.sum(y_abs, axis=-1)
    return y_abs_sum
def _area_error(self, y, y_hat, score_window=10):
    """Compute area error between predicted and expected values.
    Args:
        y (ndarray): Ground truth.
        y_hat (ndarray): Predicted values.

```



```

score_window (int):
    Optional. Size of the window over which the scores are calculated.
    If not given, 10 is used.
Returns:
    ndarray: An array of area error.
"""

smooth_y = pd.Series(y).rolling(score_window,
                                center=True,
                                min_periods=score_window // 2).apply(integrate.trapz)
smooth_y_hat = pd.Series(y_hat).rolling(score_window,
                                         center=True,
                                         min_periods=score_window // 2).apply(integrate.trapz)
errors = abs(smooth_y - smooth_y_hat)
return errors

# anomalies.py (15)
def _dtw_error(self, y, y_hat, score_window=10):
    """Compute dtw error between predicted and expected values.
    Args:
        y (ndarray): Ground truth.
        y_hat (ndarray): Predicted values.
        score_window (int):
            Optional. Size of the window over which the scores are calculated.
            If not given, 10 is used.
    Returns:
        ndarray: An array of dtw error.
    """
    length_dtw = (score_window // 2) * 2 + 1
    half_length_dtw = length_dtw // 2
    # add padding
    y_pad = np.pad(y,
                   (half_length_dtw, half_length_dtw),
                   'constant',
                   constant_values=(0, 0))
    y_hat_pad = np.pad(y_hat,
                       (half_length_dtw, half_length_dtw),
                       'constant',
                       constant_values=(0, 0))

    i = 0
    similarity_dtw = list()
    while i < len(y) - length_dtw:
        true_data = y_pad[i:i + length_dtw]
        true_data = true_data.flatten()
        pred_data = y_hat_pad[i:i + length_dtw]
        pred_data = pred_data.flatten()
        dist = dtw(true_data, pred_data)
        similarity_dtw.append(dist)
        i += 1
    errors = ([0] * half_length_dtw + similarity_dtw + [0] * (len(y) - len(similarity_dtw) -
                                                             half_length_dtw))
    return errors

# anomalies.py (16-1)
def _reconstruction_errors(self, y, y_hat, step_size=1,
                           score_window=10, smoothing_window=0.01,
                           smooth=True, rec_error_type='point'):
    """Compute an array of reconstruction errors.
    Compute the discrepancies between the expected and the
    predicted values according to the reconstruction error type.
    Args:
        y (ndarray): Ground truth.

```



`y_hat` (ndarray): Predicted values. Each timestamp has multiple predictions.
`step_size` (int):
 Optional. Indicating the number of steps between windows in the predicted values.
 If not given, 1 is used.
`score_window` (int):
 Optional. Size of the window over which the scores are calculated.
 If not given, 10 is used.
`smoothing_window` (float or int):
 Optional. Size of the smoothing window, when float it is expressed as a proportion
 of the total length of `y`. If not given, 0.01 is used.
`smooth` (bool):
 Optional. Indicates whether the returned errors should be smoothed.
 If not given, 'True' is used.
`rec_error_type` (str):
 Optional. Reconstruction error types `["point", "area", "dtw"]`.
 If not given, "point" is used.

Returns:

ndarray: Array of reconstruction errors.

"""

```
if isinstance(smoothing_window, float):
    smoothing_window = min(math.trunc(len(y) * smoothing_window), 200)
true = []
for i in range(len(y)):
    true.append(y[i][0])
for it in range(len(y[-1]) - 1): # contents of the last window included
    true.append(y[-1][it+1])
# anomalies.py (16-2)
#####
predictions = []
predictions_vs = []
pred_length = y_hat.shape[1]
num_errors = y_hat.shape[1] + step_size * (y_hat.shape[0] - 1)
for i in range(num_errors):
    intermediate = []
    for j in range(max(0, i - num_errors + pred_length), min(i+1, pred_length)):
        intermediate.append(y_hat[i-j, j])
    ave_p = []
    if intermediate:
        predictions.append(np.average(intermediate, axis=0))
        predictions_vs.append([
            np.min(np.asarray(intermediate)),
            np.percentile(np.asarray(intermediate), 25),
            np.percentile(np.asarray(intermediate), 50),
            np.percentile(np.asarray(intermediate), 75),
            np.max(np.asarray(intermediate))
        ])
true = np.asarray(true)
predictions = np.asarray(predictions)
predictions_vs = np.asarray(predictions_vs)
# Compute reconstruction errors
if rec_error_type.lower() == "point":
    errors = self._point_wise_error(true, predictions)
elif rec_error_type.lower() == "area":
    errors = self._area_error(true, predictions, score_window)
elif rec_error_type.lower() == "dtw":
    errors = self._dtw_error(true, predictions, score_window)
# Apply smoothing
if smooth:
    errors = pd.Series(errors).rolling(smoothing_window,
```



```

        center=True,
        min_periods=smoothing_window //2).mean().values
    return errors, predictions_vs

# anomalies.py (17-1)
def score_anomalies(self, y, y_hat, critic, index,
                    score_window=10, critic_smooth_window=None,
                    error_smooth_window=None, smooth=True,
                    rec_error_type="point", comb="mult", lambda_rec=0.5):
    """Compute an array of anomaly scores.
    Anomaly scores are calculated using a combination of reconstruction error and critic score.
    Args:
        y (ndarray): Ground truth.
        y_hat (ndarray): Predicted values. Each timestamp has multiple predictions.
        index (ndarray): time index for each y (start position of the window)
        critic (ndarray): Critic score. Each timestamp has multiple critic scores.
        score_window (int):
            Optional. Size of the window over which the scores are calculated.
            If not given, 10 is used.
        critic_smooth_window (int):
            Optional. Size of window over which smoothing is applied to critic.
            If not given, 200 is used.
        error_smooth_window (int):
            Optional. Size of window over which smoothing is applied to error.
            If not given, 200 is used.
        smooth (bool):
            Optional. Indicates whether errors should be smoothed.
            If not given, 'True' is used.
        rec_error_type (str):
            Optional. The method to compute reconstruction error. Can be one of
            '["point", "area", "dtw"]'. If not given, 'point' is used.
        comb (str):
            Optional. How to combine critic and reconstruction error. Can be one
            of '["mult", "sum", "rec"]'. If not given, 'mult' is used.
        lambda_rec (float):
            Optional. Used if 'comb="sum"' as a lambda weighted sum to combine
            scores. If not given, 0.5 is used.
    Returns:
        ndarray: Array of anomaly scores.
    """

# anomalies.py (17-2)
critic_smooth_window = critic_smooth_window or math.trunc(y.shape[0]*0.01)
error_smooth_window = error_smooth_window or math.trunc(y.shape[0]*0.01)
step_size = 1 # expected to be 1
true_index = list(index) # no offset
true = []
for i in range(len(y)):
    true.append(y[i][0])
for it in range(len(y[-1]) - 1): # contents of the last window included
    true.append(y[-1][it+1])
    true_index.append((index[-1] + it + 1)) # extend the index (by including the last window part)
true_index = np.array(true_index) # in order to cover the whole sequence of data
critic_extended = list()
for c in critic:
    critic_extended.extend(np.repeat(c, y_hat.shape[1]).tolist())
critic_extended = np.asarray(critic_extended).reshape((-1, y_hat.shape[1]))
critic_kde_max = []
pred_length = y_hat.shape[1]
num_errors = y_hat.shape[1] + step_size * (y_hat.shape[0]-1)

```



```
# anomalies.py (17-3)
for i in range(num_errors):
    critic_intermediate = []
    for j in range(max(0, i - num_errors + pred_length), min(i+1, pred_length)):
        critic_intermediate.append(critic_extended[i - j, j])
    if len(critic_intermediate) > 1:
        discr_intermediate = np.asarray(critic_intermediate)
        try:
            critic_kde_max.append(discr_intermediate[np.argmax(stats.gaussian_kde(discr_intermediate)(critic_intermediate))])
        except np.linalg.LinAlgError:
            critic_kde_max.append(np.median(discr_intermediate))
    else:
        critic_kde_max.append(np.median(np.asarray(critic_intermediate)))
# Compute critic scores
critic_scores = self._compute_critic_score(critic_kde_max, critic_smooth_window)
# Compute reconstruction scores
rec_scores, predictions = self._reconstruction_errors(y, y_hat, step_size, score_window, error_smooth_window, smooth, rec_error_type)
rec_scores = stats.zscore(rec_scores)
rec_scores = np.clip(rec_scores, a_min=0, a_max=None) + 1
# Combine the two scores
if comb == "mult":
    final_scores = np.multiply(critic_scores, rec_scores)
elif comb == "sum":
    final_scores = (1 - lambda_rec) * (critic_scores - 1) + lambda_rec * (rec_scores - 1)
elif comb == "rec":
    final_scores = rec_scores
else:
    raise ValueError('Unknown combination specified {}, use "mult", "sum", or "rec" instead.'.format(comb))
true = [[t] for t in true]
return final_scores, true_index, true, predictions
```

[코드 81. Anomaly 클래스 정의]

- 학습한 MTadGAN 모형으로 테스트 데이터에 대한 예측 계산을 수행한다. score_anomalies() 함수와 find_anomalies() 함수를 이용한다.



```
# load known_anomalies
known_anomalies = pd.read_csv(args.anomaly_file)
print("anomaly_file: ", args.anomaly_file)
print(known_anomalies)
#
# predict
y_hat, critic = predict(X)
print("y_hat.shape after predict = ", y_hat.shape)
print("critic.shape after predict = ", critic.shape)
print(critic[:10])
anomaly = Anomaly()
final_scores, true_index, true, predictions = anomaly.score_anomalies(X, y_hat, critic,
                                                                    X_index, comb="mult")
final_scores = np.array(final_scores)
print("final scores shape = ", final_scores.shape)
print("true_index shape = ", true_index.shape)
print("true_index vs. final_scores = ")
for i in range(len(final_scores)):
    print("%d %f" % (true_index[i], final_scores[i]))
anomalies = anomaly.find_anomalies(final_scores, true_index)
```

```
anomaly_file: ./data/preprocessed/test/Test07_NG_dchg_Label.csv
  Unnamed: 0  label
0          0      0
1          1      0
2          2      0
3          3      0
4          4      0
...         ...   ...
4589       4589     1
4590       4590     1
4591       4591     1
4592       4592     1
4593       4593     1

[4594 rows x 2 columns]
y_hat.shape after predict = (4584, 10, 3)
critic.shape after predict = (4584, 1)
[[-5.023039 ]
 [-5.0730205]]
```

[코드 82] 학습한 MTadGAN 모형의 테스트 데이터에 대한 예측 계산

- 예측 계산(predictions) 결과와 실제 Anomaly를 출력한다.



```
print("predictions : ", predictions.shape)
print(predictions)
print("known_anomalies: ")
anom_labels=known_anomalies['label']
print(anom_labels.tail(70))
time = true_index
```

```
predictions : (4593, 1, 5)
[[[-9.40793395e-01 -5.42885959e-01 -1.44978523e-01 -5.09806592e-02
  4.30172049e-02]]

 [[-9.62449014e-01 -7.43175987e-01 -1.47876039e-01 -4.62743174e-03
  5.19203693e-02]]

 [[-9.62177038e-01 -9.40221786e-01 -1.44257084e-01 -1.22755766e-04
  5.04684895e-02]]

 ...

 [[-1.97922021e-01 -5.00729913e-03  3.82925943e-02  1.85034618e-01
  3.27315897e-01]]

 [[-1.98249191e-01 -8.21473012e-02  3.49742807e-02  1.47501892e-01
  2.62454778e-01]]

 [[-1.99002460e-01 -8.01348966e-02  3.87326665e-02  1.50219271e-01
  2.61705875e-01]]]
4524 1
4525 1
4526 1
4527 1
4528 1
```

[코드 83] 예측 결과 및 실제 Anomaly

- 예측 계산 결과와 실제 Anomaly를 비교하여 Accuracy, Precision, Recall, 그리고 F-Score를 출력한다. 이 경우 F-Score=0.7956 의 값을 보였다. 앞의 코드 73 네트워크 학습 수행 부분의 설명에서 언급한 바와 같이 정확도, 정밀도, 재현율 및 F-score 등의 결과는 학습 모형에 따라 다르게 되므로 네트워크 모형을 새롭게 학습하고 테스트 수행한다면 테스트 결과는 실행 시마다 조금씩 다르게 나온다.



```

true0 = anom_labels
print("true0 :")
print(true0.tail(40))
pred_length = len(final_scores)
avg = np.average(final_scores)
sigma = math.sqrt(sum((final_scores-avg) * (final_scores-avg)) / len(final_scores))
Z_score1 = (final_scores-avg) / sigma
pred_bin=[0]*pred_length
for i in range(len(anomalies)):
    print( anomalies[i][0], anomalies[i][1])
    for k in range(anomalies[i][0]-1, anomalies[i][1]):
        pred_bin[k]=1
true = []
true = true0[: pred_length]
pred = np.array(pred_bin)
gt = np.array(true)
n_pred = len(pred)
tp=0
tn=0
fp=0
fn=0
for i in range(n_pred):
    if pred[i]==1 and gt[i] ==1:
        tp = tp+1
    if pred[i]==0 and gt[i] ==1:
        fn = fn+1
    if pred[i]==1 and gt[i] ==0:
        fp = fp+1
    if pred[i]==0 and gt[i] ==0:
        tn = tn+1
Accuracy = float(tp+tn)/float(n_pred)
Precision = float(tp)/float(tp+fp)
Recall = float(tp)/float(tp+fn)
F1_score = 2.0/((1./Precision)+(1./Recall))
print("Accuracy : {:.4f}, Precision : {:.4f}, Recall : {:.4f}, F-score : {:.4f} ".format(
    Accuracy, Precision, Recall, F1_score))

```

```

true0 :
4554    1
4555    1
4556    1
4557    1
4558    1
4559    1
4560    1
4561    1
4562    1
4563    1
4564    1
4565    1
4566    1
4567    1
4568    1
4569    1
4570    1
4571    1
4572    1
4573    1
4574    1
4575    1
4576    1
4577    1
4578    1
4579    1
4580    1
4581    1
4582    1
4583    1
4584    1
4585    1
4586    1
4587    1
4588    1
4589    1
4590    1
4591    1
4592    1
4593    1
Name: label, dtype: int64
4320 4548
Accuracy : 0.9798, Precision : 0.7904, Recall : 0.8009, F-score : 0.7956

```

[코드 84] 정확도, 정밀도, 재현율 및 F-score 계산

- Anomaly의 연속적인 sequence들을 찾아내어 리스트로 반환한다. 여기서 gt 는 실제 anomaly이고 pred는 예측된 anomaly를 가리킨다. 여기서도 앞의 코드 84 에서 설명하였듯이 결과가 학습 모형에 따라 실행 시마다 조금씩 다르게 나오는 점에 유의하여야 한다.



```
# List of Anomaly sequences
anomalies = []
anomaly_gt = []
anomaly_pred = []
length_anom = len(pred)
print("length_anom: ", length_anom)
anom_pred_init = 0 # inside a sequence or not
anom_gt_init = 0 #
for k in range(length_anom):
    if gt[k] == 1 :
        if anom_gt_init == 0 : # Now beginning of an anomalous sequence.
            anom_gt_begin = k
            anom_gt_init = 1
        else :
            anom_gt_end = k
            if k == length_anom - 1:
                anomaly_gt.append((anom_gt_begin, anom_gt_end))
    if gt[k] == 0 and anom_gt_init == 1: # End of anom. sequence
        anom_gt_end = k - 1
        anomaly_gt.append((anom_gt_begin, anom_gt_end))
        anom_gt_init = 0
    if pred[k] == 1 :
        if anom_pred_init == 0 : # Now beginning of an anomalous sequence.
            anom_pred_begin = k
            anom_pred_init = 1
        else :
            anom_pred_end = k
            if k == length_anom - 1:
                anomaly_pred.append((anom_pred_begin, anom_pred_end))
    if pred[k] == 0 and anom_pred_init == 1: # End of anom. sequence
        anom_pred_end = k - 1
        anomaly_pred.append((anom_pred_begin, anom_pred_end))
        anom_pred_init = 0
anomalies = [anomaly_gt, anomaly_pred]
print("gt:")
print(anomaly_gt)
print("pred:")
print(anomaly_pred)
```

```
length_anom: 4593
gt:
[(4367, 4592)]
pred:
[(4319, 4547)]
```

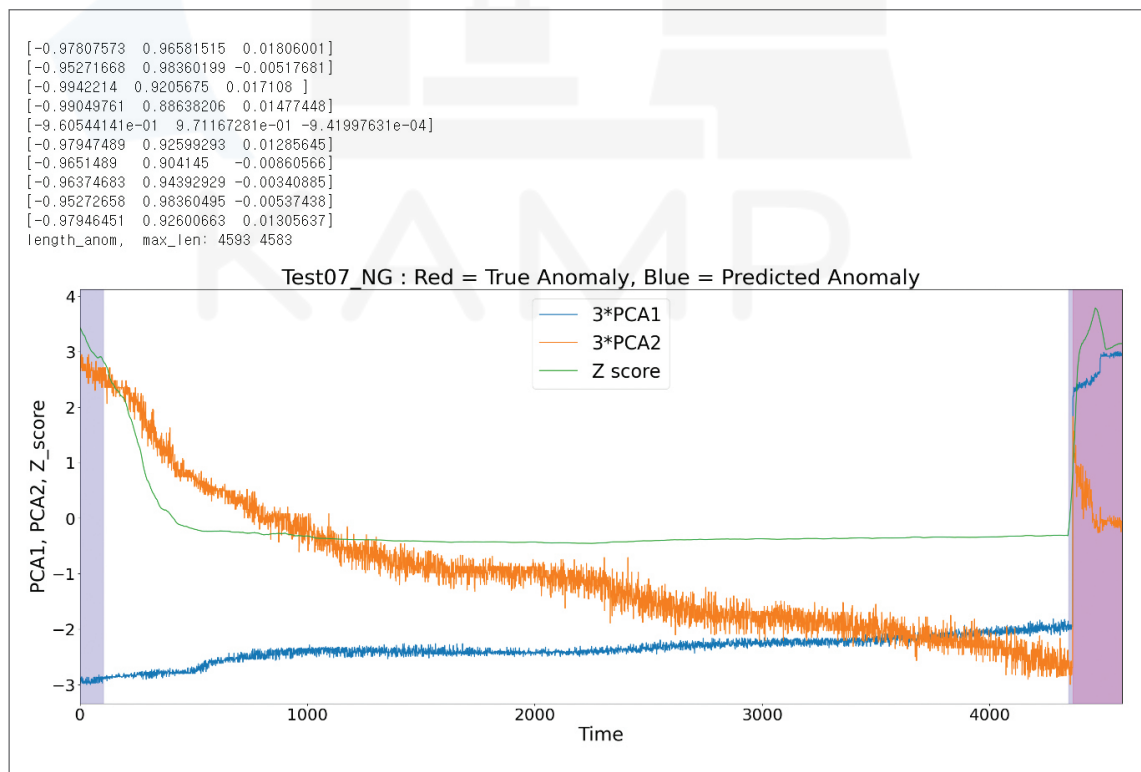
[코드 85] 테스트 데이터에 대한 Anomaly Sequence 리스트 계산

- Anomaly score의 분포에서 구한 Z score와 PCA 성분 그리고 true anomaly등을 비교할 수 있도록 그래프로 출력한다. True Anomaly 위치가 예측과 상당한 정도로 일치한다. PCA 성분값이 비정상적인 점프를 보일 때 Anomaly가 발생하는 것이 확인된다. 여기서도 앞의 코드 84 에서 설명하였듯이 결과가 학습 모형에 따라 실행 시마다 조금씩 다르게 나오는 점에 유의하여야 한다.

```

register_matplotlib_converters()
np.random.seed(0)
if not isinstance(anomalies, list):
    anomalies = [anomalies]
fig = plt.figure(figsize=(30, 12))
ax = fig.add_subplot(111)
max_len = length_anom-10
time = range(max_len)
Z_score2 = Z_score1[:max_len]
X_signal = []
for kk in range(max_len):
    X_signal.append(X[kk, 1])
for kk in range(10):
    print( X[kk, 1] )
X_signal_2 = np.array(X_signal)
plt.plot(time, 3*X_signal_2[:, 0], label='3*PCA1')
plt.plot(time, 3*X_signal_2[:, 1], label='3*PCA2')
plt.plot(time, Z_score2, label='Z score')
plt.legend(loc=0, fontsize=30)
print("length_anom, max_len:", length_anom, max_len)
colors = ['red'] + ['blue'] * (len(anomalies)-1)
for i, anomaly in enumerate(anomalies):
    if not isinstance(anomaly, list):
        anomaly = list(anomaly[['start', 'end']].itertuples(index=False))
    for _, anom in enumerate(anomaly):
        t1 = anom[0]
        t2 = anom[1]
        plt.axvspan(t1, t2, color=colors[i], alpha=0.2)
plt.title('Test07_NG : Red = True Anomaly, Blue = Predicted Anomaly', size=34)
plt.ylabel('PCA1, PCA2, Z_score', size=30)
plt.xlabel('Time', size=30)
plt.xticks(size=26)
plt.yticks(size=26)
plt.xlim([time[0], time[-1]])
plt.show()

```



[코드 86] 테스트 데이터 1에 대한 Anomaly 계산 결과



[단계 ⑦] 테스트 셋 결과 분석

⑦-1. 테스트 셋 결과 분석

- 단계 ⑥에서 구한 이상점수의 분포에서 표준편차를 구하고 이로부터 이른바 Z score를 구할 수 있다. 즉, $Z \equiv (S - m) / \sigma$ 이다. 여기서 m은 평균값 그리고 σ 는 표준 편차이다. 각 시점의 Z score가 일정한 threshold값을 넘어설 때 이를 anomaly라고 한다.
- 위의 프로그램을 실행하면 ground truth anomaly인 gt와 예측값인 pred로부터 이상구간(anomaly interval)을 찾을 수 있다.

- 이로부터 주어진 테스트 셋에 대한 MTadGAN 알고리즘의 정확도(Accuracy), 정밀도(Precision), 재현율(Recall)을 계산할 수 있고 또한 F-Score를 구한다. 계산 공식은 다음과 같다.

* 정확도(A) = $(TP + TN) / (TN + FP + FN + TP)$

* 정밀도(P) = $TP / (TP + FP)$

* 재현율(R) = $TP / (TP + FN)$ 인데 여기서

TP= True Positive, TN=True Negative, FP=False Positive, FN=False Negative를 각각 나타낸다.

- 정밀도와 재현율을 결합하여 다음과 같이 F-Score를 얻는다.

$$F \equiv \frac{(P + R)}{2PR} = \frac{1}{2} \left(\frac{1}{P} + \frac{1}{R} \right)^{-1}$$

- 위의 테스트 데이터에 적용한 계산의 경우에

* 정확도(Accuracy)=0.9798

* 정밀도(Precision)=0.7904

* 재현율(Recall) = 0.8009

* F-score = 0.7956 이다.

- 코드 84 에서 설명하였듯이 결과가 학습 모형에 따라 실행 시마다 조금씩 다르게 나오기 때문에 코드 84의 결과를 참고하여 분석할 수 있도록 한다.



[단계 ⑧] 알고리즘 평가

⑧-1. 알고리즘 평가

- MTadGAN 알고리즘을 배터리 충전, 방전 데이터에 적용하였다. 배터리 셀에서의 갑작스런 작은 전압 증가 또는 감소가 Anomaly와 밀접하게 관련된다는 것을 MTadGAN은 보여주었다. 보다 많은 학습데이터를 이용하여 학습하고 다양한 테스트 데이터에 적용함으로써 MTadGAN의 성능을 더 정밀하게 측정할 수 있으며 그 성능을 개선할 수 있다고 본다.

3. 유사 타 현장의 「전자부품(배터리팩) 품질보증 AI 데이터셋」 분석 적용

3.1 본 분석이 적용 가능한 제조현장 소개

- 배터리팩은 전기자동차의 핵심 부품이며, 제한된 공간에서 사용되기 때문에 주행 중 전달 되는 충격을 견디면서도 악조건의 환경에서도 버틸 수 있도록 안정성과 내구성이 갖춰져야 한다.
- 또한, 배터리팩은 지속적으로 관리되지 않으면 수명에 영향을 미칠 수 있으므로 제품 생산부터 시험까지 품질관리에 특별히 주의가 필요하다.
- 실제 E공장에서는 배터리팩을 구성하는 최소 단위인 배터리셀 하나의 불량으로 인해 주변 배터리셀을 구성하는 모듈 전체를 교체해야 하는 경우가 발생하고 있어 불량 배터리를 사전에 발견하여 처리할 수 있는 방안들을 도입하고 있으며, 최적의 상태를 유지하기 위해 지속적으로 모니터링하기 위한 기반을 구축하고 있는 상황이다.
- 본 분석의 결과는 배터리 제품의 품질관리가 필요한 제조현장에서 적용될 수 있다고 판단된다. 즉, 제조현장에서 운용중인 설비를 통해 공정 데이터를 수집하고, 이를 기반으로 품질검사 데이터를 확보하여 데이터셋을 구성할 수 있는 경우 해당 모델을 적용하여 유의미한 결과를 도출할 수 있다.
- 실제 배터리팩은 적게는 수십개에서 많게는 수백개의 배터리셀들로 구성되어 있으며, 일반적으로 전압, 온도, 저항 등의 데이터를 수집할 수 있다. 실제 배터리 전압과 온도는 시간에 대해 높은 상관관계를 가지고 있으므로, 이러한 다변량 시계열 데이터를 효율적으로 다루기 위해 본 분석에서와 같이 PCA(Principal Component Analysis, 주성분분석) 알고리즘을 채택하고 이러한 이상 탐지에 적합한 MTadGAN(Multivariate Time Series Anomaly Detection by Generative Adversarial Network) 같은 적대적 생성 모형을 적용할 수 있다.
- 또한, 배터리팩 품질관리에 제한되지 않고 최소 단위인 배터리셀들의 품질까지 관리할 수 있으므로 배터리셀 또는 배터리모듈 생산에 특화되어 있는 제조현장에 본 분석이 적용될 수 있다.



3.2 본 「전자부품(배터리팩) 품질보증 AI 데이터셋」 분석을 원용하여 타 제조현장 적용시, 주요 고려사항

- 대상 설비에 대한 공정 데이터를 실시간 수집하여 활용할 수 있는 인프라가 구축되어 있어야 한다.
- 특히, 본 분석에서는 AAS 표준기반 제조데이터 수집/저장 체계를 통해 수집한 실시간 공정 데이터를 기반으로 품질 보증 모델을 만드는 과정을 학습할 수 있다는 점에서 의미하는 바가 크다. 단, AAS 표준기반 제조데이터 수집/저장 체계가 준비되어 있어야 하므로 2장 분석실습에서 설명한 바와 같이 깃허브 사이트(<https://github.com/kosmo-nestfield>)에서 제공하는 가이드에 따라 현장에 적용되어 있어야 한다.
- 대상 설비로부터 수집한 공정 데이터를 기반으로 결론을 도출할 수 있으나 추가적인 센서를 활용(예: 진동, 습도 등)하면 보다 정확한 분석 결과를 얻을 수 있다.
- 배터리팩을 구성하는 배터리셀 또는 모듈의 전압, 온도 등의 데이터를 추적하여 양/불 여부를 판단할 수 있으나 최소 단위인 배터리셀 사양이 모두 동일해야 한다는 조건이 따른다. 즉, 제품의 사양이 빈번하게 변경되는 환경에서는 본 분석이 제한적으로 적용될 수 있으므로 다른 제품과 명확하게 구분될 수 있도록 데이터를 추가하거나 분류하는 과정이 필요하다.
- 배터리팩 충·방전 시험 중 수집되는 데이터와 실제 운용 중 수집되는 데이터를 AI 분석하기 위해 많은 전처리 과정이 필요하다. AI 분석자는 현장 담당자와 커뮤니케이션 하면서 최적의 수집 환경을 구축할 필요가 있다.

3 부록 _ 분석환경 구축을 위한 설치 가이드

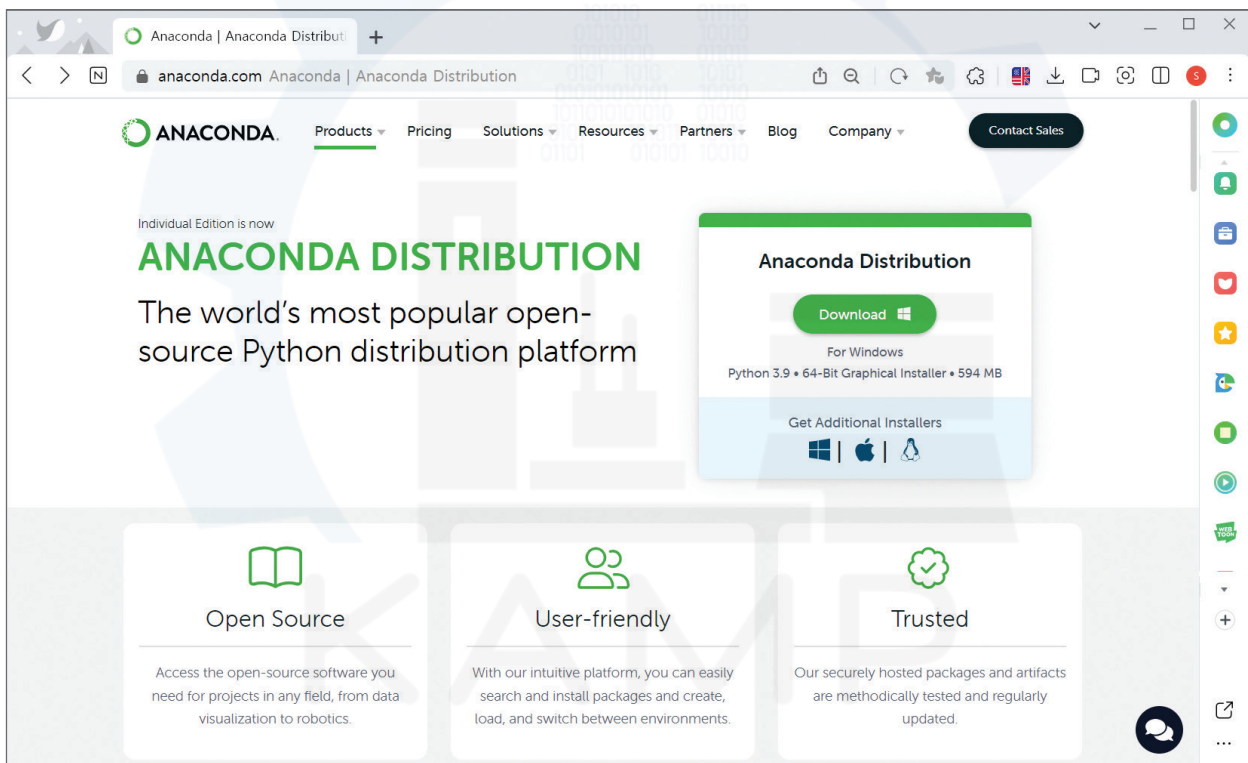
KAMP
인공지능 제조 플랫폼

데이터 분석을 위한 Python 환경은 Anaconda를 설치함으로써 간단하게 구축할 수 있다. Anaconda는 데이터 분석 및 예측을 위한 모델 생성 등을 위한 package 들을 모아놓은 Python 배포판으로서, Numpy, SciPy, matplotlib, pandas, Ipython, Jupyter Notebook, scikit-learn 을 모두 포함하고 있다.

① Anaconda 설치 파일 다운로드

Anaconda 다운로드 사이트인

“<https://www.anaconda.com/products/distribution>”에서 설치 파일을 다운로드 받을 수 있다. 이 URL을 주소창에 입력하면 다음 화면이 표시되고 여기에서 Download를 클릭한다.



Anaconda의 설치 파일은 “Anaconda3-2022.05-Windows-x86_64.exe”이며, 현재 사용 중인 컴퓨터에 64비트 윈도우가 설치돼 있으면 64비트용 설치 파일을 다운로드 받고, 32비트 윈도우가 설치돼 있다면 32비트용 설치 파일을 다운로드 받으면 된다. 다운로드 받은 Anaconda는 2022.05 버전이며, Python 3.9를 포함하고 있다.

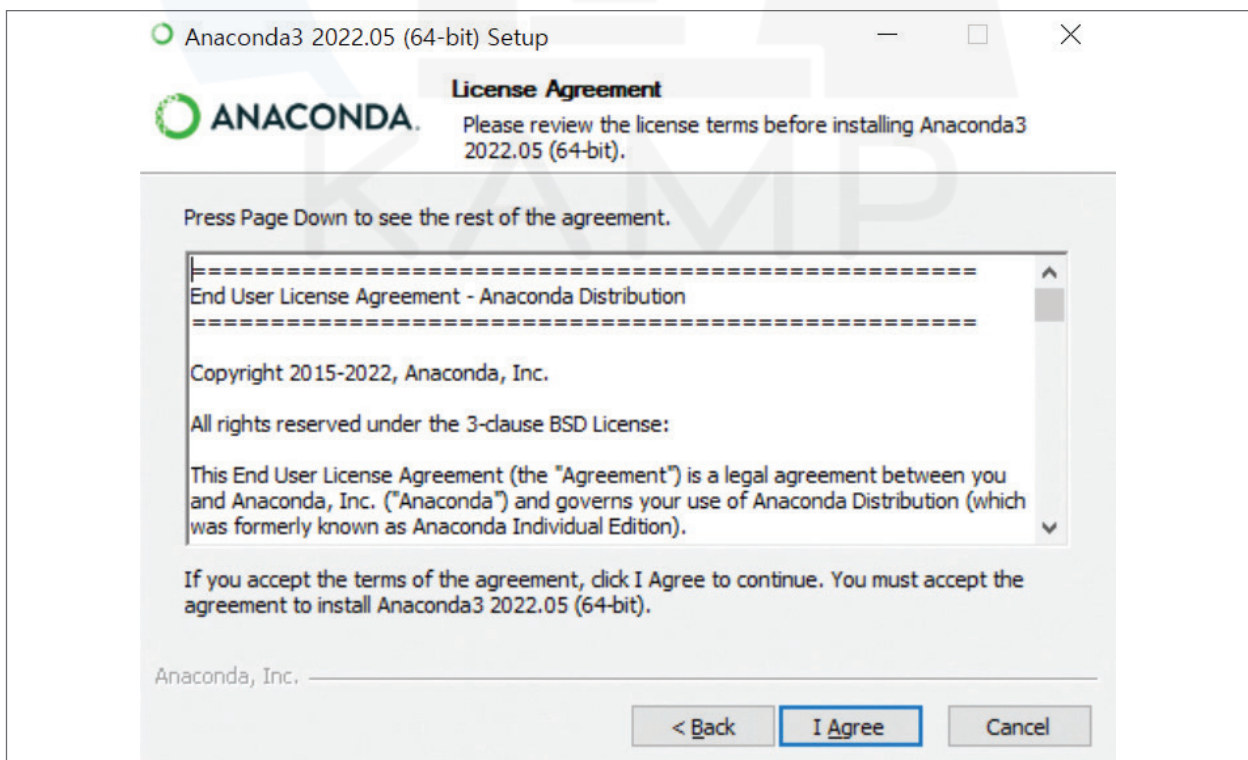


② 설치 파일을 내려받은 후, 해당 파일

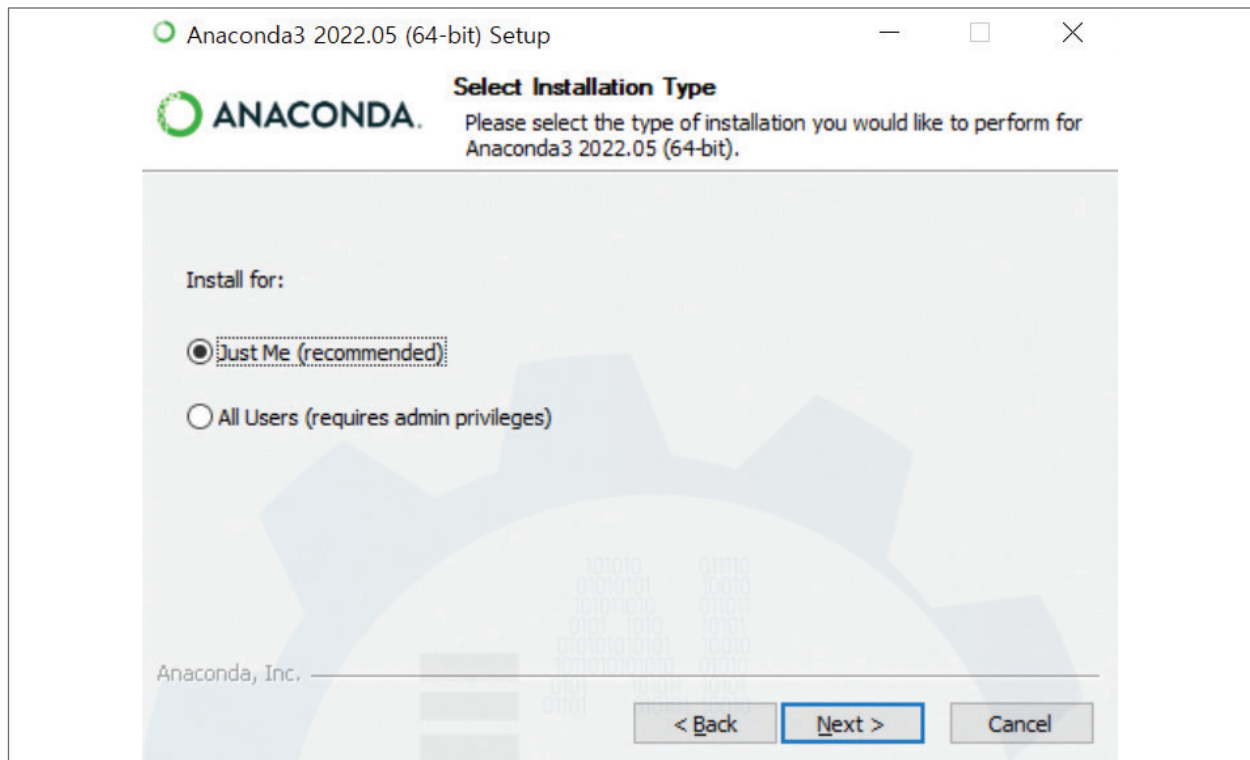
“Anaconda3-2022.05-Windows-x86_64.exe”을 클릭하여 설치를 시작한다. 다음 그림과 같이 설치 화면이 표시되면 [Next] 버튼을 클릭하여 다음 단계로 진행한다.



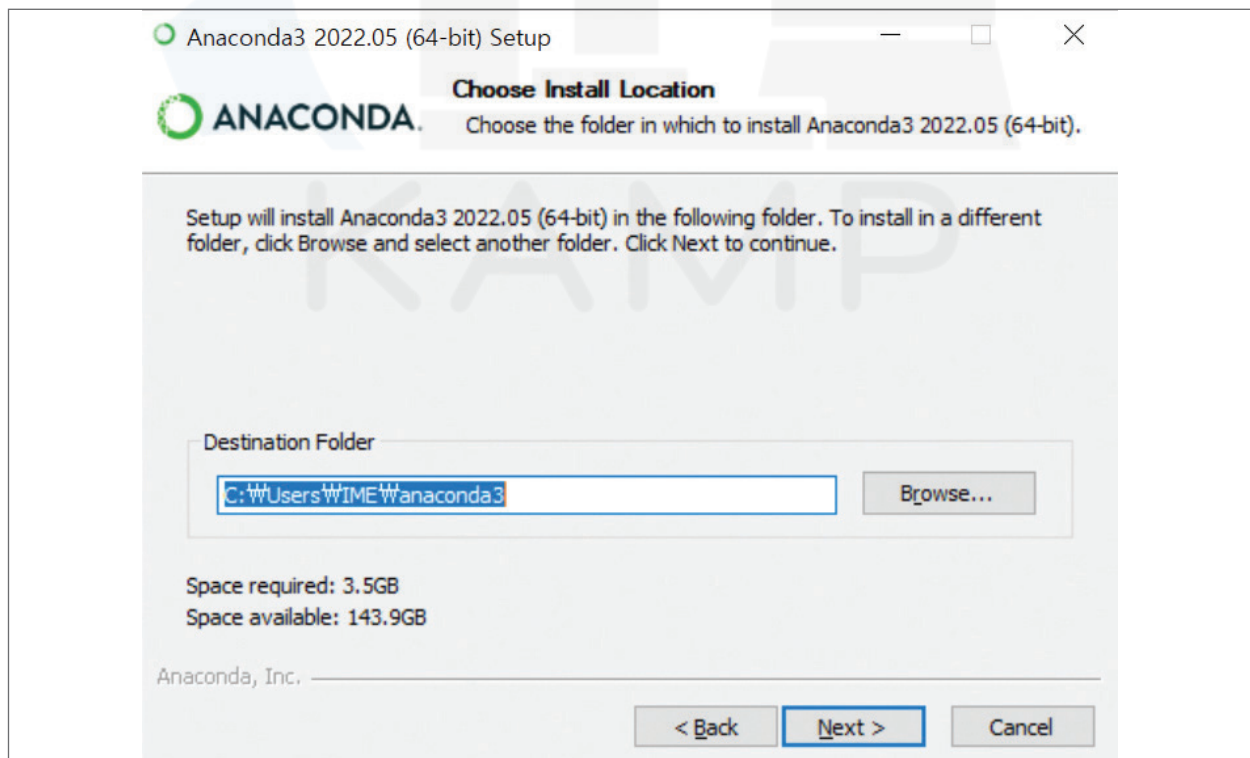
③ 다음에 표시되는 화면은 라이선스 동의에 관한 화면이다. 여기에서 [I Agree] 버튼을 클릭하여 다음 단계로 진행한다.



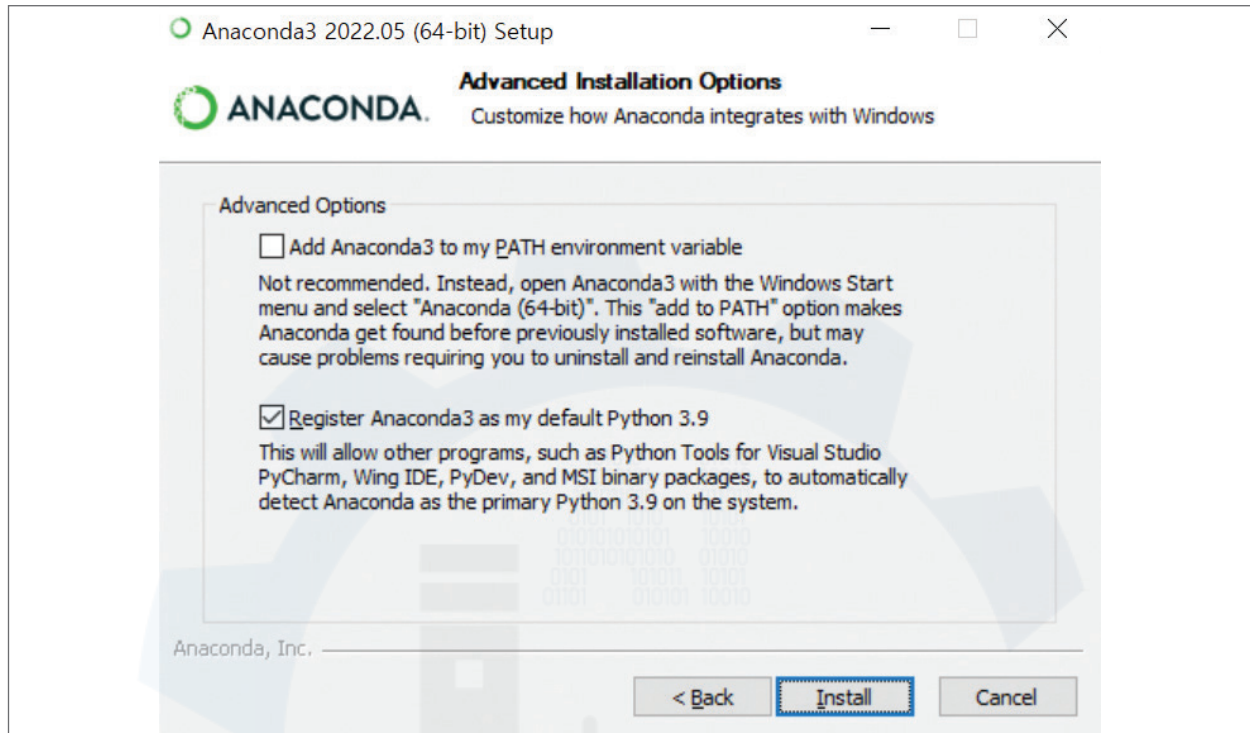
- ④ 다음에 표시되는 화면은 설치 유형을 선택하는 화면이다. [Just Me]를 선택한 후 [Next] 버튼을 클릭한다.



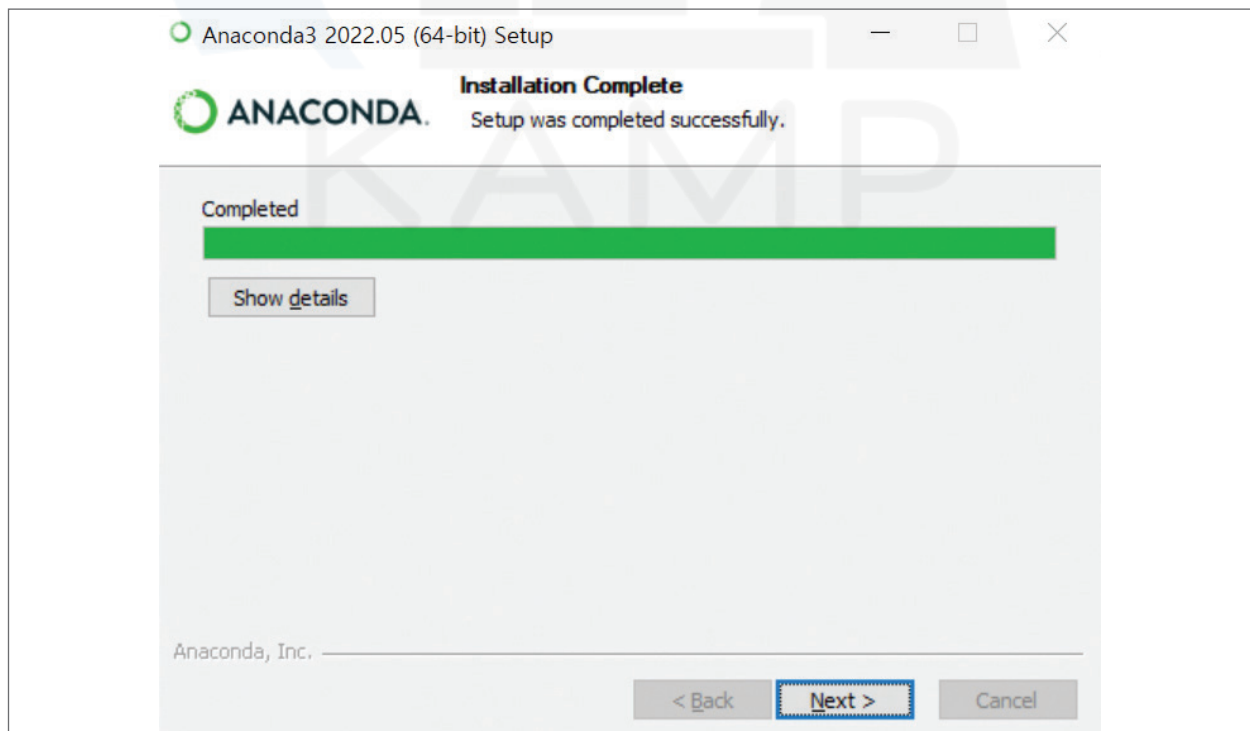
- ⑤ 다음으로 Anaconda를 설치할 폴더를 지정하는 화면이 표시된다. 여기에서 적절한 폴더를 지정하고 [Next] 버튼을 클릭한다.



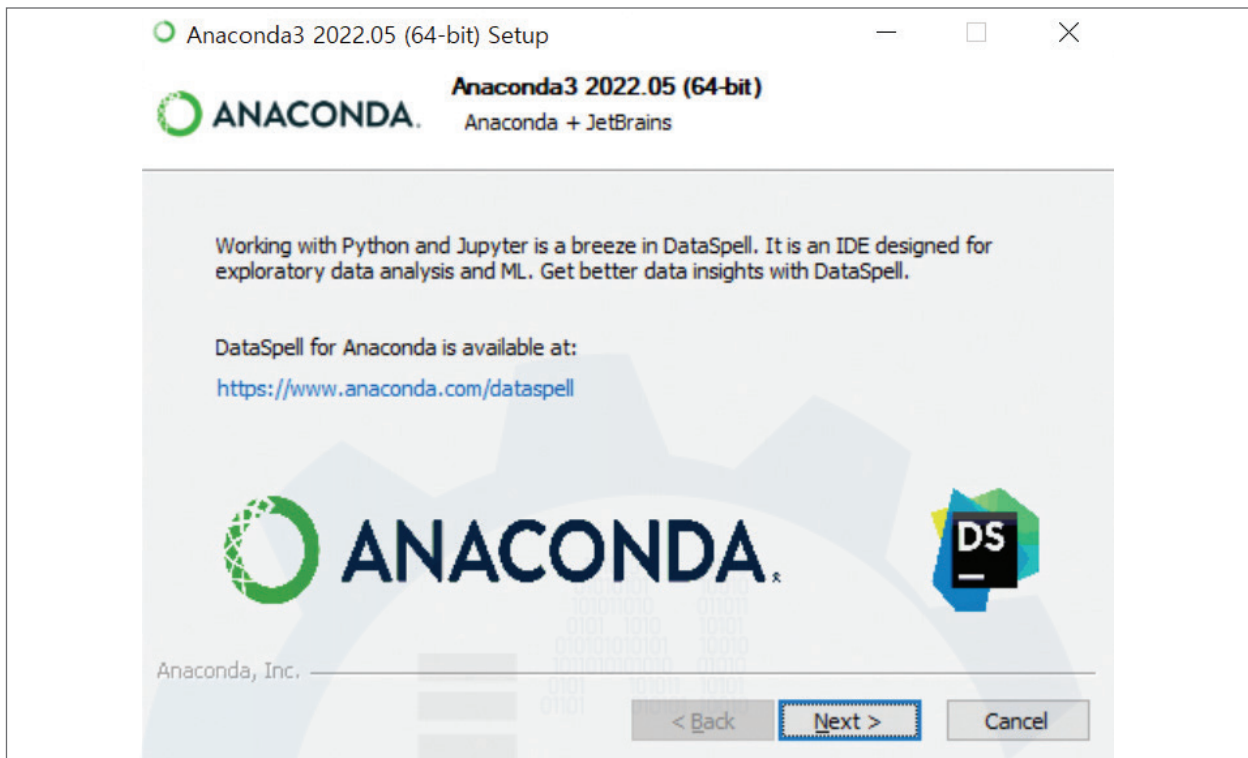
- ⑥ 다음에 표시되는 화면은 Advanced Options를 선택하는 화면이다. 첫째 Option은 Anaconda를 PATH 환경변수에 추가하여 사용하는 것이며, 둘째 Option은 다른 프로그램이 자동으로 Anaconda를 인식하게 하는 것이다. 여기에서는 두 번째 옵션을 선택한 후 [Install] 버튼을 클릭하여 다음 단계로 진행한다.



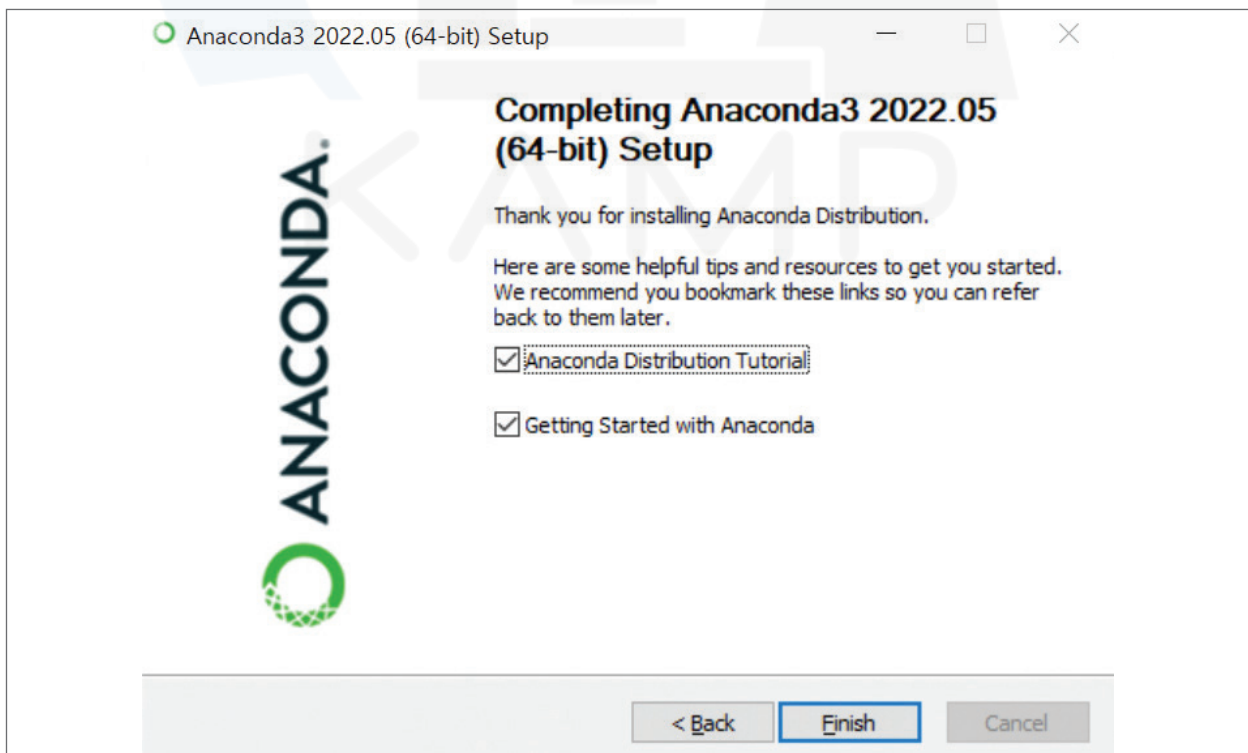
- ⑦ Anaconda 설치가 완료되면 다음 화면에 “Installation Completed”라는 메시지가 표시된다. 이 화면에서 [Next] 버튼을 눌러 다음 단계로 진행한다.



- ⑧ 다음과 같이 데이터 분석가를 위한 도구인 DataSpell에 관한 안내 화면이 표시되며, 이 화면에서 [Next] 버튼을 클릭하여 다음 단계로 진행한다.



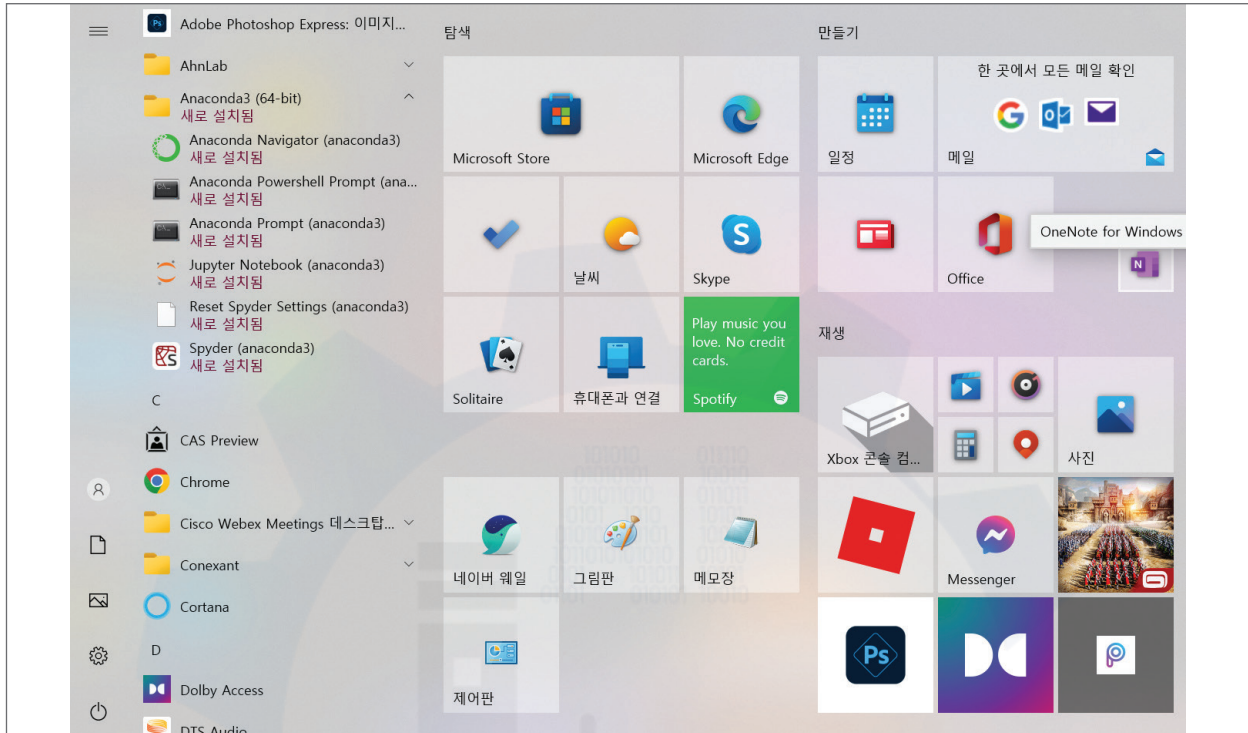
- ⑨ 다음 화면이 표시되면 Anaconda 설치 과정이 모두 완료되었음을 나타내며, Anaconda에 대한 정보를 추가적으로 얻으려면 이 화면에서 두 가지 Option을 선택하여 내용을 살펴볼 수 있다. 모든 설치 과정을 종료하기 위하여 [Finish] 버튼을 클릭한다.



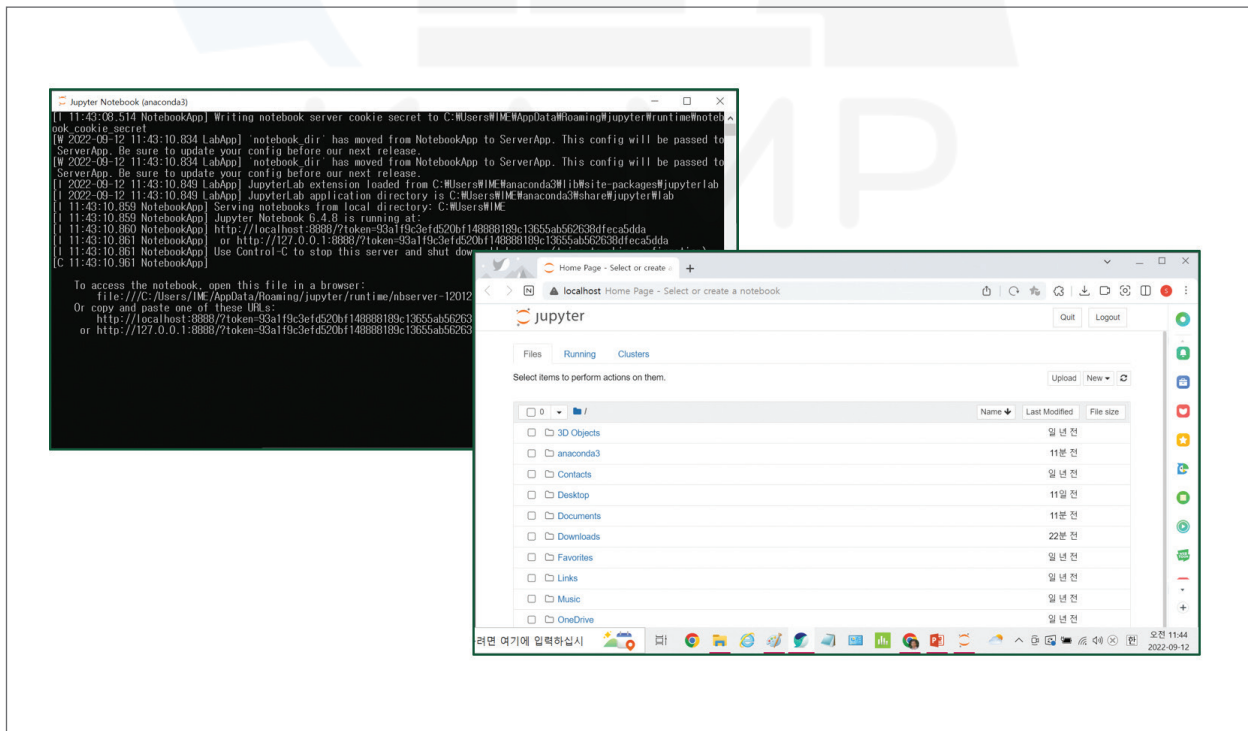


⑩ Jupyter Notebook 시작하기

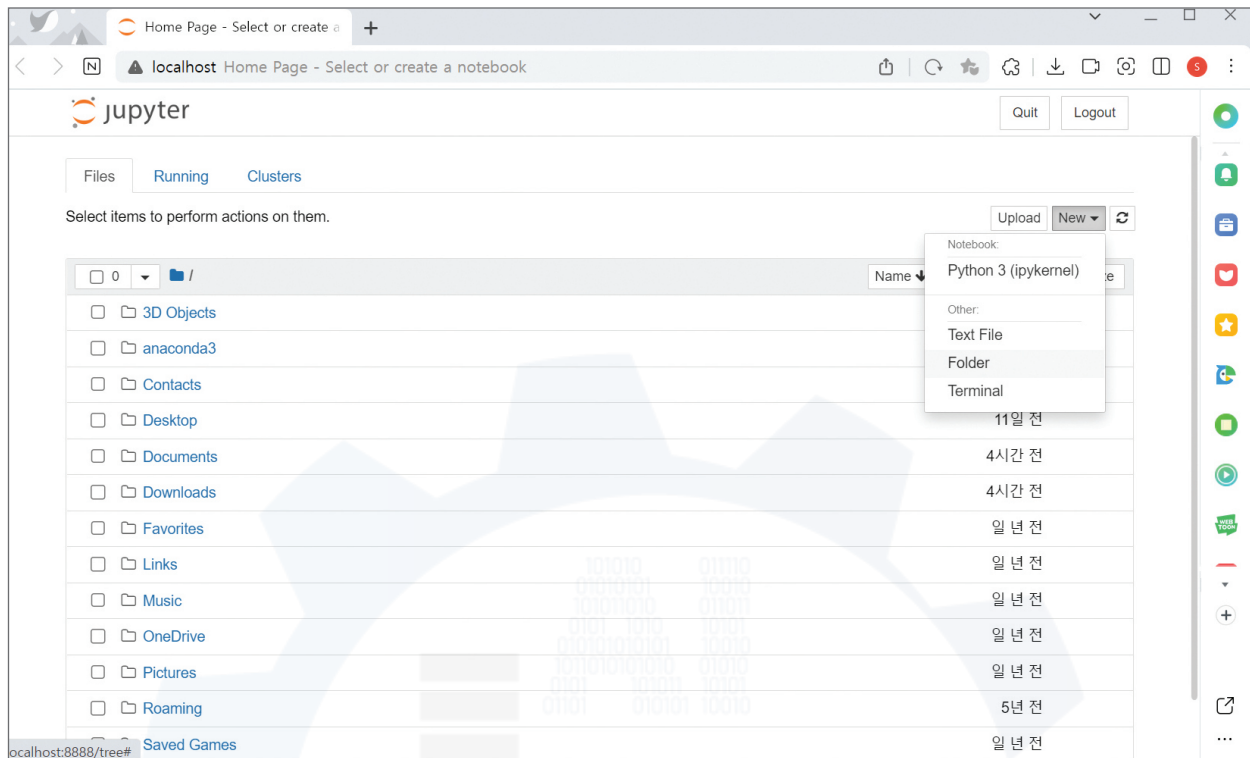
Anaconda를 설치하면 Windows 처음 화면에 다음과 같이 설치된 프로그램의 리스트가 표시된다. 여기에서 “Jupyter Notebook (anaconda3)”를 클릭하여 Jupyter Notebook을 시작한다.



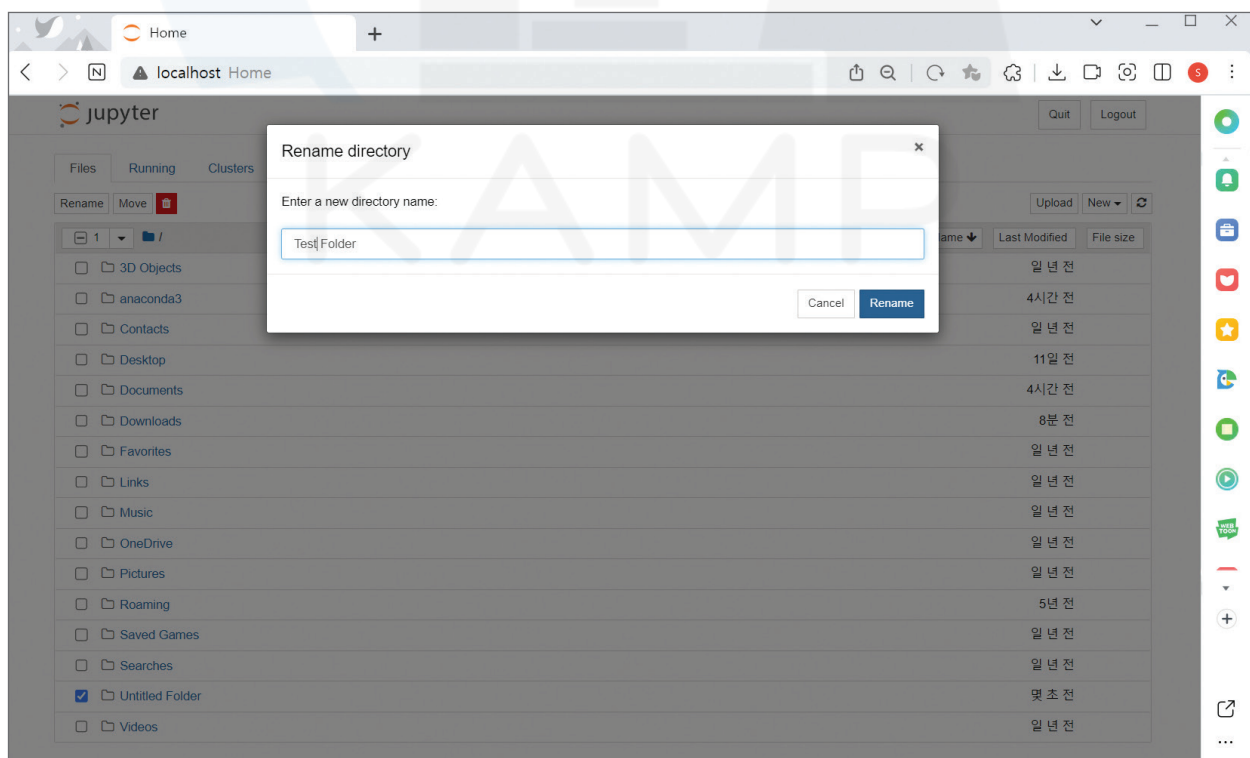
⑪ Jupyter Notebook을 시작하면 다음과 같이 Jupyter Notebook 콘솔 및 프로그래밍 작업을 할 수 있는 메인 화면이 표시된다.



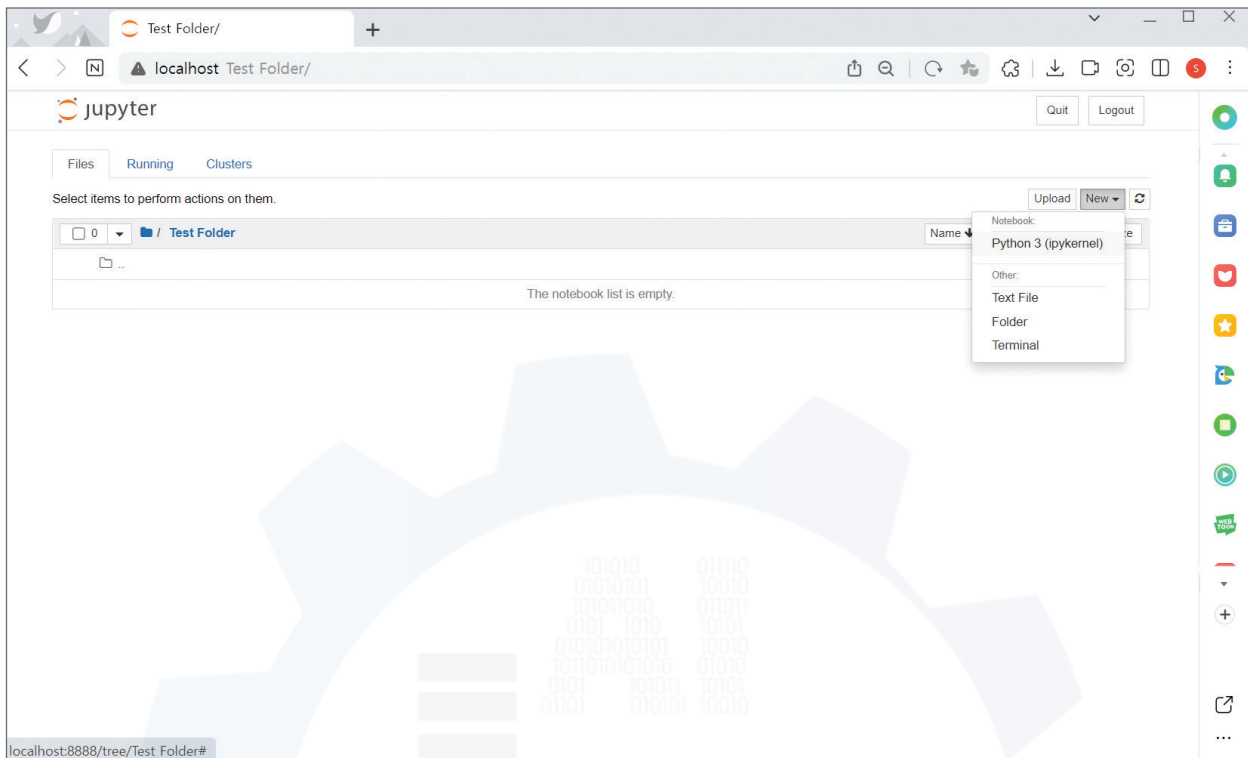
- ⑫ 새로운 폴더 생성: 다음 화면의 오른쪽에서 New를 클릭하여 Folder를 선택하면 새로운 폴더가 “Untitled Folder”라는 이름으로 생성된다.



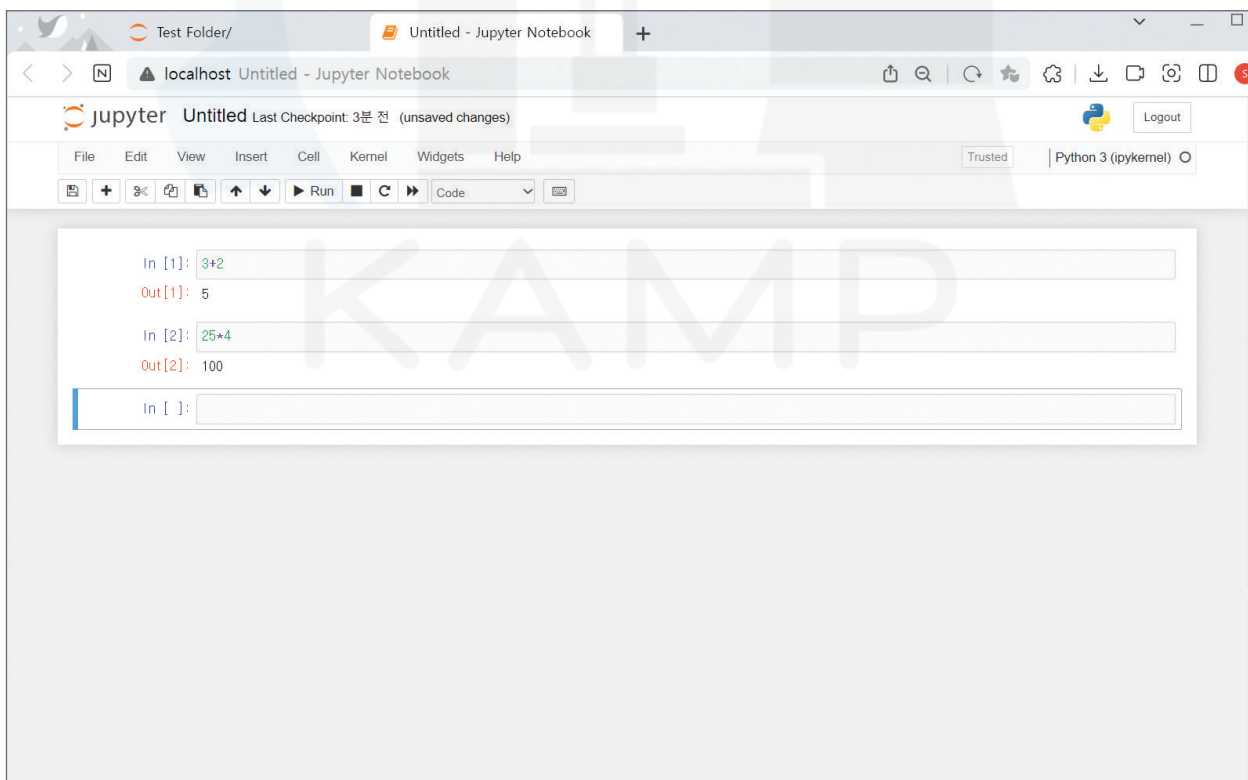
- ⑬ 생성된 “Untitled Folder”를 클릭하여 선택하고 Rename을 클릭하면 폴더의 이름을 지정하는 팝업창이 표시된다. 이 창에서 폴더 이름을 주고 Rename 버튼을 클릭하여 생성된 폴더에 이름을 부여할 수 있다.



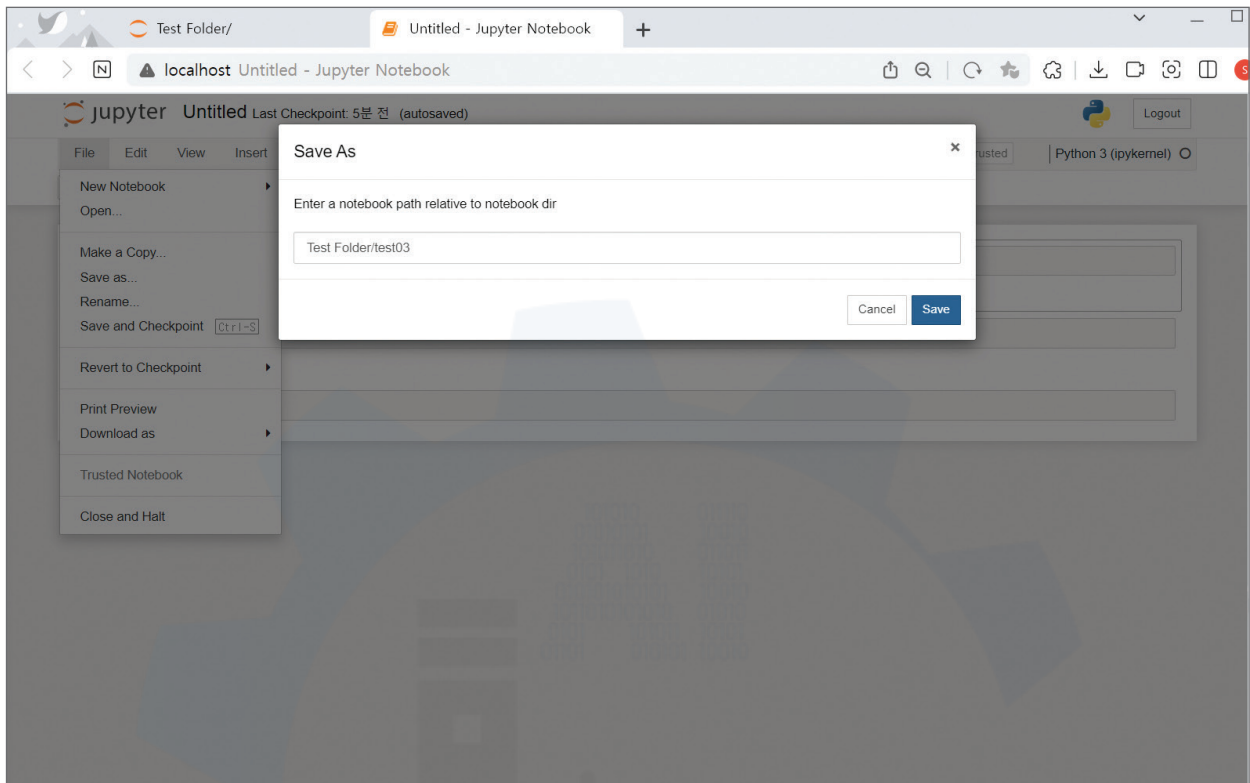
- ⑭ 새로운 파일 생성: 생성된 폴더로 이동하여 화면의 오른쪽에서 New를 클릭하여 Python 3 (ipykernel)을 선택하면 새로운 파일이 “Untitled”라는 이름으로 생성된다.



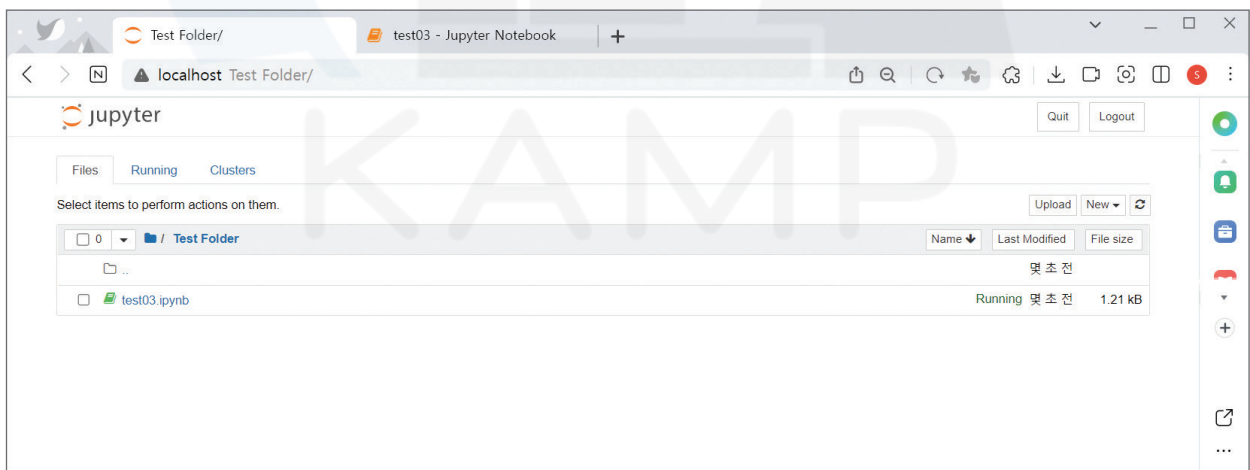
- ⑮ 표시된 화면에서 Python 코드를 작성한다.



- ⑩ 코드를 모두 작성한 후, 파일을 저장하기 위하여 File 메뉴를 클릭하고 “Save As”를 선택하면 파일의 이름을 지정하는 팝업창이 표시된다. 이 창에서 파일 이름을 주고 Save 버튼을 클릭하여 생성된 폴더에 파일을 저장할 수 있다.



- ⑪ 파일을 저장한 후, 다음 화면에서 작업을 마친 파일을 확인할 수 있다. 모든 작업이 완료된 후에는 Jupyter Notebook 화면을 닫고 콘솔에서 Ctrl+c 를 눌러 Anaconda를 종료한다.



4 부록_데이터 품질 전처리 [실습 코드]

KAMP
인공지능 제조 플랫폼

데이터 품질 전처리에 필요한 6가지의 데이터 품질 평가지표 (완전성, 유일성, 유효성, 일관성, 정확성, 무결성)의 산출 방법에 대하여 기술한다.

▶ 품질 전처리를 위한 라이브러리 및 파일을 다음 코드로 불러온 후, 확인한다.

```
import pandas as pd
import numpy as np
from datetime import date
from datetime import time
from datetime import datetime
from dateutil.parser import parse
file_path = 'data/raw_data/train/1000_chg.csv' # the file path is the archive location of the file in our computer
data = pd.read_csv(file_path) # using pandas library (pd) to read the csv file.
data
```

	Date	Time	SerialNumber	Voltage	Current	RSOCmin	RSOCmax	RSOCavg	USOCmin	USOCmax	...	M12T01
0	2020-08-04	15:51:49	1000	641.3	0.0	33.43	34.29	33.84	33	34	...	31.6
1	2020-08-04	15:51:50	1000	641.3	0.0	33.43	34.29	33.84	33	34	...	31.6
2	2020-08-04	15:51:51	1000	641.3	0.0	33.43	34.29	33.84	33	34	...	31.6
3	2020-08-04	15:51:52	1000	641.3	0.0	33.43	34.29	33.84	33	34	...	31.6
4	2020-08-04	15:51:53	1000	641.3	0.0	33.43	34.29	33.84	33	34	...	31.6
...	...	...	...	...	...	...	...	...	...	...	...	...
6004	2020-08-04	17:33:01	1000	714.5	0.0	89.60	90.46	90.01	100	100	...	35.9
6005	2020-08-04	17:33:02	1000	714.5	0.0	89.60	90.46	90.01	100	100	...	35.9
6006	2020-08-04	17:33:03	1000	714.5	0.0	89.60	90.46	90.01	100	100	...	35.9
6007	2020-08-04	17:33:04	1000	714.5	0.0	89.60	90.46	90.01	100	100	...	35.9
6008	2020-08-04	17:33:05	1000	714.5	0.0	89.60	90.46	90.01	100	100	...	35.9

6009 rows x 231 columns

[코드 87] 품질 전처리를 위한 라이브러리 및 파일 불러오기



▶ 품질 전처리를 위하여 주어진 문자열이 date, time 형식인지 체크하는 함수를 정의한다.

```
# Check if a string is a date
def is_date(string, fuzzy=False):
    try:
        parse(string, fuzzy=fuzzy)
        return True
    except ValueError:
        return False

# Check if string is time
def string_is_time(string):
    try:
        datetime.strptime(string, '%H:%M:%S')
        return True
    except ValueError:
        return False
```

[코드 88] 문자열이 date, time 형식인지 체크하는 함수 정의

▶ 완전성 품질지수

$$\text{완전성 품질지수} = \left(1 - \frac{\text{결측데이터의 개수}}{\text{전체 데이터의 개수}}\right) \times 100$$

- ① 총 결측 데이터의 개수를 확인하기 위해 'isnull()' 함수를 사용하여 각 열에서의 결측 데이터 개수를 구한 뒤 'sum()' 함수를 이용하여 합산한다. 다음 함수 getCompleteness()는 완전성 품질지수를 구하는 함수이다.

```
# Function to check completeness of a dataframe
# Returns total of null values and percentage of completeness
def getCompleteness(data):
    total_null_values = data.isnull().sum().sum()
    total_notNull_values = data.count().sum()
    total_values = total_null_values + total_notNull_values
    completeness = (total_notNull_values / total_values)*100
    return total_null_values, completeness

total_null_values, completeness = getCompleteness(data)
print("Total of null values = %d values"%total_null_values)
print("완전성 품질지수 : %.2f%%"%completeness)
```

```
Total of null values = 0 values
완전성 품질지수 : 100.00%
```

[코드 88] 분석도구를 활용한 완전성 품질지수 구하기

- ② 위 함수의 출력인 완전성 품질지수는 100%로 결측 데이터가 존재하지 않음을 알 수 있다.
만일 결측치가 존재할 경우에는 다음의 결측치 처리 함수인 `handleMissingValue()`를 사용하여 결측 데이터를 처리한 후, 다시 함수 `getCompleteness()`를 호출하여 완전성 품질지수를 구한다.

```
# Function for handling missing value
# Input parameter: the dataframe
# Output: dataframe with all missing value already fill in
def handleMissingValue(data):
    df = data.copy()
    numeric_df = df.columns[df.dtypes != 'object']
    categorical_df = df.columns[df.dtypes == 'object']
    #handling missing value for categorical
    for i in categorical_df:
        df[i].fillna(df[i].mode()[0], inplace=True)

    #drop columns that contain only null values
    for col in df.columns:
        if df[col].isnull().all():
            df = df.drop([col], axis=1)
    print(df.columns)
    df_flag_null = df.isnull()
    i, c = np.where(df_flag_null)

    #handling missing value for numerical
    for j in range(len(i)):
        #if missing value in the beginning
        if i[j] == 0:
            s = df.iloc[:, c[j]]
            id_s = s.notna().idxmax()
            val = df.iat[id_s, c[j]]
            df.iat[i[j], c[j]] = val
            #df.iloc[[i[j]], [c[j]]] = df.iloc[[id_s], [c[j]]]

            #print(type(val))
        #if missing value in the end
        elif i[j] == (len(df) - 1):
            print("end!")
            print(i[j], c[j])
            s = df.iloc[:, c[j]]
            id_s = s.notna()[::-1].idxmax() # error is this line
            print(id_s, c[j])
            val = df.iat[id_s, c[j]]
            print(val)
            df.iat[i[j], c[j]] = val
        #if missing value in the middle
        else:
            low = df.iat[i[j] - 1, c[j]]
            high = df.iat[i[j] + 1, c[j]]
            #print(high)
            if (math.isnan(high)):
                val = low
            else:
                val = (low + high) / 2
            df.iat[i[j], c[j]] = val

    return df
```

[코드 89] 결측치 처리 함수 정의



▶ 유일성 품질지수

$$\text{유일성 품질지수} = \left(\frac{\text{유일한 데이터의 개수}}{\text{전체 데이터의 개수}} \right) \times 100$$

- ① 데이터에서 유일한 값을 갖는 열을 찾는다. 시계열 데이터는 주로 Timestamp이다.
- ② 이 테이블에서는 Date 및 Time 칼럼이 고유 식별자로 사용되었으며, Date 및 Time 칼럼에 중복되는 항목이 없으므로 유일성의 값은 100%로 계산된다.
- ③ 다음 함수 getUniqueness()는 유일성을 구하는 함수이다.

```
# Function to get data uniqueness
# takes a series of data as an input
time_series = data["Time"]
def getUniqueness(data):
    total_data = data.count()
    unique = len(data.unique())
    uniqueness = (unique / total_data) * 100
    return uniqueness
print("유일성 품질지수 : ", getUniqueness(time_series), "%")
```

[코드 90] 분석도구를 활용한 유일성 품질지수 구하기

▶ 유효성 품질지수

$$\text{유효성 품질 지수} = \left(\frac{\text{유효성을 만족한 데이터의 개수}}{\text{전체 데이터의 개수}} \right) \times 100$$

- ① 데이터가 주어진 유효범위를 벗어나는지를 확인한다.
- ② 데이터의 유효 범위를 range_min과 range_max 값으로 지정하며, 유효한 범위 내에 속하는 데이터 항목을 count 하여 유효성을 계산한다.
- ③ 다음 함수 getValidity()는 유효성을 구하는 함수이다.



```
# Function to get data validity of a data series
def getSeriesValidity(data, range_min, range_max):
    column_name = data.name
    total_data = data.count()
    if(len(column_name) >= 4 and column_name[0:4] == 'Time'):
        time_conv = pd.to_datetime(data).dt.time
        temp = data.loc[time_conv >= range_min]
        temp = data.loc[time_conv <= range_max]
        in_range_data = temp.count()
    elif(len(column_name) >= 4 and column_name[0:4] == 'Date'):
        date_conv = pd.to_datetime(data).dt.date
        temp = data.loc[date_conv >= range_min]
        temp = data.loc[date_conv <= range_max]
        in_range_data = temp.count()
    else:
        in_range_data = data.loc[(data >= range_min) & (data <= range_max)].count()
    validity = in_range_data / total_data * 100
    return validity

def getValidity(data, ranges):
    validities = []
    for col in ranges:
        current_validity = getSeriesValidity(data[col], ranges[col][0], ranges[col][1])
        #print(col + " validity is: %.2f%%" % current_validity)
        validities.append(current_validity)
    average_validity = sum(validities) / len(validities)
    return average_validity

# Calculate the validity of all columns.
columns = data.columns
range_dictionary = {}
default_range = (-200, 200)
# If there is a special range to be checked, please add in this list.
# In format: "Name_of_Column": (range_min, range_max)
# For example,
set_range = {
    "Voltage": (0, 1000),
    "Date": (date(1999, 1, 1), date(2023, 12, 12)),
    "Time": (time(0, 0), time(23, 59)),
    "SerialNumber": (0, 1000)
}

# date format: date(year, month, day)
# time format: time(hour, minutes)
# Columns with Date and Time must be or start with the words Date and Time, respectively.
for column_name in columns:
    if column_name in set_range:
        range_dictionary[column_name] = set_range[column_name]
    else:
        range_dictionary[column_name] = default_range

print("유효성 품질지수 : %.2f%%" % getValidity(data, range_dictionary))
```

[코드 91] 분석도구를 활용한 유효성 품질 지수 구하기



▶ 일관성 품질지수

$$\text{일관성 품질지수} = \left(\frac{\text{일관성을 만족한 데이터의 개수}}{\text{전체 데이터의 개수}} \right) \times 100$$

- ① 데이터의 타입이 정해진 형식에 맞는지 확인한다.
- ② 전체 데이터의 개수와 형식에 맞는 데이터의 개수를 구하여 일관성 품질지수를 계산한다.
- ③ 다음 함수 getConsistency()는 일관성을 구하는 함수이다.

```
def getSeriesConsistency(data, data_type):
    total_data = data.count()
    #print(data_type)
    if(data.name == 'Date'):
        types = list(map(lambda item: is_date(item), data))
    elif(data.name == 'Time'):
        types = list(map(lambda item: string_is_time(item), data))
    else:
        types = list(map(lambda item: isinstance(item, data_type), data))
    types = pd.Series(types)
    consistent_data = types.sum()
    consistency = (consistent_data / total_data) * 100
    return consistency

def getConsistency(data, types):
    consistent_data = []
    for col in types:
        current_const = getSeriesConsistency(data[col], types[col])
        #print(col + " consistency is: %.2f%%" % current_const)
        consistent_data.append(current_const)
    average_const = sum(consistent_data) / len(consistent_data)
    return average_const

# We will calculate the consistency of all columns
columns = data.columns
const_dictionary = {}
default_type = (float, int)
numeric_type = (float, int)
# If there is a special type to be checked, please add in this list.
# In format: "Name_of_Column": (range_min, range_max)
# For example,
set_types = {
    "Voltage": numeric_type,
    "Current": numeric_type,
    "Power": numeric_type,
}
# Time and Date types will be set automatically
# Columns names with Date and Time must be or start with the words Date and Time, respectively.
for column_name in columns:
    if column_name in set_types:
        const_dictionary[column_name] = set_types[column_name]
    else:
        const_dictionary[column_name] = default_type
print("일관성 품질지수 : %.2f%%" % getConsistency(data, const_dictionary))
```

[코드 92] 분석도구를 활용한 일관성 품질지수 구하기



▶ 정확성 품질지수

$$\text{정확성 품질지수} = 1 - \left(\frac{\text{정확성에 위배되는 데이터의 개수}}{\text{전체 데이터의 개수}} \right) \times 100$$

- ① 데이터 항목에 대하여 검증된 정확한 정보(참조 데이터)가 있는지를 확인하고, 참조 데이터가 있는 경우, 데이터 항목 값과 참조 데이터의 값을 비교한다.
- ② 여기에서 배터리 데이터는 참조 데이터가 존재하지 않으므로 정확성을 판단하지 않는다.

▶ 무결성 품질지수

$$\text{무결성 품질지수} = \left\{ 1 - \left(\frac{\text{유일성, 유효성, 일관성 중 100%가 아닌 지수}}{3} \right) \right\} \times 100$$

- 무결성 품질지수는 앞에서 구한 ‘유일성’, ‘유효성’, ‘일관성’ 지수를 기초로 위의 식으로 계산된다. 이 지수들이 모두 100%이므로 무결성 품질지수도 100%가 된다.

▶ 가중치 지수

$$\text{가중치지수} = \text{품질지수} \times \text{가중치}$$

- 데이터 및 적용 업무의 특성에 따라 품질지수의 중요도가 달라지며, 이에 따라 적절한 가중치 지수를 부여한다.

▶ 데이터 품질지수

$$\text{데이터 품질지수} = \sum \text{가중치지수}$$

- ① 앞에서 구한 6가지의 데이터 품질지수에 적절한 가중치를 반영하여 최종적으로 데이터 품질지수를 구한다. 다음 테이블과 같이 최종 품질지수를 산출하면 100%가 된다.

항목	품질지수	가중치	가중치지수	오류율
완전성	100%	20%	20%	0%
유일성	100%	20%	20%	0%
유효성	100%	20%	20%	0%
일관성	100%	20%	20%	0%
정확성	-	-	-	-
무결성	100%	20%	20%	0%
최종 품질지수	100%	100%	100%	0%

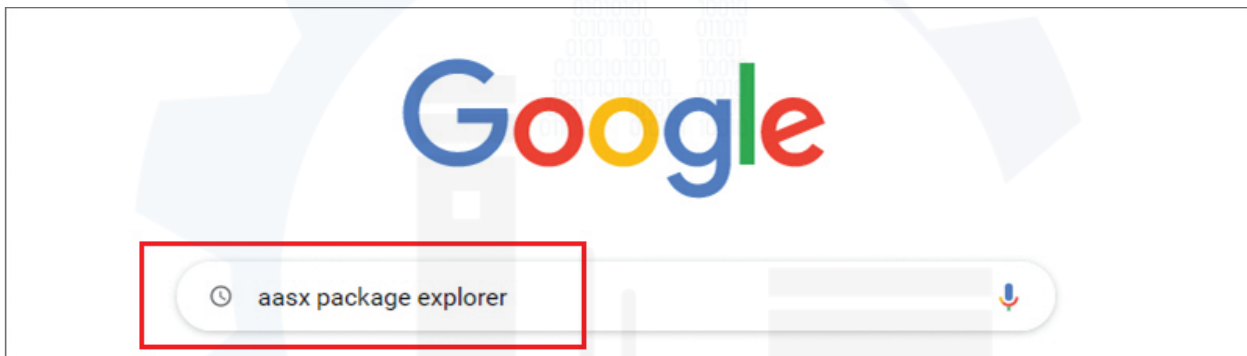
데이터 품질 지수 산출

5 부록_AAS S/W 설치 가이드

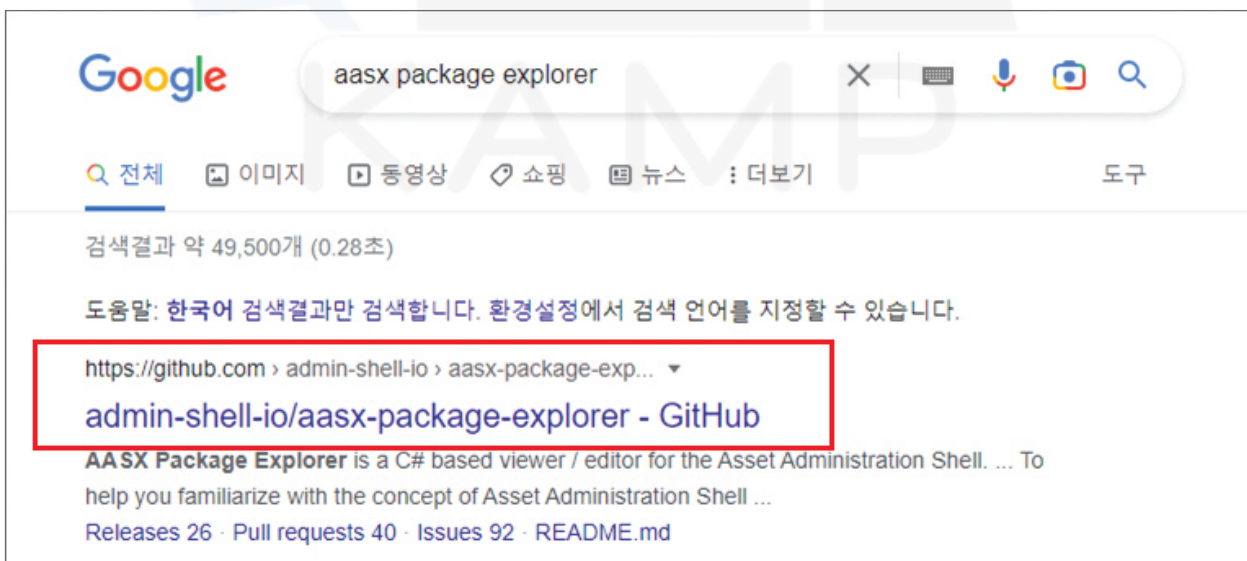
1. AASX Package Explorer 설치

AAS 모델은 독일 IDTA(Industrial Digital Twin Association)에서 제공하는 C# 기반 오픈 소스 소프트웨어 ‘AASX Package Explorer’를 사용하여 개발할 수 있다. 특히, AASX Package Explorer는 윈도우즈 10 및 윈도우즈 11 운영체제에서 실행 가능하고 GUI 또한 제공하기 때문에 처음 접하는 사용자도 어렵지 않게 AAS 모델을 개발 할 수 있다. 본 부록에서는 AAS 모델링에 필요한 소프트웨어 패키지 다운로드 방법과 AAS 파일 오픈 및 편집하기 위해 기본적인 메뉴 설명에 대해 안내한다.

① google 등의 검색 엔진에 ‘aasx package explorer’를 검색한다.



② 아래와 그림과 같이 ‘admin-shell-io/aasx-package-explorer GitHub’ 링크를 선택한다.



- ③ 페이지 하단의 Installation의 'the releases' 링크를 클릭한다.

Installation

We provide the binaries for Windows 10 in [the releases](#).

(Remark: In special cases you may like to use a current build. Please click on a green check mark and select "Check-release" details.)

- ④ aasx-package-explorer.날짜명.zip 이름의 파일을 다운로드 받는다. 단, 본 가이드북에서는 당시 최신 버전인 'aasx-package-explorer.2022-08-06.zip'를 사용

AASX Package Explorer 2022-08-06.alpha Latest

Add first version of signature ([https://github.com/admin-shell-io/aasx-package-explorer/pull/501\[\]](https://github.com/admin-shell-io/aasx-package-explorer/pull/501[])) (<https://github.com/admin-shell-io/aasx-package-explorer/commit/dc5984429699dc50a51f982a7ae273d54df73624>)

Signing and validating of submodels or collections

Prototype according actual work of Task Force Secure AAS

Click on submodel or collection and choose "File / Security / Sign or Validate"

Multiple signatures with different X509 certificates are possible

[Update test certificate](#)

▼ Assets 5

aasx-package-explorer-blazorui.2022-08-06.zip	3.63 MB	06 Aug 2022
aasx-package-explorer-small.2022-08-06.zip	36.9 MB	06 Aug 2022
aasx-package-explorer.2022-08-06.zip	115 MB	06 Aug 2022
Source code (zip)		05 Aug 2022
Source code (tar.gz)		05 Aug 2022

- ⑤ 다운로드한 'aasx-package-explorer.2022-08-06.zip' 파일의 압축을 해제한다.

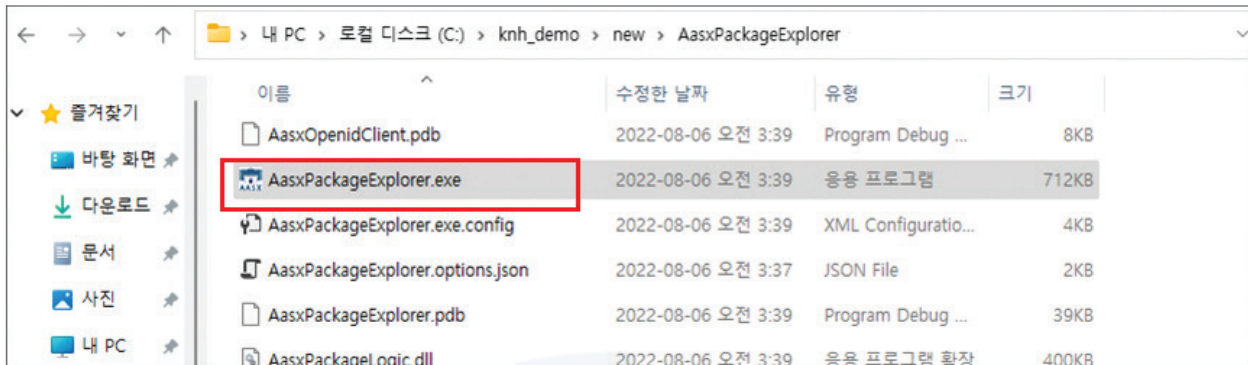
new

새로 만들기 | | | | | | | 정렬 | 보기 | ...

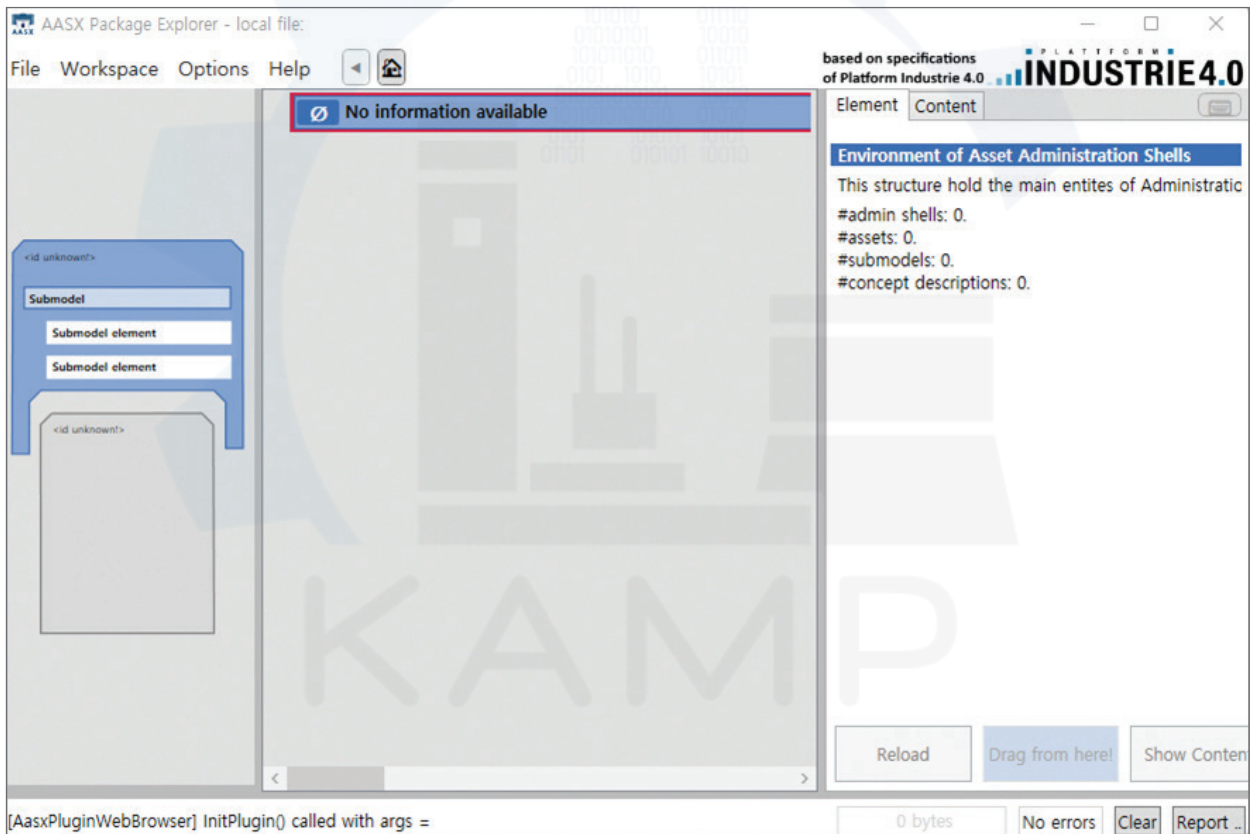
← → ^ ↑ > 내 PC > 로컬 디스크 (C:) > knh_demo > new

이름	수정한 날짜	유형	크기
AasxPackageExplorer	2022-10-27 오후 1:15	파일 폴더	
aasx-package-explorer.2022-08-06.zip	2022-10-27 오후 1:13	압축(ZIP) 파일	117,850KB

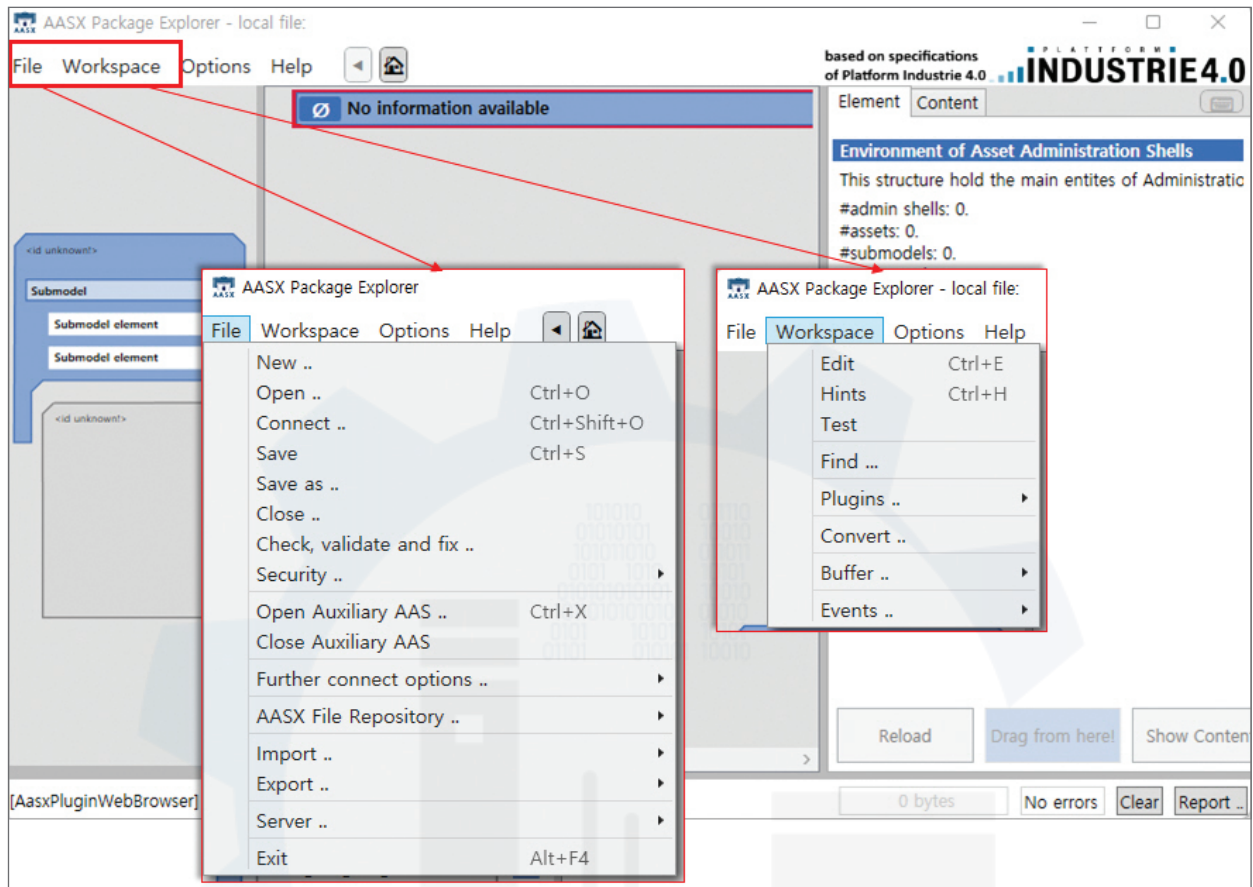
- ⑥ 디렉토리 'AasxPackageExplorer'을 열어서 실행파일인 'AasxPackageExplorer.exe'를 클릭하여 프로그램을 실행한다.



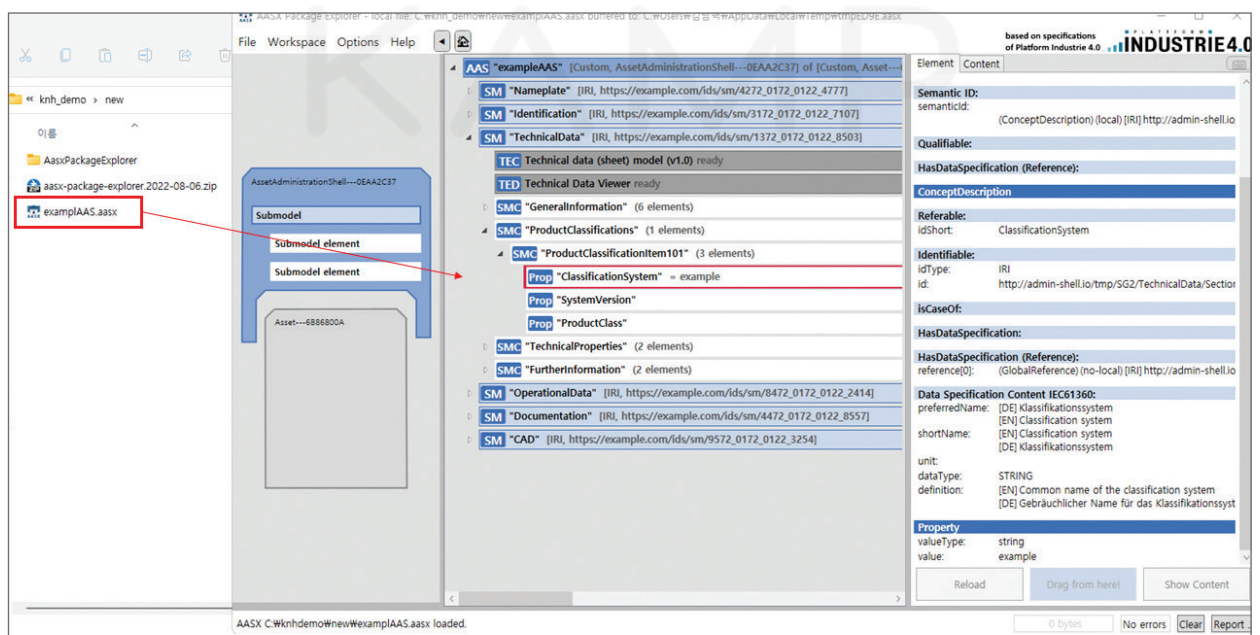
- ⑦ 아래와 같이 실행화면이 나타나면 AASX Package Explorer가 정상적으로 잘 동작하는 것이므로 AAS 개발 환경이 준비되었음을 의미한다.



- ⑧ AASX Package Explorer에서 AAS 생성/편집/뷰 기능은 아래와 같이 'File'과 'Workspace' 메뉴를 통해 사용 가능하다. 기본 확장자는 'aasx'이며, 필요한 경우 'Save as' 또는 'Export' 기능을 통해 JSON이나 XML 확장자로 변환하여 사용 할 수 있다.



- ⑨ aasx 파일은 AASX Package Explorer의 File 메뉴에서 'Open'을 클릭하거나 화면에 aasx 파일을 드래그앤 드롭하여 작성된 AAS 모델을 뷰/편집 할 수 있다.





「전자부품(배터리팩) 품질보증 AI 데이터셋」 분석실습 가이드북